

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent	12418647
Kind Code	B2
Date of Patent	September 16, 2025
Inventor(s)	Zhu; Weijia et al.

Using quantization groups in video coding

Abstract

An example method of video processing includes applying, in a conversion between a video comprising multiple components and a bitstream representation of the video, a deblocking filter to video blocks of the multiple components. A deblocking filter strength for the deblocking filter of each of the multiple components is determined according to a rule that specifies to use a different manner for determining the deblocking filter strength for the video blocks of each of the multiple components.

Inventors: Zhu; Weijia (San Diego, CA), Zhang; Li (San Diego, CA), Xu; Jizheng (San Diego, CA), Zhang; Kai (San Diego, CA), Wang; Ye-kui (San Diego, CA)

Applicant: Bytedance Inc. (Los Angeles, CA)

Family ID: 1000008818054

Assignee: Bytedance Inc. (Los Angeles, CA)

Appl. No.: 17/835647

Filed: June 08, 2022

Prior Publication Data

Document Identifier	Publication Date
US 20220321882 A1	Oct. 06, 2022

Foreign Application Priority Data

WO	PCT/CN2019/123951	Dec. 09, 2019
WO	PCT/CN2019/130851	Dec. 31, 2019

Related U.S. Application Data

continuation parent-doc WO PCT/US2020/063746 20201208 PENDING child-doc US 17835647

Publication Classification

Int. Cl.: H04N19/176 (20140101); H04N19/117 (20140101); H04N19/174 (20140101); H04N19/186 (20140101); H04N19/70 (20140101); H04N19/80 (20140101)

U.S. Cl.:

CPC H04N19/117 (20141101); H04N19/174 (20141101); H04N19/176 (20141101); H04N19/186 (20141101); H04N19/70 (20141101); H04N19/80 (20141101);

Field of Classification Search

CPC: H04N (19/117); H04N (19/174); H04N (19/176); H04N (19/186); H04N (19/70); H04N (19/80)

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
5512953	12/1995	Nahumi	N/A	N/A
6016163	12/1999	Rodriguez et al.	N/A	N/A
6833993	12/2003	Wang et al.	N/A	N/A
7474357	12/2008	Mrudock et al.	N/A	N/A
7843414	12/2009	Hsu et al.	N/A	N/A
D661490	12/2011	Jin et al.	N/A	N/A
8211570	12/2011	He et al.	N/A	N/A

8252463	12/2011	He et al.	N/A	N/A
8306124	12/2011	Gao et al.	N/A	N/A
8817208	12/2013	Li et al.	N/A	N/A
9137547	12/2014	Van et al.	N/A	N/A
9229877	12/2015	Bivens et al.	N/A	N/A
9538200	12/2016	Van et al.	N/A	N/A
9591302	12/2016	Sullivan	N/A	N/A
9723331	12/2016	Van et al.	N/A	N/A
9734026	12/2016	Hack et al.	N/A	N/A
9807403	12/2016	Chong et al.	N/A	N/A
10057574	12/2017	Li et al.	N/A	N/A
10085024	12/2017	Yu et al.	N/A	N/A
10095698	12/2017	Cullen et al.	N/A	N/A
10097832	12/2017	Sullivan	N/A	N/A
10319100	12/2018	Dai et al.	N/A	N/A
10321130	12/2018	Dong et al.	N/A	N/A
10506230	12/2018	Zhang et al.	N/A	N/A
10582213	12/2019	Li et al.	N/A	N/A
10708591	12/2019	Zhang et al.	N/A	N/A
10708592	12/2019	Dong et al.	N/A	N/A
10778974	12/2019	Karczewicz et al.	N/A	N/A
10855985	12/2019	Zhang et al.	N/A	N/A
11184643	12/2020	Kotra et al.	N/A	N/A
11539956	12/2021	Li et al.	N/A	N/A
11622120	12/2022	Zhu et al.	N/A	N/A
11750806	12/2022	Zhu et al.	N/A	N/A
11785260	12/2022	Zhu et al.	N/A	N/A
11863715	12/2023	Zhu et al.	N/A	N/A
11902518	12/2023	Zhu	N/A	N/A
11973959	12/2023	Zhu et al.	N/A	N/A
11985329	12/2023	Zhu et al.	N/A	N/A
11991390	12/2023	Yoo et al.	N/A	N/A
2002/0106025	12/2001	Tsukagoshi et al.	N/A	N/A
2003/0206585	12/2002	Kerofsky	N/A	N/A
2004/0062313	12/2003	Schoenblum	N/A	N/A
2004/0101059	12/2003	Joch et al.	N/A	N/A
2004/0146108	12/2003	Hsia	N/A	N/A
2007/0098799	12/2006	Zhang et al.	N/A	N/A
2007/0162591	12/2006	Mo et al.	N/A	N/A
2007/0201564	12/2006	Joch et al.	N/A	N/A
2009/0262798	12/2008	Chiu et al.	N/A	N/A
2009/0289320	12/2008	Cohen	N/A	N/A
2012/0183052	12/2011	Lou et al.	N/A	N/A
2012/0192051	12/2011	Rothschiller et al.	N/A	N/A
2013/0016772	12/2012	Matsunobu et al.	N/A	N/A
2013/0022107	12/2012	Van Der Auwera et al.	N/A	N/A
2013/0077676	12/2012	Sato	N/A	N/A
2013/0101018	12/2012	Chong et al.	N/A	N/A
2013/0101025	12/2012	Van Der Auwera et al.	N/A	N/A
2013/0101031	12/2012	Van Der Auwera et al.	N/A	N/A
2013/0107973	12/2012	Wang et al.	N/A	N/A
2013/0188733	12/2012	Van Der Auwera et al.	N/A	N/A
2013/0259141	12/2012	Van Der Auwera et al.	N/A	N/A
2013/0329785	12/2012	Lim et al.	N/A	N/A
2014/0003497	12/2013	Sullivan et al.	N/A	N/A
2014/0003498	12/2013	Sullivan	N/A	N/A
2014/0192904	12/2013	Rosewarne	N/A	N/A
2014/0321552	12/2013	He et al.	N/A	N/A
2014/0355689	12/2013	Tourapis	N/A	N/A
2014/0362917	12/2013	Joshi et al.	N/A	N/A
2014/0376611	12/2013	Kim et al.	N/A	N/A
2015/0003518	12/2014	Nguyen et al.	N/A	N/A
2015/0016506	12/2014	Fu et al.	N/A	N/A
2015/0071345	12/2014	Tourapis et al.	N/A	N/A
2015/0181976	12/2014	Cooper et al.	N/A	N/A
2015/0264354	12/2014	Zhang et al.	N/A	N/A
2015/0264374	12/2014	Xiu et al.	N/A	N/A
2015/0264402	12/2014	Zhang et al.	N/A	N/A
2015/0319438	12/2014	Shima et al.	N/A	N/A
2015/0358631	12/2014	Zhang et al.	N/A	N/A
2015/0365671	12/2014	Pu et al.	N/A	N/A
2015/0365695	12/2014	Pu et al.	N/A	N/A
2015/0373327	12/2014	Zhang et al.	N/A	N/A
2016/0065991	12/2015	Chen et al.	N/A	N/A
2016/0100167	12/2015	Rapaka et al.	N/A	N/A

2016/0100175	12/2015	Laroche et al.	N/A	N/A
2016/0212373	12/2015	Aharon et al.	N/A	N/A
2016/0227224	12/2015	Hsieh et al.	N/A	N/A
2016/0241853	12/2015	Lim et al.	N/A	N/A
2016/0241858	12/2015	Li et al.	N/A	N/A
2016/0261864	12/2015	Samuelsson et al.	N/A	N/A
2016/0261884	12/2015	Li	N/A	H04N 19/157
2016/0286226	12/2015	Ridge et al.	N/A	N/A
2016/0286235	12/2015	Yamamoto et al.	N/A	N/A
2016/0337661	12/2015	Pang et al.	N/A	N/A
2017/0034532	12/2016	Yamamoto et al.	N/A	N/A
2017/0070754	12/2016	Lim et al.	N/A	N/A
2017/0105014	12/2016	Lee et al.	N/A	N/A
2017/0127077	12/2016	Chuang et al.	N/A	N/A
2017/0127090	12/2016	Rosewarne et al.	N/A	N/A
2017/0134728	12/2016	Sullivan	N/A	N/A
2017/0150151	12/2016	Samuelsson et al.	N/A	N/A
2017/0150157	12/2016	Yamori	N/A	N/A
2017/0238020	12/2016	Karczewicz et al.	N/A	N/A
2017/0310969	12/2016	Chen et al.	N/A	N/A
2017/0332075	12/2016	Karczewicz et al.	N/A	N/A
2018/0027246	12/2017	Liu et al.	N/A	N/A
2018/0041778	12/2017	Zhang et al.	N/A	N/A
2018/0041779	12/2017	Zhang et al.	N/A	N/A
2018/0048901	12/2017	Zhang et al.	N/A	N/A
2018/0063527	12/2017	Chen et al.	N/A	N/A
2018/0084284	12/2017	Rosewarne et al.	N/A	N/A
2018/0091812	12/2017	Guo et al.	N/A	N/A
2018/0091829	12/2017	Liu et al.	N/A	N/A
2018/0109794	12/2017	Wang et al.	N/A	N/A
2018/0115787	12/2017	Koo et al.	N/A	N/A
2018/0160112	12/2017	Gamei et al.	N/A	N/A
2018/0199057	12/2017	Chuang et al.	N/A	N/A
2018/0278934	12/2017	Andersson et al.	N/A	N/A
2018/0338161	12/2017	Zhai et al.	N/A	N/A
2018/0352264	12/2017	Guo et al.	N/A	N/A
2018/0373710	12/2017	Cullen et al.	N/A	N/A
2019/0020875	12/2018	Liu et al.	N/A	N/A
2019/0089984	12/2018	He	N/A	N/A
2019/0098307	12/2018	Sullivan	N/A	N/A
2019/0116358	12/2018	Zhang	N/A	H04N 19/117
2019/0124330	12/2018	Chien et al.	N/A	N/A
2019/0158831	12/2018	Jung et al.	N/A	N/A
2019/0208206	12/2018	Nakagami	N/A	N/A
2019/0230353	12/2018	Gadde et al.	N/A	N/A
2019/0238845	12/2018	Zhang et al.	N/A	N/A
2019/0238849	12/2018	Fang et al.	N/A	N/A
2019/0273923	12/2018	Huang et al.	N/A	N/A
2019/0289320	12/2018	Chuang et al.	N/A	N/A
2019/0306502	12/2018	Gadde et al.	N/A	N/A
2020/0029072	12/2019	Xu et al.	N/A	N/A
2020/0177910	12/2019	Li et al.	N/A	N/A
2020/0213570	12/2019	Shih et al.	N/A	N/A
2020/0236381	12/2019	Chujoh et al.	N/A	N/A
2020/0267381	12/2019	Vanam et al.	N/A	N/A
2020/0329239	12/2019	Hsiao et al.	N/A	N/A
2020/0396467	12/2019	Lai et al.	N/A	N/A
2020/0413038	12/2019	Zhang et al.	N/A	N/A
2021/0006792	12/2020	Han et al.	N/A	N/A
2021/0021841	12/2020	Xu et al.	N/A	N/A
2021/0021863	12/2020	Kalva	N/A	H04N 19/543
2021/0044820	12/2020	Furht et al.	N/A	N/A
2021/0051320	12/2020	Tourapis et al.	N/A	N/A
2021/0076032	12/2020	Hu et al.	N/A	N/A
2021/0084343	12/2020	Ramasubramonian et al.	N/A	N/A
2021/0099727	12/2020	Deng et al.	N/A	N/A
2021/0099732	12/2020	Ray et al.	N/A	N/A
2021/0120239	12/2020	Zhu et al.	N/A	N/A
2021/0160479	12/2020	Hsiang	N/A	N/A
2021/0176464	12/2020	Ray et al.	N/A	N/A
2021/0176479	12/2020	Liao et al.	N/A	N/A
2021/0195201	12/2020	Li et al.	N/A	N/A
2021/0211700	12/2020	Li et al.	N/A	N/A
2021/0266550	12/2020	Li et al.	N/A	N/A
2021/0266552	12/2020	Kotra et al.	N/A	N/A

2021/0266556	12/2020	Choi et al.	N/A	N/A
2021/0314628	12/2020	Zhang et al.	N/A	N/A
2021/0321095	12/2020	Zhang et al.	N/A	N/A
2021/0321121	12/2020	Zhang et al.	N/A	N/A
2021/0337239	12/2020	Zhang et al.	N/A	N/A
2021/0344903	12/2020	Yu et al.	N/A	N/A
2021/0368171	12/2020	Zhang et al.	N/A	N/A
2021/0377524	12/2020	Zhang. et al.	N/A	N/A
2021/0385446	12/2020	Liu et al.	N/A	N/A
2021/0385454	12/2020	Fleureau et al.	N/A	N/A
2021/0392381	12/2020	Wang et al.	N/A	N/A
2021/0409701	12/2020	Zhu et al.	N/A	N/A
2022/0014782	12/2021	Chon et al.	N/A	N/A
2022/0141495	12/2021	Kim et al.	N/A	N/A
2022/0159282	12/2021	Sim et al.	N/A	N/A
2022/0182641	12/2021	Nam et al.	N/A	N/A
2022/0191496	12/2021	Kotra et al.	N/A	N/A
2022/0201294	12/2021	Nam et al.	N/A	N/A
2022/0210408	12/2021	Zhu et al.	N/A	N/A
2022/0210433	12/2021	Zhu et al.	N/A	N/A
2022/0210448	12/2021	Zhu et al.	N/A	N/A
2022/0217410	12/2021	Wang	N/A	N/A
2022/0264122	12/2021	Zhu et al.	N/A	N/A
2022/0272347	12/2021	Zhu et al.	N/A	N/A
2022/0295091	12/2021	Chujoh et al.	N/A	N/A
2022/0303567	12/2021	Jung et al.	N/A	N/A
2022/0312042	12/2021	Deshpande	N/A	N/A
2022/0321916	12/2021	Zhu et al.	N/A	N/A
2022/0329794	12/2021	Kotra et al.	N/A	N/A
2022/0337836	12/2021	Zhao et al.	N/A	N/A
2022/0345697	12/2021	Choi et al.	N/A	N/A
2022/0345726	12/2021	Zhao et al.	N/A	N/A
2022/0368898	12/2021	Zhu et al.	N/A	N/A
2023/0007255	12/2022	Tsukuba	N/A	N/A
2023/0018055	12/2022	Hendry et al.	N/A	N/A
2023/0130131	12/2022	Chiang et al.	N/A	N/A

FOREIGN PATENT DOCUMENTS

Patent No.	Application Date	Country	CPC
104205836	12/2013	CN	N/A
104221378	12/2013	CN	N/A
104584559	12/2014	CN	N/A
104584560	12/2014	CN	N/A
105960802	12/2015	CN	N/A
105979271	12/2015	CN	N/A
106105202	12/2015	CN	N/A
106105203	12/2015	CN	N/A
106105205	12/2015	CN	N/A
106416249	12/2016	CN	N/A
106797465	12/2016	CN	N/A
107079150	12/2016	CN	N/A
107079157	12/2016	CN	N/A
107211122	12/2016	CN	N/A
107409215	12/2016	CN	N/A
107431826	12/2016	CN	N/A
107534782	12/2017	CN	N/A
107534783	12/2017	CN	N/A
107846591	12/2017	CN	N/A
108293124	12/2017	CN	N/A
110121884	12/2018	CN	N/A
114946180	12/2023	CN	N/A
3425911	12/2018	EP	N/A
3507984	12/2018	EP	N/A
201217444 D	12/2011	GB	N/A
2506852	12/2013	GB	N/A
2506852	12/2014	GB	N/A
201810671 D	12/2017	GB	N/A
2575090	12/2019	GB	N/A
2015512600	12/2014	JP	N/A
2019525679	12/2018	JP	N/A
2020017970	12/2019	JP	N/A
2022542851	12/2021	JP	N/A
2022544164	12/2021	JP	N/A
2023011955	12/2022	JP	N/A
2023500644	12/2022	JP	N/A

7508558	12/2023	JP	N/A
7622153	12/2024	JP	N/A
20150003246	12/2014	KR	N/A
2636103	12/2016	RU	N/A
201424378	12/2013	TW	N/A
2008016219	12/2007	WO	N/A
2012117744	12/2011	WO	N/A
2012177202	12/2011	WO	N/A
2014008212	12/2013	WO	N/A
2014039547	12/2013	WO	N/A
2015015058	12/2014	WO	N/A
2015187978	12/2014	WO	N/A
2016040865	12/2015	WO	N/A
2016048186	12/2015	WO	N/A
2016049894	12/2015	WO	N/A
2016057665	12/2015	WO	N/A
2016123232	12/2015	WO	N/A
2017052440	12/2016	WO	N/A
2018234716	12/2017	WO	N/A
2018237146	12/2017	WO	N/A
2019009776	12/2018	WO	N/A
2019069950	12/2018	WO	N/A
2020243206	12/2019	WO	N/A

OTHER PUBLICATIONS

Final Office Action from U.S. Appl. No. 17/694,305 dated Sep. 30, 2022. cited by applicant

Non Final Office Action from U.S. Appl. No. 17/720,582 dated Nov. 14, 2022. cited by applicant

Abdoli et al. "Intra Block-DPCM with Layer Separation of Screen Content in WC," 2019 IEEE International Conference on Image Processing (ICIP), IEEE, 2019, Sep. 25, 2019, retrieved on Feb. 21, 2021 from <<https://ieeexplore.ieee.org/abstract/document/8803389>>. cited by applicant

Bross et al. "Versatile Video Coding (Draft 6)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 15th Meeting: Gothenburg, SE, Jul. 3-12, 2019, document JVET-O2001, 2019. cited by applicant

Bross et al. "Versatile Video Coding (Draft 7)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 16th Meeting: Geneva, CH, Oct. 1-11, 2019, document JVET-P2001, 2019. cited by applicant

Chen et al. "Algorithm Description of Joint Exploration Test Model 7 (JEM 7)," Joint Video Exploration Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 7th Meeting: Torino, IT, Jul. 13-21, 2017, document JVET-G1001, 2017. cited by applicant

Gao et al. "AVS—The Chinese Next-Generation Video Coding Standard," National Association Broadcasters, 2004, retrieved on Dec. 16, 2020 from <<http://www.avs.org.cn/reference/AVS%20NAB%20Paper%20Final03.pdf>>. cited by applicant

Gordon et al. "Mismatch on BDPCM Luma/Chroma Context Indices Between VTM7 and Spec," in Fraunhofer.de [online] Nov. 14, 2019, retrieved Mar. 1, 2021, retrieved from the internet <URL: <https://jvet.hhi.fraunhofer.de/trac/vvc/ticket/708>>. cited by applicant

"High Efficiency Video Coding," Series H: Audiovisual and Multimedia Systems: Infrastructure of Audiovisual Services—Coding of Moving Video, ITU-T Telecommunication Standardization Sector of ITU, H.265, Feb. 2018. cited by applicant

Marpe et al. "An Adaptive Color Transform Approach and its Application in 4:4:4 Video Coding," 2006 14th European Signal Processing Conference, IEEE, 2006, September 8, 2006, retrieved on Feb. 21, 2021 from <<https://ieeexplore.ieee.org/abstract/document/7071266>>. cited by applicant

Misra et al. "On Cross Component Adaptive Loop Filter for Video Compression," 2019 Picture Coding Symposium (PCS), Nov. 12-15, 2019, Ningbo, China, retrieved from the internet ?URL:<https://ieeexplore.ieee.org/document/8954547>>. cited by applicant

Misra et al. "CE5 Common Base: Cross Component Adaptive Loop Filter," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, document JVET-Q0058, 2020. cited by applicant

Norkin et al. "HEVC Deblocking Filter," IEEE Transactions on Circuits and Systems for Video Technology, Dec. 2012, 22(12):1746-1754. cited by applicant

Ramasubramonian et al. "AHG15: On Signalling of Chroma QP Tables," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 15th Meeting: Gothenburg, SE, Jul. 3-12, 2019, document JVET-O0650, 2019. cited by applicant

Rosewarne et al. "High Efficiency Video Coding (HEVC) Test Model 16 (HM 16) Improved Encoder Description Update 7," Joint Collaborative Team on Video Coding (JCT-VC) of ITU-T SG16 WP3 and ISO/IEC JTC1/SC29/WG1125th Meeting: Chengdu, CN, Oct. 14-21, 2016, document JCTVC-Y1002, 2016. cited by applicant

Sullivan et al. "Standardized Extensions of High Efficiency Video Coding (HEVC)," IEEE Journal of Selected Topics in Signal Processing, Dec. 2013, 7(6):1001-1016. cited by applicant

https://vcgit.hhi.fraunhofer.de/jvet/VVCSoftware_VTM/tags/VTM-6.0. cited by applicant

JEM-7.0: https://jvet.hhi.fraunhofer.de/svn/svn_HMJEMSoftware/tags/HM-16.6-JEM-. cited by applicant

International Search Report and Written Opinion from International Patent Application No. PCT/US2020/050638 dated Nov. 30, 2020 (8 pages). cited by applicant

International Search Report and Written Opinion from International Patent Application No. PCT/US2020/050644 dated Dec. 18, 2020 (9 pages). cited by applicant

International Search Report and Written Opinion from International Patent Application No. PCT/US2020/050649 dated Feb. 12, 2021 (11 pages). cited by applicant

International Search Report and Written Opinion from International Patent Application No. PCT/US2020/051689 dated Dec. 17, 2020 (9 pages). cited by applicant

International Search Report and Written Opinion from International Patent Application No. PCT/US2020/054959 dated Feb. 12, 2021 (11 pages). cited by applicant

International Search Report and Written Opinion from International Patent Application No. PCT/US2020/055329 dated Jan. 19, 2021 (10 pages).=. cited by applicant

International Search Report and Written Opinion from International Patent Application No. PCT/US2020/055332 dated Jan. 29, 2021 (9 pages). cited by applicant

International Search Report and Written Opinion from International Patent Application No. PCT/US2020/063746 dated Apr. 8, 2021 (14 pages). cited by applicant

International Search Report and Written Opinion from International Patent Application No. PCT/US2020/067264 dated Mar. 23, 2021 (11 pages). cited by applicant

International Search Report and Written Opinion from International Patent Application No. PCT/US2020/067651 dated Mar. 30, 2021 (10 pages). cited by applicant

International Search Report and Written Opinion from International Patent Application No. PCT/US2020/067655 dated Mar. 30, 2021 (10 pages). cited by applicant

Non Final Office Action from U.S. Appl. No. 17/694,305 dated Jun. 9, 2022. cited by applicant

Non Final Office Action from U.S. Appl. No. 17/694,253 dated Jul. 12, 2022. cited by applicant

Non Final Office Action from U.S. Appl. No. 17/720,634 dated Jul. 19, 2022. cited by applicant

Non Final Office Action from U.S. Appl. No. 17/716,447 dated Aug. 4, 2022. cited by applicant

Examination Report from Indian Patent Application No. 202247013800 dated Jul. 21, 2022 (7 pages). cited by applicant

Chen et al. "Algorithm Description for Versatile Video Coding and Test Model 6 (VTM 6)," Joint Video Experts Team (JVET) of ITU-T SG WP 3 and SI/IEC JTC 1/SC 29/WG 11 15th Meeting, Gothenburg, SE, Jul. 3-12, 2019, document JVET-O2002, 2019. cited by applicant

Han et al. "Cu Level Chroma QP Control for WC," Joint Video Experts Team (JVET) of ITU-T SG WP 3 and SI/IEC JTC 1/SC 29/WG 11 15th Meeting, Gothenburg, SE, Jul. 3-12, 2019, document JVET-O1168, 2019. cited by applicant

Kanumuri et al. "Use of Chroma QP Offsets in Deblocking," Joint Collaborative Team on Video Coding (JCT-VC) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 11th Meeting, Shanghai, CN, Oct. 10-19, 2012, document JCTVC-K0220, 2012. cited by applicant

Wan et al. "Consistent Chroma QP Derivation in the Deblocking and Inverse Quantization Processes," Joint Collaborative Team on Video Coding (JCT-VC) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 11th Meeting, Shanghai, CN, Oct. 10-19, 2012, document JCTVC-K0145, 2012. cited by applicant

Xu et al. "Non-CE5: Chroma QP Derivation Fix for Deblocking Filter (Combination of JVET-P0105 and JVET-P0539)," Joint Video Experts Team (JVET) of ITU-T SG WP 3 and SI/IEC JTC 1/SC 29/WG 11 16th Meeting, Geneva, CH, Oct. 1-11, 2019, document JVET-P1002, 2019. cited by applicant

Zhao et al. "AHG15: On CU Adaptive Chroma Offset Signalling," Joint Video Experts Team (JVET) of ITU-T SG WP 3 and SI/IEC JTC 1/SC 29/WG 11 16th Meeting, Geneva, CH, Oct. 1-11, 2019, document JVET-P0436, 2019. cited by applicant

Extended European Search Report from European Patent Application No. 20862153.2 dated Oct. 5, 2022 (12 pages). cited by applicant

Extended European Search Report from European Patent Application No. 20876422.5 dated Oct. 20, 2022 (7 pages). cited by applicant

Partial Supplementary European Search Report from European Patent Application No. 20877874.6 dated Oct. 31, 2022 (15 pages). cited by applicant

Non Final Office Action from U.S. Appl. No. 17/856,631 dated Nov. 10, 2022. cited by applicant

Kim et al. "AhG5: Deblocking Filter in 4:4:4 Chroma Format," Joint Collaborative Team on Video Coding (JCT-VC) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 15th Meeting: Geneva, CH, Oct. 23-Nov. 1, 2013. document JCTVC-O0089, 2013. (cited in EP20899673.6 EESR mailed Dec. 15, 2022). cited by applicant

Misra et al. "Non-CE11: On ISP Transform Boundary Deblocking," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 14th Meeting: Geneva, CH, Mar. 19-27, 2019, document JVET-N0473, 2019. (cited in EP20899673.6 EESR mailed Dec. 15, 2022). cited by applicant

Sjoberg et al. "HEVC High-Level Syntax" In: "High Efficiency Video Coding (HEVC): Algorithms and Architectures" Aug. 23, 2014, Springer, ISBN: 978-3-319-06895-4 pp. 13-48, 301:10.1007/978-3-319-06895-4_2 (cited in EP20899673.6 EESR mailed Dec. 15, 2022). cited by applicant

Wan et al. "AHG17: Text for Picture Header," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 16th Meeting: Geneva, CH, Oct. 1-11, 2019, document JVET-P1006, 2019. (cited in EP20899673.6 EESR mailed Dec. 15, 2022). cited by applicant

Extended European Search Report from European Patent Application No. 20899673.6 dated Dec. 15, 2022 (10 pages). cited by applicant

De-Luxian-Hernandez et al. "Non-CE3/Non-CE8: Enable Transform Skip in CUs Using ISP," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 14th Meeting, Geneva, CH, Mar. 19-27, 2019, document JVET-N0401, 2019. cited by applicant

Xiu et al. "On Signaling Adaptive Color Transform at TU Level," Joint Collaborative Team on Video Coding (JCT-VC) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 21st Meeting, Warsaw, PL, Jun. 19-26, 2015, document JCTVC-U0106, 2015. cited by applicant

Xu et al. "Non-CE5: Consistent Deblocking for Chroma Components," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 15th Meeting: Gothenburg, SE, Jul. 3-12, 2019, document JVET-O566, 2019. cited by applicant

Extended European Search Report from European Patent Application No. 20909161.0 dated Feb. 7, 2023 (10 pages). cited by applicant

Extended European Search Report from European Patent Application No. 20877874.6 dated Feb. 10, 2023 (17 pages). cited by applicant

Non Final Office Action from U.S. Appl. No. 17/852,934 dated Jan. 12, 2023. cited by applicant

Notice of Allowance from U.S. Appl. No. 17/694,305 dated Feb. 23, 2023. cited by applicant

Non Final Office Action from U.S. Appl. No. 17/810,187 dated Jan. 9, 2023. cited by applicant

Notice of Reasons for Refusal from Japanese Patent Application No. 2022-534625 dated May 16, 2023. cited by applicant

Chubach et al. "CE5-Related: On CC-ALF Modifications Related to Coefficients and Signalling," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, document JVET-Q0190, 2020. cited by applicant

Kotra et al. "CE-5 Related: High Level Syntax Modification for CCALF," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, document JVET-Q0253, 2020. cited by applicant

"JVET-P1006_SpecText-v2" Bross et al. "Versatile Video coding (Draft 6)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 15th Meeting: Gothenburg, SE, Jul. 3-12, 2019, document JVET-O2001, 2019. cited by applicant

Non Final Office Action from U.S. Appl. No. 17/856,631 dated Nov. 24, 2023. cited by applicant

Final Office Action from U.S. Appl. No. 17/694,305 dated Dec. 11, 2023. cited by applicant

First Office Action from Chinese Patent Application No. 202080088208.7 dated May 31, 2024. cited by applicant

Non Final Office Action from U.S. Appl. No. 17/864,011 dated Apr. 25, 2024. cited by applicant

Final Office Action from U.S. Appl. No. 17/856,631 dated Jun. 4, 2024. cited by applicant

Non Final Office Action from U.S. Appl. No. 18/524,994 dated Jun. 21, 2024. cited by applicant

Chang et al. "AhG9: On General Constraint Information Syntax," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 18th Meeting, by teleconference, Apr. 15-24, 2020, document JVET-R0286, 2020. cited by applicant

Chen et al. "Algorithm Description for Versatile Video Coding and Test Model 7 (VTM 7)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 16th Meeting: Geneva, CH, Oct. 1-11, 2019, document JVET-P2002, 2019. cited by applicant

Clare et al. "CE8-related: BDPCM for Chroma," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 15th Meeting: Gothenburg, SE, Jul. 3-12, 2019, document JVET-O0166, 2019. cited by applicant

Dou et al. "Disallowing JCCR Mode for ACT Coded CUs," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, document JVET-Q0305, 2020. cited by applicant

Jung et al. "On QP Adjustment in Adaptive Color Transform," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, document JVET-Q0241, 2020. cited by applicant

Kotra et al. "Non-CE5: Chroma QP Derivation Fix for Deblocking Filter (Combination of JVET-P0105 and JVET-P0539)," Joint Video Experts

Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 1 16th Meeting: Geneva, CH, Oct. 1-11, 2019, document JVET-P1001, 2019. cited by applicant

Li et al. "Fix of Initial QP Signaling," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 12th Meeting: Macao, CN, Oct. 3-12, 2018, document JVET-L0553, 2018. cited by applicant

Li et al. "Interaction Between ACT and BDPCM Chroma," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-20, 2020, document JVET-Q0423, 2020. cited by applicant

McCarthy et al. "AHG9: Modification of General Constraint Information," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 19th Meeting: by teleconference, Jun. 22-Jul. 1, 2020, document JVET-S0105, 2020. cited by applicant

Said et al. "CE5: Per-Context CABAC Initialization with Single Window (Test 5.1.4)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 1 13th Meeting: Marrakech, MA, Jan.18-20, 2019, document JVET-M0413, 2019. cited by applicant

Suehring, K., et al., Retrieved from the Internet: https://vcgit.hhi.fraunhofer.de/jvetAA/CSoftware_VTM/tags/VTM-7.0, Sep. 25, 2022, 2 pages. cited by applicant

Wang et al. "AHG2: Editorial Input on VVC Draft Text," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, document JVET-Q0041, 2020. cited by applicant

Wang, Ye-Kui. "AHG9: A Few More General Constraints Flags," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, document JVET-Q0114, 2020. cited by applicant

Wang et al. "AHG9: Cleanups on Signaling for CC-ALF, BDPCM, ACT and Palette," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, document JVET-Q0520, 2020. cited by applicant

Wang et al. "AHG8/AHG9/AHG12: On the Combination of RPR, Subpictures, and Scalability," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 18th Meeting: by teleconference, Apr. 15-24, 2020, document JVET-R0058, 2020. cited by applicant

Xiu et al. "AHG16: Clipping Residual Samples for ACT," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, document JVET-Q0513, 2020. cited by applicant

Xiu et al. "Support of Adaptive Color Transform for 444 Video Coding in VVC," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 16th Meeting: Geneva, CH, Oct. 1-11, 2019, document JVET-P0517, 2019. cited by applicant

Xiu et al. "AHG16: Clipping Residual Samples for ACT," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, document JVET-Q0513, 2020. cited by applicant

Zhang et al. "AHG12: Signaling of Chroma Presence in PPS and APS," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, document JVET-Q0420, 2020. cited by applicant

Zhang et al. "BoG on ACT," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, document JVET-Q0815, 2020. cited by applicant

Zhao et al. "An Implementation of Adaptive Color Transform in VVC," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 14th Meeting: Geneva, CH, Mar. 19-27, 2019, JVET-N0368, 2019. cited by applicant

Zhu et al. "Alignment of BDPCM for ACT," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, document JVET-Q0521, 2020. cited by applicant

Zhu et al. "ACT: Common Text for Bug Fixes and Transform Change," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, document JVET-Q0820, 2020. cited by applicant

Office Action from Chinese Patent Application No. CN202180009357.4 dated Apr. 7, 2023. cited by applicant

Extended European Search Report from European Patent Application No. 21736195.5 dated Mar. 15, 2023 (14 pages). cited by applicant

Notice of Reasons for Refusal from Japanese Patent Application No. 2022-541250 dated Aug. 9, 2023. cited by applicant

Decision to Grant a Patent from Japanese Patent Application No. 2022-541250 dated Jan. 9, 2024. cited by applicant

International Search Report and Written Opinion from International Patent Application No. PCT/CN2020/137941 dated Mar. 22, 2021 (10 pages). cited by applicant

International Search Report and Written Opinion from International Patent Application No. PCT/CN2020/137946 dated Mar. 22, 2021 (9 pages). cited by applicant

International Search Report and Written Opinion from International Patent Application No. PCT/CN2021/070265 dated Mar. 26, 2021 (14 pages). cited by applicant

International Search Report and Written Opinion from International Patent Application No. PCT/CN2021/070279 dated Apr. 1, 2021 (11 pages). cited by applicant

International Search Report and Written Opinion from International Patent Application No. PCT/CN2021/070282 dated Mar. 23, 2021 (13 pages). cited by applicant

International Search Report and Written Opinion from International Patent Application No. PCT/CN2021/070580 dated Mar. 22, 2021 (12 pages). cited by applicant

International Search Report and Written Opinion from International Patent Application No. PCT/CN2021/071659 dated Apr. 6, 2021 (14 pages). cited by applicant

International Search Report and Written Opinion from International Patent Application No. PCT/CN2021/071660 dated Mar. 31, 2021 (11 pages). cited by applicant

International Search Report and Written Opinion from International Patent Application No. PCT/CN2021/072017 dated Apr. 19, 2021 (14 pages). cited by applicant

International Search Report and Written Opinion from International Patent Application No. PCT/CN2021/072396 dated Mar. 29, 2021 (14 pages). cited by applicant

Non-Final Office Action from U.S. Appl. No. 17/842,025 dated Sep. 13, 2022. cited by applicant

Final Office Action from U.S. Appl. No. 17/842,025 dated Jan. 13, 2023. cited by applicant

Notice of Allowance from U.S. Appl. No. 17/857,874 dated Jan. 26, 2023. cited by applicant

Non-Final Office Action from U.S. Appl. No. 17/857,924 dated Feb. 3, 2023. cited by applicant

Notice of Allowance from U.S. Appl. No. 17/857,874 dated Jul. 6, 2023. cited by applicant

Notice of Allowance from U.S. Appl. No. 17/842,025 dated Aug. 10, 2023. cited by applicant

Non Final Office Action from U.S. Appl. No. 17/694,305 dated Mar. 27, 2024. cited by applicant

Notice of Allowance from U.S. Appl. No. 17/857,924 dated Apr. 17, 2024. cited by applicant

Li et al. "AHG12: Signaling of Chroma Presence in PPS and APS," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, document JVET-Q0420, 2020. cited by applicant

Non-Final Office Action from U.S. Appl. No. 17/810,187 dated Aug. 1, 2024, 34 pages. cited by applicant

Non-Final Office Action from U.S. Appl. No. 17/856,631 dated Sep. 26, 2024, 26 pages. cited by applicant

Non-Final Office Action from U.S. Appl. No. 18/519,994 dated Aug. 14, 2024, 125 pages. cited by applicant

Non-Final Office Action from U.S. Appl. No. 18/498,665 dated Sep. 26, 2024, 106 pages. cited by applicant

Non-Final Office Action for U.S. Appl. No. 17/720,582, mailed Jun. 26, 2023, 33 Pages. cited by applicant

Extended European Search Report for European Application No. 20862628.3, mailed Dec. 14, 2022, 07 Pages. cited by applicant
Extended European Search Report for European Application No. 20862629.1, mailed Oct. 27, 2022, 11 Pages. cited by applicant
Corrected Notice of Allowability for U.S. Appl. No. 17/716,447, mailed Mar. 17, 2023, 07 Pages. cited by applicant
Corrected Notice of Allowability for U.S. Appl. No. 17/716,447, mailed May 18, 2023, 11 Pages. cited by applicant
Notice of Allowance for U.S. Appl. No. 17/716,447, mailed Feb. 1, 2023, 19 Pages. cited by applicant
Corrected Notice of Allowability for U.S. Appl. No. 17/835,647, mailed Mar. 29, 2023, 11 Pages. cited by applicant
Corrected Notice of Allowability for U.S. Appl. No. 17/835,647, mailed Apr. 19, 2023, 7 Pages. cited by applicant
Corrected Notice of Allowability for U.S. Appl. No. 17/835,647, mailed Jun. 8, 2023, 06 Pages. cited by applicant
Notice of Allowance for U.S. Appl. No. 17/835,647, mailed Jan. 18, 2023, 31 Pages. cited by applicant
Chen et al., “Description of Core Experiment 4 (CE4): Inter Prediction with Geometric Partitioning,” Joint Video Expertf Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 16th Meeting: Geneva, CH, Oct. 1-11, 2019, Document JVET-P2024, 2019. cited by applicant
Final Office Action for U.S. Appl. No. 17/856,631, mailed Mar. 2, 2023, 19 pages. cited by applicant
Bross B., et al., “Versatile Video Coding (Draft 7),” Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 16th Meeting: Geneva, CH, Oct. 1-11, 2019, Document: JVET-P2001-v9, 495 Pages. cited by applicant
Xu J., et al., “Non-CE5: Consistent Deblocking for Chroma Components,” Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 15th Meeting: Gothenburg, SE, Jul. 3-12, 2019, Document: JVET-O0566-v2, 4 Pages. cited by applicant
Non-Final Office Action from U.S. Appl. No. 18/322,428 dated Mar. 28, 2024, 67 pages. cited by applicant
Final Office Action from U.S. Appl. No. 18/322,466 dated Jun. 21, 2024, 25 pages. cited by applicant
Non-Final Office Action for U.S. Appl. No. 18/498,652 dated May 22, 2024, 54 pages. cited by applicant
Final Office Action for U.S. Appl. No. 18/498,652 dated Oct. 11, 2024, 51 pages. cited by applicant
Document JVET-O2002-vE, Bross, B., et al., “Versatile Video Coding (Draft 6),” Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 15th Meeting: Gothenburg, SE, Jul. 3-12, 2019, 455 pages. cited by applicant
Communication Pursuant to Article 94(3) for European Application No. 200899673.6, mailed Feb. 3, 2025, 6 Pages. cited by applicant
Decision to Grant a Patent for Japanese Application No. 2023-203628, mailed Jan. 21, 2025, 5 pages. cited by applicant
Final Office Action for U.S. Appl. No. 18/498,665, dated Feb. 6, 2025, 74 pages. cited by applicant
Hearing Notice for Indian Patent Application No. 202247013800, mailed Feb. 6, 2025, 3 Pages. cited by applicant
Wan (Broadcom) W et al: “AHG17: Picture Header”, 128. MPEG Meeting; 201 91 007—Oct. 11, 2019; Geneva; (Motion Picture Expert Group or ISO/IEC JTC1/SC29/WG11), No. m50203 Sep. 25, 2019 (Sep. 25, 2019), XP030206176, Retrieved from the Internet URL: http://phenix.int-evry.fr/mpeg/doc_end_user/documents/128_Geneva/wg11/m50203-JVET-P0239-v1-JVET-P0239-v1.zip JVET-P0239-v1.docx [retrieved on Sep. 25, 2019]. cited by applicant
Written Decision on Registration for Korean Application No. 10-2022-7005294, mailed Feb. 10, 2025, 5 Pages. cited by applicant
Written Decision on Registration for Korean Application No. 10-2022-7005307, mailed Jan. 22, 2025, 5 Pages. cited by applicant
Communication Pursuant to Article 94(3) for European Application No. 20862629.1, mailed Jan. 31, 2025, 11 Pages. cited by applicant
Document: JVET-P1006-v2, Wan, W., et al., “AHG17: Text for picture header,” Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 16th Meeting: Geneva, CH, Oct. 1-11, 2019, 5 pages. cited by applicant
Communication Pursuant to Article 94(3) for European Application No. 20899673.6, mailed Mar. 2, 2025, 6 Pages. cited by applicant
Notice of Allowance from U.S. Appl. No. 18/524,994 dated Feb. 18, 2025, 8 pages. cited by applicant
Office Action from Chinese Patent Application No. 202180008570.3 dated Jan. 16, 2025, 22 pages. cited by applicant

Primary Examiner: Becker; Joseph W

Attorney, Agent or Firm: Conley Rose, P.C.

Background/Summary

CROSS REFERENCE TO RELATED APPLICATIONS (1) This application is a continuation of International Patent Application No. PCT/US2020/063746, filed on Dec. 8, 2020, which claims the priority to and benefits of International Patent Application No. PCT/CN2019/123951, filed on Dec. 9, 2019, and International Patent Application No. PCT/CN2019/130851, filed on Dec. 31, 2019. All the aforementioned patent applications are hereby incorporated by reference in their entireties.

TECHNICAL FIELD

(1) This patent document relates to video coding techniques, devices and systems.

BACKGROUND

(2) Currently, efforts are underway to improve the performance of current video codec technologies to provide better compression ratios or provide video coding and decoding schemes that allow for lower complexity or parallelized implementations. Industry experts have recently proposed several new video coding tools and tests are currently underway for determining their effectivity.

SUMMARY

(3) Devices, systems and methods related to digital video coding, and specifically, to management of motion vectors are described. The described methods may be applied to existing video coding standards (e.g., High Efficiency Video Coding (HEVC) or Versatile Video Coding) and future video coding standards or video codecs.

(4) In one representative aspect, the disclosed technology can be used to provide a method for video processing. The method includes applying, in a conversion between a video comprising multiple components and a bitstream representation of the video, a deblocking filter to video blocks of the multiple components. A deblocking filter strength for the deblocking filter of each of the multiple components is determined according to a rule that specifies to use a different manner for determining the deblocking filter strength for the video blocks of each of the multiple components.

(5) In another representative aspect, the disclosed technology can be used to provide a method for video processing. The method includes performing a conversion between a first video unit of a video and a bitstream representation of the video. During the conversion, a deblocking filtering process is applied to the first video unit. A deblocking control offset of the first video unit is determined based on accumulating one or more deblocking control offset values at other video unit levels.

(6) In another representative aspect, the disclosed technology can be used to provide a method for video processing. The method includes determining, for a conversion between a block of a video and a bitstream representation of the video, a quantization parameter used in a deblocking process based on usage of a transform skip (TS) mode or an adaptive color transform (ACT) mode for coding the block. The method also includes performing the conversion based on the determining.

(7) In another representative aspect, the disclosed technology can be used to provide a method for video processing. The method includes performing a conversion between a block of a color component of a video and a bitstream representation of the video. The bitstream representation conforms to a

rule specifying that a size of a quantization group of the chroma component is greater than a threshold K. The quantization group includes one or more coding units carrying a quantization parameter.

(8) In one representative aspect, the disclosed technology may be used to provide a method for video processing. This method includes performing a conversion between a video unit and a coded representation of the video unit, wherein, during the conversion, a deblocking filter is used on boundaries of the video unit such that when a chroma quantization parameter (QP) table is used to derive parameters of the deblocking filter, processing by the chroma QP table is performed on individual chroma QP values.

(9) In another representative aspect, the disclosed technology may be used to provide another method for video processing. This method includes performing a conversion between a video unit and a bitstream representation of the video unit, wherein, during the conversion, a deblocking filter is used on boundaries of the video unit such that chroma QP offsets are used in the deblocking filter, wherein the chroma QP offsets are at picture/slice/tile/brick/subpicture level.

(10) In another representative aspect, the disclosed technology may be used to provide another method for video processing. This method includes performing a conversion between a video unit and a bitstream representation of the video unit, wherein, during the conversion, a deblocking filter is used on boundaries of the video unit such that chroma QP offsets are used in the deblocking filter, wherein information pertaining to a same luma coding unit is used in the deblocking filter and for deriving a chroma QP offset.

(11) In another representative aspect, the disclosed technology may be used to provide another method for video processing. This method includes performing a conversion between a video unit and a bitstream representation of the video unit, wherein, during the conversion, a deblocking filter is used on boundaries of the video unit such that chroma QP offsets are used in the deblocking filter, wherein an indication of enabling usage of the chroma QP offsets is signaled in the bitstream representation.

(12) In another representative aspect, the disclosed technology may be used to provide another method for video processing. This method includes performing a conversion between a video unit and a bitstream representation of the video unit, wherein, during the conversion, a deblocking filter is used on boundaries of the video unit such that chroma QP offsets are used in the deblocking filter, wherein the chroma QP offsets used in the deblocking filter are identical of whether JCCR coding method is applied on a boundary of the video unit or a method different from the JCCR coding method is applied on the boundary of the video unit.

(13) In another representative aspect, the disclosed technology may be used to provide another method for video processing. This method includes performing a conversion between a video unit and a bitstream representation of the video unit, wherein, during the conversion, a deblocking filter is used on boundaries of the video unit such that chroma QP offsets are used in the deblocking filter, wherein a boundary strength (BS) of the deblocking filter is calculated without comparing reference pictures and/or a number of motion vectors (MVs) associated with the video unit at a P side boundary with reference pictures of the video unit at a Q side boundary.

(14) In another example aspect, another method of video processing is disclosed. The method includes determining, for a conversion between a video unit of a component of a video and a coded representation of the video, a size of a quantization group for the video unit, based on a constraint rule that specifies that the size must be larger than K, where K is a positive number and performing the conversion based on the determining.

(15) Further, in a representative aspect, an apparatus in a video system comprising a processor and a non-transitory memory with instructions thereon is disclosed. The instructions upon execution by the processor, cause the processor to implement any one or more of the disclosed methods.

(16) Additionally, in a representative aspect, a video decoding apparatus comprising a processor configured to implement any one or more of the disclosed methods.

(17) In another representative aspect, a video encoding apparatus comprising a processor configured to implement any one or more of the disclosed methods.

(18) Also, a computer program product stored on a non-transitory computer readable media, the computer program product including program code for carrying out any one or more of the disclosed methods is disclosed.

(19) The above and other aspects and features of the disclosed technology are described in greater detail in the drawings, the description and the claims.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

(1) FIG. 1 shows an example of an overall processing flow of a blocking deblocking filter process.

(2) FIG. 2 shows an example of a flow diagram of a Bs calculation.

(3) FIG. 3 shows an example of a referred information for Bs calculation at CTU boundary.

(4) FIG. 4 shows an example of pixels involved in filter on/off decision and strong/weak filter selection.

(5) FIG. 5 shows an example of an overall processing flow of deblocking filter process in VVC.

(6) FIG. 6 shows an example of a luma deblocking filter process in VVC.

(7) FIG. 7 shows an example of a chroma deblocking filter process in VVC.

(8) FIG. 8 shows an example of a filter length determination for sub PU boundaries.

(9) FIG. 9A shows an example of center positions of a chroma block.

(10) FIG. 9B shows another example of center positions of a chroma block.

(11) FIG. 10 shows examples of blocks at P side and Q side.

(12) FIG. 11 shows examples of usage of a luma block's decoded information.

(13) FIG. 12 is a block diagram of an example of a hardware platform for implementing a visual media decoding or a visual media encoding technique described in the present document.

(14) FIG. 13 shows a flowchart of an example method for video coding.

(15) FIG. 14A shows an example of Placement of CC-ALF with respect to other loop filters (b) Diamond shaped filter.

(16) FIG. 14B shows an example of Placement of CC-ALF with respect to Diamond shaped filter.

(17) FIG. 15 shows an example flowchart for an adaptive color transform (ACT) process.

(18) FIG. 16 shows an example flowchart for high-level deblocking control mechanism.

(19) FIG. 17 shows an example flowchart for high-level deblocking control mechanism.

(20) FIG. 18 is a block diagram that illustrates an example video coding system.

(21) FIG. 19 is a block diagram that illustrates an encoder in accordance with some embodiments of the present disclosure.

(22) FIG. 20 is a block diagram that illustrates a decoder in accordance with some embodiments of the present disclosure.

(23) FIG. 21 is a block diagram of an example video processing system in which disclosed techniques may be implemented.

(24) FIG. 22 is a flowchart representation of a method for video processing in accordance with the present technology.

(25) FIG. 23 is a flowchart representation of another method for video processing in accordance with the present technology.

(26) FIG. 24 is a flowchart representation of another method for video processing in accordance with the present technology.

(27) FIG. 25 is a flowchart representation of yet another method for video processing in accordance with the present technology.

DETAILED DESCRIPTION

1. Video Coding in HEVC/H.265

$p_{\text{sub.0}}' = \text{Clip1.sub.Y}(p_{\text{sub.0}} + \Delta)$
 $q_{\text{sub.0}}' = \text{Clip1.sub.Y}(q_{\text{sub.0}} - \Delta)$
 If $dEp1$ is equal to 1,
 $\Delta p = \text{Clip3}(-(t_{\text{sub.C}} > 1), t_{\text{sub.C}} > 1, (((p_{\text{sub.2}} + p_{\text{sub.0}} + 1) > 1) - p_{\text{sub.1}} + \Delta) > 1)$
 $p_{\text{sub.1}}' = \text{Clip1.sub.Y}(p_{\text{sub.1}} + \Delta p)$
 If $dEq1$ is equal to 1,
 $\Delta q = \text{Clip3}(-(t_{\text{sub.C}} > 1), t_{\text{sub.C}} > 1, (((q_{\text{sub.2}} + q_{\text{sub.0}} + 1) > 1) - q_{\text{sub.1}} - \Delta) > 1)$
 $q_{\text{sub.1}}' = \text{Clip1.sub.Y}(q_{\text{sub.1}} + \Delta q)$

(48) It is worth to note that maximum two pixels are modified using three pixels as an input for each P and Q block, respectively.

2.1.4.3. Chroma Filtering

(49) Bs of chroma filtering is inherited from luma. If $B_s > 1$ or if coded chroma coefficient existing case, chroma filtering is performed. No other filtering decision is there. And only one filter is applied for chroma. No filter selection process for chroma is used. The filtered sample values $p_{\text{sub.0}}'$ and $q_{\text{sub.0}}'$ are derived as follows.

$\Delta = \text{Clip3}(-t_{\text{sub.C}}, t_{\text{sub.C}}, (((q_{\text{sub.0}} - p_{\text{sub.0}}) < 2) + p_{\text{sub.1}} - q_{\text{sub.1}} + 4) > 3))$
 $p_{\text{sub.0}}' = \text{Clip1.sub.C}(p_{\text{sub.0}} + \Delta)$
 $q_{\text{sub.0}}' = \text{Clip1.sub.C}(q_{\text{sub.0}} - \Delta)$

2.2 Deblocking Scheme in VVC

(50) In the VTM6, deblocking filtering process is mostly the same to those in HEVC. However, the following modifications are added.

(51) A) The filter strength of the deblocking filter dependent of the averaged luma level of the reconstructed samples.

(52) B) Deblocking tC table extension and adaptation to 10-bit video.

(53) C) 4×4 grid deblocking for luma.

(54) D) Stronger deblocking filter for luma.

(55) E) Stronger deblocking filter for chroma.

(56) F) Deblocking filter for subblock boundary.

(57) G) Deblocking decision adapted to smaller difference in motion.

(58) FIG. 5 depicts a flowchart of deblocking filters process in VVC for a coding unit.

2.2.1. Filter Strength Dependent on Reconstructed Average Luma

(59) In HEVC, the filter strength of the deblocking filter is controlled by the variables β and $t_{\text{sub.C}}$ which are derived from the averaged quantization parameters $qP_{\text{sub.L}}$. In the VTM6, deblocking filter controls the strength of the deblocking filter by adding offset to $qP_{\text{sub.L}}$ according to the luma level of the reconstructed samples if the SPS flag of this method is true. The reconstructed luma level LL is derived as follow:

$LL = ((p_{\text{sub.0,0}} + p_{\text{sub.0,3}} + q_{\text{sub.0,0}} + q_{\text{sub.0,3}}) > 2) / (1 < \text{bitDepth}) \quad (3-1)$

where, the sample values $p_{\text{sub.i,k}}$ and $q_{\text{sub.i,k}}$ with $i=0 \dots 3$ and $k=0$ and 3 can be derived. Then LL is used to decide the offset $qpOffset$ based on the threshold signaled in SPS. After that, the $qP_{\text{sub.L}}$, which is derived as follows, is employed to derive the β and $t_{\text{sub.C}}$.

$qP_{\text{sub.L}} = ((Qp_{\text{sub.Q}} + Qp_{\text{sub.P}} + 1) > 1) + qpOffset \quad (3-2)$

where $Qp_{\text{sub.Q}}$ and $Qp_{\text{sub.P}}$ denote the quantization parameters of the coding units containing the sample $q_{\text{sub.0,0}}$ and $p_{\text{sub.0,0}}$, respectively. In the current VVC, this method is only applied on the luma deblocking process.

2.2.2. 4×4 Deblocking Grid for Luma

(60) HEVC uses an 8×8 deblocking grid for both luma and chroma. In VTM6, deblocking on a 4×4 grid for luma boundaries was introduced to handle blocking artifacts from rectangular transform shapes. Parallel friendly luma deblocking on a 4×4 grid is achieved by restricting the number of samples to be deblocked to 1 sample on each side of a vertical luma boundary where one side has a width of 4 or less or to 1 sample on each side of a horizontal luma boundary where one side has a height of 4 or less.

2.2.3. Boundary Strength Derivation for Luma

(61) The detailed boundary strength derivation could be found in Table 2. The conditions in Table 2 are checked sequentially.

(62) TABLE-US-00002 TABLE 2 Boundary strength derivation Conditions Y U V P and Q are BDPCM 0 N/A N/A P or Q is intra 2 2 2 It is a transform block edge, and P or Q is CIIP 2 2 2 It is a transform block edge, and P or Q has non-zero transform coefficients 1 1 1 It is a transform block edge, and P or Q is JCCR N/A 1 1 P and Q are in different coding modes 1 1 1 One or more of the following conditions are true: 1 N/A N/A 1. P and Q are both IBC, and the BV distance $\geq$ half-pel in x- or y-dir 2. P and Q have different ref pictures*, or have different number of MVs 3. Both P and Q have only one mv, and the MV distance $\geq$ half -pel in x- or y-dir 4. P has two MVs pointing to two different ref pictures, and P and Q have same ref pictures in the list 0, the MV pair in the list 0 or list 1 has a distance $\geq$ half-pel in x- or y-dir 5. P has two MVs pointing to two different ref pictures, and P and Q have different ref pictures in the list 0, the MV of P in the list 0 and the MV of Q in the list 1 have the distance $\geq$ half-pel in x- or y-dir, or the MV of P in the list 1 and the MV of Q in the list 0 have the distance $\geq$ half-pel in x- or y-dir 6. Both P and Q have two MVs pointing to the same ref pictures, and both of the following two conditions are satisfied: The MV of P in the list 0 and the MV of Q in the list 0 has a distance $\geq$ half-pel in x- or y-dir or the MV of P in the list 1 and the MV of Q in the list 1 has a distance $\geq$ half-pel in x- or y-dir The MV of P in the list 0 and the MV of Q in the list 1 has a distance $\geq$ half-pel in x- or y-dir or the MV of P in the list 1 and the MV of Q in the list 0 has a distance $\geq$ half-pel in x- or y-dir *Note: The determination of whether the reference pictures used for the two coding subblocks are the same or different is based only on which pictures are referenced, without regard to whether a prediction is formed using an index into reference picture list 0 or an index into reference picture list 1, and also without regard to whether the index position within a reference picture list is different. Otherwise 0 0 0

2.2.4. Stronger Deblocking Filter for Luma

(63) The proposal uses a bilinear filter when samples at either one side of a boundary belong to a large block. A sample belonging to a large block is defined as when the width ≥ 32 for a vertical edge, and when height ≥ 32 for a horizontal edge.

(64) The bilinear filter is listed below.

(65) Block boundary samples p_i for $i=0$ to S_p-1 and q_j for $j=0$ to S_q-1 (p_i and q_j follow the definitions in HBEVC deblocking described above) are then replaced by linear interpolation as follows:

$p_{\text{sub.i}}' = (f_{\text{sub.i}} * \text{Middle.sub.s.t} + (64 - f_{\text{sub.i}}) * P_{\text{sub.s}} + 32) > 6$, clipped to $p_{\text{sub.i}} \pm tcPD_{\text{sub.i}}$

$q_{\text{sub.j}}' = (g_{\text{sub.j}} * \text{Middle.sub.s.t} + (64 - g_{\text{sub.j}}) * Q_{\text{sub.s}} + 32) > 6$, clipped to $q_{\text{sub.j}} \pm tcPD_{\text{sub.j}}$

where $tcPD_{\text{sub.i}}$ and $tcPD_{\text{sub.j}}$ term is a position dependent clipping described in Section 2.2.5 and $g_{\text{sub.j}}$, $f_{\text{sub.i}}$, Middle.sub.s.t , $P_{\text{sub.s}}$ and $Q_{\text{sub.s}}$ are given below:

(66) TABLE-US-00003 S_p , S_q $f_{\text{sub.i}} = 59 - i * 9$, can also be described as $f = \{59, 50, 41, 32, 23, 14, 5\}$ 7, 7 $g_{\text{sub.j}} = 59 - j * 9$, can also be described as $g = \{59, 50, 41, 32, 23, 14, 5\}$ (p side: 7, Middle.sub.7, 7 = $(2 * (p_{\text{sub.o}} + q_{\text{sub.o}}) + p_{\text{sub.1}} + q_{\text{sub.1}} + p_{\text{sub.2}} + q_{\text{sub.2}} + p_{\text{sub.3}} + q_{\text{sub.3}} + p_{\text{sub.4}} + p_{\text{sub.5}} + p_{\text{sub.6}} + q_{\text{sub.6}} + q_{\text{sub.7}} + 8) > 4$ $P_{\text{sub.7}} = (p_{\text{sub.6}} + p_{\text{sub.7}} + 1) > 1$, $Q_{\text{sub.7}} = (q_{\text{sub.6}} + q_{\text{sub.7}} + 1) > 1$ 7, 3 $f_{\text{sub.i}} = 59 - i * 9$, can also be described as $f = \{59, 50, 41, 32, 23, 14, 5\}$ (p side: 7 $g_{\text{sub.j}} = 53 - j * 21$, can also be described as $g = \{53, 32, 11\}$ q side: 3) Middle.sub.7, 3 = $(2 * (p_{\text{sub.o}} + q_{\text{sub.o}}) + q_{\text{sub.0}} + 2 * (q_{\text{sub.1}} + q_{\text{sub.2}}) + p_{\text{sub.1}} + q_{\text{sub.1}} + p_{\text{sub.2}} + p_{\text{sub.3}} + p_{\text{sub.4}} + p_{\text{sub.5}} + p_{\text{sub.6}} + 8) > 4$ $P_{\text{sub.7}} = (p_{\text{sub.6}} + p_{\text{sub.7}} + 1) > 1$, $Q_{\text{sub.3}} = (q_{\text{sub.2}} + q_{\text{sub.3}} + 1) > 1$ 3, 7 $g_{\text{sub.j}} = 59 - j * 9$, can also be described as $g = \{59, 50, 41, 32, 23, 14, 5\}$ (p side: 3 $f_{\text{sub.i}} = 53 - i * 21$, can also be described as $f = \{53, 32, 11\}$ q side: 7) Middle.sub.3.7 = $(2 * (q_{\text{sub.o}} + p_{\text{sub.o}}) + p_{\text{sub.0}} + 2 * (p_{\text{sub.1}} + p_{\text{sub.2}}) + q_{\text{sub.1}} + p_{\text{sub.1}} + q_{\text{sub.2}} + q_{\text{sub.3}} + q_{\text{sub.4}} + q_{\text{sub.5}} + q_{\text{sub.6}} + 8) > 4$ $Q_{\text{sub.7}} =$

$q.sub.7 + 1) >> 1$, $P.sub.3 = (p.sub.2 + p.sub.3 + 1) >> 1$, $5\ g.sub.j = 58 - j * 13$, can also be described as $g = \{58, 45, 32, 19, 6\}$ (p side: $7\ f.sub.i = 59 - i * 9$, can also be described as $f = \{59, 50, 41, 32, 23, 14, 5\}$ q side: $5\ Middle7, 5 = (2 * (p.sub.o + q.sub.o + p.sub.1 + q.sub.1) + q.sub.2 + p.sub.2 + q.sub.3 + p.sub.3 + q.sub.4 + p.sub.4 + q.sub.5 + q.sub.5 + 8) >> 4$ $Q.sub.5 = (q.sub.4 + q.sub.5 + 1) >> 1$, $P.sub.7 = (p.sub.6 + p.sub.7 + 1) >> 1$, $5\ g.sub.j = 59 - j * 9$, can also be described as $g = \{59, 50, 41, 32, 23, 14, 5\}$ (p side: $5\ f.sub.i = 58 - i * 13$, can also be described as $f = \{58, 45, 32, 19, 6\}$ q side: $7\ Middle5, 7 = (2 * (q.sub.o + p.sub.o + p.sub.1 + q.sub.1) + q.sub.2 + p.sub.2 + q.sub.3 + p.sub.3 + q.sub.4 + p.sub.4 + q.sub.5 + p.sub.5 + 8) >> 4$ $Q.sub.7 = (q.sub.6 + q.sub.7 + 1) >> 1$, $P.sub.5 = (p.sub.4 + p.sub.5 + 1) >> 1$, $5\ g.sub.j = 58 - j * 13$, can also be described as $g = \{58, 45, 32, 19, 6\}$ (p side: $5\ f.sub.i = 58 - i * 13$, can also be described as $f = \{58, 45, 32, 19, 6\}$ q side: $5\ Middle5, 5 = (2 * (q.sub.o + p.sub.o + p.sub.1 + q.sub.1 + q.sub.2 + p.sub.2) + q.sub.3 + p.sub.3 + q.sub.4 + p.sub.4 + 8) >> 4$ $Q.sub.5 = (q.sub.4 + q.sub.5 + 1) >> 1$, $P.sub.5 = (p.sub.4 + p.sub.5 + 1) >> 1$, $3\ g.sub.j = 53 - j * 21$, can also be described as $g = \{53, 32, 11\}$ (p side: $5\ f.sub.i = 58 - i * 13$, can also be described as $f = \{58, 45, 32, 19, 6\}$ q side: $3\ Middle5, 3 = (q.sub.o + p.sub.o + p.sub.1 + q.sub.1 + q.sub.2 + p.sub.2 + q.sub.3 + p.sub.3 + 4) >> 3$ $Q.sub.3 = (q.sub.2 + q.sub.3 + 1) >> 1$, $P.sub.5 = (p.sub.4 + p.sub.5 + 1) >> 1$, $3\ g.sub.j = 58 - j * 13$, can also be described as $g = \{58, 45, 32, 19, 6\}$ (p side: $3\ f.sub.i = 53 - i * 21$, can also be described as $f = \{53, 32, 11\}$ q side: $5\ Middle3, 5 = (q.sub.o + p.sub.o + p.sub.1 + q.sub.1 + q.sub.2 + p.sub.2 + q.sub.3 + p.sub.3 + 4) >> 3$ $Q.sub.5 = (q.sub.4 + q.sub.5 + 1) >> 1$, $P.sub.3 = (p.sub.2 + p.sub.3 + 1) >> 1$

2.2.5. Deblocking Control for Luma

(67) The deblocking decision process is described in this sub-section.

(68) Wider-stronger luma filter is filters are used only if all of the Condition1, Condition2 and Condition 3 are TRUE.

(69) The condition 1 is the “large block condition”. This condition detects whether the samples at P-side and Q-side belong to large blocks, which are represented by the variable bSidePisLargeBlk and bSideQisLargeBlk respectively. The bSidePisLargeBlk and bSideQisLargeBlk are defined as follows.

$bSidePisLargeBlk = ((edge\ type\ is\ vertical\ and\ p.sub.0\ belongs\ to\ CU\ with\ width \geq 32) \parallel (edge\ type\ is\ horizontal\ and\ p.sub.0\ belongs\ to\ CU\ with\ height \geq 32)) ? TRUE : FALSE$

$bSideQisLargeBlk = ((edge\ type\ is\ vertical\ and\ q.sub.0\ belongs\ to\ CU\ with\ width \geq 32) \parallel (edge\ type\ is\ horizontal\ and\ q.sub.0\ belongs\ to\ CU\ with\ height \geq 32)) ? TRUE : FALSE$

(70) Based on bSidePisLargeBlk and bSideQisLargeBlk, the condition 1 is defined as follows.

$Condition1 = (bSidePisLargeBlk \parallel bSideQisLargeBlk) ? TRUE : FALSE$

(71) Next, if Condition 1 is true, the condition 2 will be further checked. First, the following variables are derived:

(72) TABLE-US-00004 dp0, dp3, dq0, dq3 are first derived as in HEVC if (p side is greater than or equal to 32) $dp0 = (dp0 + Abs(p.sub.5,0 - 2 * p.sub.4,0 + p.sub.3,0) + 1) >> 1$ $dp3 = (dp3 + Abs(p.sub.5,3 - 2 * p.sub.4,3 + p.sub.3,3) + 1) >> 1$ if (q side is greater than or equal to 32) $dq0 = (dq0 + Abs(q.sub.5,0 - 2 * q.sub.4,0 + q.sub.3,0) + 1) >> 1$ $dq3 = (dq3 + Abs(q.sub.5,3 - 2 * q.sub.4,3 + q.sub.3,3) + 1) >> 1$ dpq0, dpq3, dp, dq, d are then derived as in HEVC.

(73) Then the condition 2 is defined as follows.

$Condition2 = (d < \beta) ? TRUE : FALSE$

Where $d = dp0 + dq0 + dp3 + dq3$, as shown in section 2.1.4.

(74) If Condition1 and Condition2 are valid it is checked if any of the blocks uses sub-blocks:

(75) TABLE-US-00005 If(bSidePisLargeBlk) If(mode block P == SUBBLOCKMODE) Sp=5 else Sp=7 else Sp=3
If(bSideQisLargeBlk) If(mode block Q == SUBBLOCKMODE) Sq=5 else Sq=7 else Sq=3

(76) Finally, if both the Condition 1 and Condition 2 are valid, the proposed deblocking method will check the condition 3 (the large block Strongfilter condition), which is defined as follows. In the Condition3 StrongFilterCondition, the following variables are derived:

(77) TABLE-US-00006 dpq is derived as in HEVC. $sp3 = Abs(p3 - p0)$, derived as in HEVC if (p side is greater than or equal to 32) if(Sp==5) $sp3 = (sp3 + Abs(p5 - p3) + 1) >> 1$ else $sp3 = (sp3 + Abs(p7 - p3) + 1) >> 1$ $sq3 = Abs(q0 - q3)$, derived as in HEVC if (q side is greater than or equal to 32) If(Sq==5) $sq3 = (sq3 + Abs(q5 - q3) + 1) >> 1$ else $sq3 = (sq3 + Abs(q7 - q3) + 1) >> 1$

(78) As in HEVC derive, StrongFilterCondition=(dpq is less than ($\beta >> 2$), $sp3 + sq3$ is less than ($3 * \beta >> 5$), and $Abs(p0 - q0)$ is less than ($5 * tC + 1) >> 1$)? TRUE: FALSE

(79) FIG. 6 depicts the flowchart of luma deblocking filter process.

2.2.6. Strong Deblocking Filter for Chroma

(80) The following strong deblocking filter for chroma is defined:

$p.sub.2' = (3 * p.sub.3 + 2 * p.sub.2 + p.sub.1 + p.sub.0 + q.sub.0 + 4) >> 3$

$p.sub.1' = (2 * p.sub.3 + p.sub.2 + 2 * p.sub.1 + p.sub.0 + q.sub.0 + q.sub.1 + 4) >> 3$

$p.sub.0' = (p.sub.3 + p.sub.2 + p.sub.1 + 2 * p.sub.0 + q.sub.0 + q.sub.1 + q.sub.2 + 4) >> 3$

(81) The proposed chroma filter performs deblocking on a 4×4 chroma sample grid.

2.2.7. Deblocking Control for Chroma

(82) The above chroma filter performs deblocking on a 8×8 chroma sample grid. The chroma strong filters are used on both sides of the block boundary. Here, the chroma filter is selected when both sides of the chroma edge are greater than or equal to 8 (in unit of chroma sample), and the following decision with three conditions are satisfied. The first one is for decision of boundary strength as well as large block. The second and third one are basically the same as for HEVC luma decision, which are on/off decision and strong filter decision, respectively.

(83) FIG. 7 depicts the flowchart of chroma deblocking filter process.

2.2.8. Position Dependent Clipping

(84) The proposal also introduces a position dependent clipping tcPD which is applied to the output samples of the luma filtering process involving strong and long filters that are modifying 7, 5 and 3 samples at the boundary. Assuming quantization error distribution, it is proposed to increase clipping value for samples which are expected to have higher quantization noise, thus expected to have higher deviation of the reconstructed sample value from the true sample value.

(85) For each P or Q boundary filtered with proposed asymmetrical filter, depending on the result of decision making process described in Section 2.2, position dependent threshold table is selected from Tc7 and Tc3 tables that are provided to decoder as a side information:

$Tc7 = \{6, 5, 4, 3, 2, 1, 1\};$

$Tc3 = \{6, 4, 2\};$

$tcPD = (SP == 3) ? Tc3 : Tc7;$

$tcQD = (SQ == 3) ? Tc3 : Tc7;$

(86) For the P or Q boundaries being filtered with a short symmetrical filter, position dependent threshold of lower magnitude is applied:

$Tc3 = \{3, 2, 1\};$

(87) Following defining the threshold, filtered p'i and q'i sample values are clipped according to tcP and tcQ clipping values:

$p''.sub.i = clip3(p'.sub.i + tcP.sub.i, p'.sub.i - tcP.sub.i, p'.sub.i);$

$q''.sub.j = clip3(q'.sub.j + tcQ.sub.j, q'.sub.j - tcQ.sub.j, q'.sub.j);$

where p'.sub.i and q'.sub.i are filtered sample values, p''.sub.i and q''.sub.j are output sample value after the clipping and tcP.sub.i tcQ.sub.i are clipping thresholds that are derived from the VVC tc parameter and tcPD and tcQD. Term clip3 is a clipping function as it is specified in VVC.

2.2.9. Sub-Block Deblocking Adjustment

(88) To enable parallel processing using both long filters and sub-block deblocking to modify at most 5 samples on a side that uses sub-block deblocking (AFFINE or ATMVP) as shown in the luma control for long filters. Additionally, the sub-block deblocking is adjusted such that that sub-block boundaries on an 8x8 grid that are close to a CU or an implicit TU boundary is restricted to modify at most two samples on each side.

(89) Following applies to sub-block boundaries that not are aligned with the CU boundary.

(90) TABLE-US-00007 If(mode block Q == SUBBLOCKMODE && edge!=0){ if (!implicitTU && (edge == (64 / 4))) if (edge == 2 || edge == (orthogonalLength - 2) || edge == (56 / 4) || edge == (72 / 4)) Sp = Sq = 2; else Sp = Sq = 3; else Sp = Sq = bSideQisLargeBlk? 5:3 }

(91) Where edge equal to 0 corresponds to CU boundary, edge equal to 2 or equal to orthogonalLength-2 corresponds to sub-block boundary 8 samples from a CU boundary etc. Where implicit TU is true if implicit split of TU is used. FIG. 8 show the flowcharts of determination process for TU boundaries and sub-PU boundaries.

(92) Filtering of horizontal boundary is limiting Sp=3 for luma, Sp=1 and Sq=1 for chroma, when the horizontal boundary is aligned with the CTU boundary.

2.2.10. Deblocking Decision Adapted to Smaller Difference in Motion

(93) HEVC enables deblocking of a prediction unit boundary when the difference in at least one motion vector component between blocks on respective side of the boundary is equal to or larger than a threshold of 1 sample. In VTM6, a threshold of a half luma sample is introduced to also enable removal of blocking artifacts originating from boundaries between inter prediction units that have a small difference in motion vectors.

2.3. Combined Inter and Intra Prediction (CIIP)

(94) In VTM6, when a CU is coded in merge mode, if the CU contains at least 64 luma samples (that is, CU width times CU height is equal to or larger than 64), and if both CU width and CU height are less than 128 luma samples, an additional flag is signalled to indicate if the combined inter/intra prediction (CIIP) mode is applied to the current CU. As its name indicates, the CIIP prediction combines an inter prediction signal with an intra prediction signal. The inter prediction signal in the CIIP mode P.sub.inter is derived using the same inter prediction process applied to regular merge mode; and the intra prediction signal P.sub.intra is derived following the regular intra prediction process with the planar mode. Then, the intra and inter prediction signals are combined using weighted averaging, where the weight value is calculated depending on the coding modes of the top and left neighbouring blocks as follows: If the top neighbor is available and intra coded, then set isIntraTop to 1, otherwise set isIntraTop to 0; If the left neighbor is available and intra coded, then set isIntraLeft to 1, otherwise set isIntraLeft to 0; If (isIntraLeft+isIntraTop) is equal to 2, then wt is set to 3; Otherwise, if (isIntraLeft+isIntraTop) is equal to 1, then wt is set to 2; Otherwise, set wt to 1.

(95) The CIIP prediction is formed as follows:

$$P_{\text{sub.CIIP}} = ((4 - wt) * P_{\text{sub.inter}} + wt * P_{\text{sub.intra}} + 2) >> 2$$

2.4. Chroma QP Table Design in VTM-6.0

(96) In some embodiments, a chroma QP table is used. In some embodiments, a signalling mechanism is used for chroma QP tables, which enables that it is flexible to provide encoders the opportunity to optimize the table for SDR and HDR content. It supports for signalling the tables separately for Cb and Cr components. The proposed mechanism signals the chroma QP table as a piece-wise linear function.

2.5. Transform Skip(TS)

(97) As in HEVC, the residual of a block can be coded with transform skip mode. To avoid the redundancy of syntax coding, the transform skip flag is not signalled when the CU level MTS_CU_flag is not equal to zero. The block size limitation for transform skip is the same to that for MTS in JEM4, which indicate that transform skip is applicable for a CU when both block width and height are equal to or less than 32. Note that implicit MTS transform is set to DCT2 when LFNST or MIP is activated for the current CU. Also the implicit MTS can be still enabled when MTS is enabled for inter coded blocks.

(98) In addition, for transform skip block, minimum allowed Quantization Parameter (QP) is defined as $6 * (\text{internalBitDepth} - \text{inputBitDepth}) + 4$.

2.6. Joint Coding of Chroma Residuals (JCCR)

(99) In some embodiments, the chroma residuals are coded jointly. The usage (activation) of a joint chroma coding mode is indicated by a TU-level flag tu_joint_cbr_residual_flag and the selected mode is implicitly indicated by the chroma CBFs. The flag tu_joint_cbr_residual_flag is present if either or both chroma CBFs for a TU are equal to 1. In the PPS and slice header, chroma QP offset values are signalled for the joint chroma residual coding mode to differentiate from the usual chroma QP offset values signalled for regular chroma residual coding mode. These chroma QP offset values are used to derive the chroma QP values for those blocks coded using the joint chroma residual coding mode. When a corresponding joint chroma coding mode (modes 2 in Table 3) is active in a TU, this chroma QP offset is added to the applied luma-derived chroma QP during quantization and decoding of that TU. For the other modes (modes 1 and 3 in Table 3 Table 3 Reconstruction of chroma residuals. The value CSign is a sign value (+1 or -1), which is specified in the slice header; resJointC[][] is the transmitted residual.), the chroma QPs are derived in the same way as for conventional Cb or Cr blocks. The reconstruction process of the chroma residuals (resCb and resCr) from the transmitted transform blocks is depicted in Table 3. When this mode is activated, one single joint chroma residual block (resJointC[x][y] in Table 3) is signalled, and residual block for Cb (resCb) and residual block for Cr (resCr) are derived considering information such as tu_cbf_cb, tu_cbf_cr, and CSign, which is a sign value specified in the slice header.

(100) At the encoder side, the joint chroma components are derived as explained in the following. Depending on the mode (listed in the tables above), resJointC{1,2} are generated by the encoder as follows: If mode is equal to 2 (single residual with reconstruction Cb=C, Cr=CSign*C), the joint residual is determined according to

$\text{resJointC}[x][y] = (\text{resCb}[x][y] + \text{CSign} * \text{resCr}[x][y]) / 2$. Otherwise, if mode is equal to 1 (single residual with reconstruction Cb=C, Cr=(CSign*C)/2), the joint residual is determined according to

$\text{resJointC}[x][y] = (4 * \text{resCb}[x][y] + 2 * \text{CSign} * \text{resCr}[x][y]) / 5$. Otherwise (mode is equal to 3, i. e., single residual, reconstruction Cr=C, Cb=(CSign*C)/2), the joint residual is determined according to

$\text{resJointC}[x][y] = (4 * \text{resCr}[x][y] + 2 * \text{CSign} * \text{resCb}[x][y]) / 5$.

(101) TABLE-US-00008 TABLE 3 Reconstruction of chroma residuals. The value CSign is a sign value (+1 or -1), which is specified in the slice header, resJointC[][] is the transmitted residual. tu_cbf_cb tu_cbf_cr reconstruction of Cb and Cr residuals mode 1 0 resCb[x][y] = resJointC[x][y] 1 resCr[x][y] = (CSign * resJointC[x][y]) >> 1 1 1 resCb[x][y] = resJointCf x][y] 2 resCr[x][y] = CSign * resJointC[x][y] 0 1 resCb[x][y] = 3 (CSign * resJointC[x][y]) >> 1 resCr[x][y] = resJointC[x][y]

(102) Different QPs are utilized are the above three modes. For mode 2, the QP offset signaled in PPS for JCCR coded block is applied, while for other two modes, it is not applied, instead, the QP offset signaled in PPS for non-JCCR coded block is applied.

(103) The corresponding specification is as follows:

(104) 8.7.1 Derivation Process for Quantization Parameters

(105) The variable Qp.sub.Y is derived as follows:

$$Qp_{\text{sub.Y}} = ((Qp_{\text{sub.Y_PRED}} + CuQpDeltaVal + 64 + 2 * QpBdOffset_{\text{sub.Y}}) \% (64 + QpBdOffset_{\text{sub.Y}})) - QpBdOffset_{\text{sub.Y}} \quad (8-933)$$

The luma quantization parameter Qp'.sub.Y is derived as follows:

$$Qp'_{\text{sub.Y}} = Qp_{\text{sub.Y}} + QpBdOffset_{\text{sub.Y}} \quad (8-934)$$

When ChromaArrayType is not equal to 0 and tree Type is equal to SINGLE_TREE or DUAL_TREE_CHROMA, the following applies: When tree Type is equal to DUAL_TREE_CHROMA, the variable Qp.sub.Y is set equal to the luma quantization parameter Qp.sub.Y of the luma coding unit

that covers the luma location (xCb+cbWidth/2, yCb+cbHeight/2). The variables qP.sub.Cb, qP.sub.Cr and qP.sub.CbCr are derived as follows:

$qPi.sub.Chroma = Clip3(-QpBdOffset.sub.C, 63, Qp.sub.Y)$ (8-935)

$qPi.sub.Cb = ChromaQpTable[0][qPi.sub.Chroma]$ (8-936)

$qPi.sub.Cr = ChromaQpTable[1][qPi.sub.Chroma]$ (8-937)

$qPi.sub.CbCr = ChromaQpTable[2][qPi.sub.Chroma]$ (8-938) The chroma quantization parameters for the Cb and Cr components, Qp'.sub.Cb and Qp'.sub.Cr, and joint Cb-Cr coding Qp'.sub.CbCr are derived as follows:

$Qp'.sub.Cb = Clip3(-QpBdOffset.sub.C, 63, qP.sub.Cb + pps_cb_qp_offset + slice_cb_qp_offset + CuQpOffset.sub.Cb) + QpBdOffset.sub.C$ (8-939)

$Qp'.sub.Cr = Clip3(-QpBdOffset.sub.C, 63, qP.sub.Cr + pps_cr_qp_offset + slice_cr_qp_offset + CuQpOffset.sub.Cr) + QpBdOffset.sub.C$ (8-940)

$Qp'.sub.CbCr = Clip3(-QpBdOffset.sub.C, 63, qP.sub.CbCr + pps_cbcr_qp_offset + slice_cbcr_qp_offset + CuQpOffset.sub.CbCr) + QpBdOffset.sub.C$ (8-941)

8.7.3 Scaling Process for Transform Coefficients

Inputs to this process are: a luma location(xTbY, yTbY) specifying the top-left sample of the current luma transform block relative to the top-left luma sample of the current picture, a variable nTbW specifying the transform block width, a variable nTbH specifying the transform block height, a variable cIdx specifying the colour component of the current block, a variable bitDepth specifying the bit depth of the current colour component.

Output of this process is the (nTbW)×(nTbH) array d of scaled transform coefficients with elements d[x][y]. The quantization parameter qP is derived as follows: If cIdx is equal to 0 and transform_skip_flag[xTbY][yTbY] is equal to 0, the following applies:

$qP = Qp'.sub.Y$ (8-950) Otherwise, if cIdx is equal to 0 (and transform_skip_flag[xTbY][yTbY] is equal to 1), the following applies:

$qP = Max(QpPrimeTsMin, Qp'.sub.Y)$ (8-951) Otherwise, if TuCResMode[xTbY][yTbY] is equal to 2, the following applies:

$qP = Qp'.sub.CbCr$ (8-952) Otherwise, if cIdx is equal to 1, the following applies:

$qP = Qp'.sub.Cb$ (8-953) Otherwise (cIdx is equal to 2), the following applies:

$qP = Qp'.sub.Cr$ (8-954)

2.7. Cross-Component Adaptive Loop Filter (CC-ALF)

(106) FIG. 14A illustrates the placement of CC-ALF with respect to the other loop filters. CC-ALF operates by applying a linear, diamond shaped filter (FIG. 14B) to the luma channel for each chroma component, which is expressed as

$$(107) \quad I_i(x, y) = \underset{(x_0, y_0) \in S_i}{\text{Math.}} \quad I_0(x_C + x_0, y_C + y_0) c_i(x_0, y_0),$$

where (x, y) is chroma component i location being refined (x.sub.C, y.sub.C) is the luma location based on (x, y) S.sub.i is filter support in luma for chroma component i c.sub.i(x.sub.0, y.sub.0) represents the filter coefficients

(108) Key features characteristics of the CC-ALF process include: The luma location (x.sub.C, y.sub.C), around which the support region is centered, is computed based on the spatial scaling factor between the luma and chroma planes. All filter coefficients are transmitted in the APS and have 8-bit dynamic range. An APS may be referenced in the slice header. CC-ALF coefficients used for each chroma component of a slice are also stored in a buffer corresponding to a temporal sublayer. Reuse of these sets of temporal sublayer filter coefficients is facilitated using slice-level flags. The application of the CC-ALF filters is controlled on a variable block size and signalled by a context-coded flag received for each block of samples. The block size along with an CC-ALF enabling flag is received at the slice-level for each chroma component. Boundary padding for the horizontal virtual boundaries makes use of repetition. For the remaining boundaries the same type of padding is used as for regular ALF.

2.8 Derivation Process for Quantization Parameters

(109) The QP is derived based on neighboring QPs and the decoded delta QP. The example texts related to QP derivation are given as follows.

(110) Inputs to this process are: a luma location(xCb, yCb) specifying the top-left luma sample of the current coding block relative to the top-left luma sample of the current picture, a variable cbWidth specifying the width of the current coding block in luma samples, a variable cbHeight specifying the height of the current coding block in luma samples, a variable treeType specifying whether a single tree (SINGLE_TREE) or a dual tree is used to partition the coding tree node and, when a dual tree is used, whether the luma (DUAL_TREE_LUMA) or chroma components (DUAL_TREE_CHROMA) are currently processed.

In this process, the luma quantization parameter Qp'Y and the chroma quantization parameters Qp'Cb, Qp'Cr and Qp'CbCr are derived.

The luma location (xQg, yQg), specifies the top-left luma sample of the current quantization group relative to the top left luma sample of the current picture. The horizontal and vertical positions xQg and yQg are set equal to CuQgTopLeftX and CuQgTopLeftY, respectively.

NOTE—: The current quantization group is a rectangular region inside a coding tree block that shares the same qPY_PRED. Its width and height are equal to the width and height of the coding tree node of which the top-left luma sample position is assigned to the variables CuQgTopLeftX and CuQgTopLeftY.

When treeType is equal to SINGLE_TREE or DUAL_TREE_LUMA, the predicted luma quantization parameter qPY_PRED is derived by the following ordered steps:

1. The variable qPY_PREV is derived as follows: If one or more of the following conditions are true, qPY_PREV is set equal to SliceQpY: The current quantization group is the first quantization group in a slice. The current quantization group is the first quantization group in a tile. The current quantization group is the first quantization group in a CTB row of a tile and entropy_coding_sync_enabled_flag is equal to 1. Otherwise, qPY_PREV is set equal to the luma quantization parameter QpY of the last luma coding unit in the previous quantization group in decoding order.

2. The derivation process for neighbouring block availability as specified in clause 6.4.4 is invoked with the location (xCurr, yCurr) set equal to (xCb, yCb), the neighbouring location (xNbY, yNbY) set equal to (xQg-1, yQg), check PredModeY set equal to FALSE, and cIdx set equal to 0 as inputs, and the output is assigned to available A. The variable qPY_A is derived as follows: If one or more of the following conditions are true, qPY_A is set equal to qPY_PREV: available A is equal to FALSE. The CTB containing the luma coding block covering the luma location (xQg-1, yQg) is not equal to the CTB containing the current luma coding block at (xCb, yCb), i.e. all of the following conditions are true:

$(xQg-1) >> CtbLog2SizeY$ is not equal to $(xCb) >> CtbLog2SizeY$

$(yQg) >> CtbLog2SizeY$ is not equal to $(yCb) >> CtbLog2SizeY$ Otherwise, qPY_A is set equal to the luma quantization parameter QpY of the coding unit containing the luma coding block covering (xQg-1, yQg).

3. The derivation process for neighbouring block availability as specified in clause 6.4.4 is invoked with the location (xCurr, yCurr) set equal to (xCb, yCb), the neighbouring location (xNbY, yNbY) set equal to (xQg, yQg-1), checkPredModeY set equal to FALSE, and cIdx set equal to 0 as inputs, and the output is assigned to available B. The variable qPY_B is derived as follows: If one or more of the following conditions are true, qPY_B is set equal to qPY_PREV: availableB is equal to FALSE. The CTB containing the luma coding block covering the luma location (xQg, yQg-1) is not equal to the CTB containing the current luma coding block at (xCb, yCb), i.e. all of the following conditions are true:

$(xQg) >> CtbLog2SizeY$ is not equal to $(xCb) >> CtbLog2SizeY$

$(yQg-1) >> CtbLog2SizeY$ is not equal to $(yCb) >> CtbLog2SizeY$ Otherwise, qPY_B is set equal to the luma quantization parameter QpY of the coding unit containing the luma coding block covering (xQg, yQg-1).

4. The predicted luma quantization parameter qPY_PRED is derived as follows: If all the following conditions are true, then qPY_PRED is set equal to the luma quantization parameter QpY of the coding unit containing the luma coding block covering (xQg, yQg-1): availableB is equal to TRUE. The current quantization group is the first quantization group in a CTB row within a tile. Otherwise, qPY_PRED is derived as follows:

$$qPY_PRED = (qPY_A + qPY_B + 1) >> 1 \quad (1115)$$

The variable QpY is derived as follows:

$$QpY=((qPY_PRED+CuQpDeltaVal+64+2*QpBdOffset)\%(64+QpBdOffset))-QpBdOffset \quad (1116)$$

The luma quantization parameter Qp'Y is derived as follows:

$$Qp'Y=QpY+QpBdOffset \quad (1117)$$

When ChromaArrayType is not equal to 0 and treeType is equal to SINGLE_TREE or DUAL_TREE_CHROMA, the following applies: When treeType is equal to DUAL_TREE_CHROMA, the variable QpY is set equal to the luma quantization parameter QpY of the luma coding unit that covers the luma location (xCb+cbWidth/2, yCb+cbHeight/2). The variables qPCb, qPCr and qPCbCr are derived as follows:

$$qPChroma=Clip3(-QpBdOffset,63,QpY) \quad (1118)$$

$$qPCb=ChromaQpTable[0][qPChroma] \quad (1119)$$

$$qPCr=ChromaQpTable[1][qPChroma] \quad (1120)$$

$$qPCbCr=ChromaQpTable[2][qPChroma] \quad (1121)$$

The chroma quantization parameters for the Cb and Cr components, Qp'Cb and Qp'Cr, and joint Cb-Cr coding Qp'CbCr are derived as follows:

$$Qp'Cb=Clip3(-QpBdOffset,63,qPCb+pps_cb_qp_offset+slice_cb_qp_offset+CuQpOffsetCb)+QpBdOffset \quad (1122)$$

$$Qp'Cr=Clip3(-QpBdOffset,63,qPCr+pps_cr_qp_offset+slice_cr_qp_offset+CuQpOffsetCr)+QpBdOffset \quad (1123)$$

$$Qp'CbCr=Clip3(-QpBdOffset,63,qPCbCr+pps_joint_cbcr_qp_offset+slice_joint_cbcr_qp_offset+CuQpOffsetCbCr)+QpBdOffset \quad (1124)$$

2.9 Adaptive Color Transform (ACT)

(111) FIG. 15 illustrates the decoding flowchart with the ACT be applied. As illustrated in FIG. 15, the color space conversion is carried out in residual domain. Specifically, one additional decoding module, namely inverse ACT, is introduced after inverse transform to convert the residuals from YCgCo domain back to the original domain.

(112) In the VVC, unless the maximum transform size is smaller than the width or height of one coding unit (CU), one CU leaf node is also used as the unit of transform processing. Therefore, in the proposed implementation, the ACT flag is signaled for one CU to select the color space for coding its residuals. Additionally, following the HEVC ACT design, for inter and IBC CUs, the ACT is only enabled when there is at least one non-zero coefficient in the CU. For intra CUs, the ACT is only enabled when chroma components select the same intra prediction mode of luma component, e.g., DM mode.

(113) The core transforms used for the color space conversions are kept the same as that used for the HEVC. Specifically, the following forward and inverse YCgCo color transform matrices, as described as follows, as applied.

$$(114) \begin{bmatrix} C'_0 \\ C'_1 \\ C'_2 \end{bmatrix} = \begin{bmatrix} 2 & 1 & 1 \\ 2 & -1 & -1 \\ 0 & -2 & 2 \end{bmatrix} \begin{bmatrix} C_0 \\ C_1 \\ C_2 \end{bmatrix} / 4 \quad \begin{bmatrix} C_0 \\ C_1 \\ C_2 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 0 \\ 1 & -1 & -1 \\ 1 & -1 & 1 \end{bmatrix} \begin{bmatrix} C'_0 \\ C'_1 \\ C'_2 \end{bmatrix}$$

Additionally, to compensate the dynamic range change of residuals signals before and after color transform, the QP adjustments of (-5, -5, -3) are applied to the transform residuals.

On the other hand, as shown in (1), the forward and inverse color transforms need to access the residuals of all three components. Correspondingly, in the proposed implementation, the ACT is disabled in the following two scenarios where not all residuals of three components are available. 1. Separate-tree partition: when separate-tree is applied, luma and chroma samples inside one CTU are partitioned by different structures. This results in that the CUs in the luma-tree only contains luma component and the CUs in the chroma-tree only contains two chroma components.

Intra sub-partition prediction (ISP): the ISP sub-partition is only applied to luma while chroma signals are coded without splitting. In the current ISP design, except the last ISP sub-partitions, the other sub-partitions only contain luma component.

2.10 Example High-Level Deblocking Control

(115) The control mechanism is shown in the following FIG. 16.

3. Drawbacks of Existing Implementations

(116) DMVR and BIO do not involve the original signal during refining the motion vectors, which may result in coding blocks with inaccurate motion information. Also, DMVR and BIO sometimes employ the fractional motion vectors after the motion refinements while screen videos usually have integer motion vectors, which makes the current motion information more inaccurate and make the coding performance worse. 1. The interaction between chroma QP table and chroma deblocking may have problems, e.g chroma QP table should be applied to individual QP but not weighted sum of QPs. 2. The logic of luma deblocking filtering process is complicated for hardware design. 3. The logic of boundary strength derivation is too complicated for both software and hardware design. 4. In the BS decision process, JCCR is treated separately from those blocks coded without JCCT applied. However, JCCR is only a special way to code the residual. Therefore, such design may bring additional complexity without no clear benefits. 5. In chroma edge decision, Qp.sub.Q and Qp.sub.P are set equal to the Qp.sub.Y values of the coding units which include the coding blocks containing the sample q.sub.0,0 and p.sub.0,0, respectively. However, in the quantization/de-quantization process, the QP for a chroma sample is derived from the QP of luma block covering the corresponding luma sample of the center position of current chroma CU. When dual tree is enabled, the different locations of luma blocks may result in different QPs. Therefore, in the chroma deblocking process, wrong QPs may be used for filter decision. Such a misalignment may result in visual artifacts. An example is shown in FIGS. 9A-B. FIG. 9A shows the corresponding CTB partitioning for luma block and FIG. 9B shows the chroma CTB partitioning under dual tree. When determining the QP for chroma block, denoted by CU.sub.C1, the center position of CU.sub.C1 is firstly derived. Then the corresponding luma sample of the center position of CU.sub.C1 is identified and luma QP associated with the luma CU that covers the corresponding luma sample, i.e., CU.sub.Y3 is then utilized to derive the QP for CU.sub.c1. However, when making filter decisions for the depicted three samples (with solid circles), the QPs of CUs that cover the corresponding 3 samples are selected. Therefore, for the 1.sup.st, 2.sup.nd, and 3.sup.rd chroma sample (depicted in FIG. 9B), the QPs of CU.sub.Y2, CU.sub.Y3, CU.sub.Y4 are utilized, respectively. That is, chroma samples in the same CU may use different QPs for filter decision which may result in wrong decisions. 6. A different picture level QP offset (i.e., pps_joint_cbcr_qp_offset) is applied to JCCR coded blocks which is different from the picture level offsets for Cb/Cr (e.g., pps_cb_qp_offset and pps_cr_qp_offset) applied to non-JCCR coded blocks. However, in the chroma deblocking filter decision process, only those offsets for non-JCCR coded blocks are utilized. The missing of consideration of coded modes may result in wrong filter decision. 7. The TS and non-TS coded blocks employ different QPs in the de-quantization process, which may be also considered in the deblocking process. 8. Different QPs are used in the scaling process (quantization/dequantization) for JCCR coded blocks with different modes. Such a design is not consistent. 9. The chroma deblocking for Cb/Cr could be unified for parallel design. 10. The chroma QP in the deblocking is derived based on the QP used in the chroma dequantization process (e.g. qP), however, the qP should be clipped or minus 5 for TS and ACT blocks when it is used in the deblocking process. 11. The prediction process for three components may be not same when both BDPCM and ACT are enabled.

4. Example Techniques and Embodiments

(117) The detailed embodiments described below should be considered as examples to explain general concepts. These embodiments should not be interpreted narrowly way. Furthermore, these embodiments can be combined in any manner.

(118) The methods described below may be also applicable to other decoder motion information derivation technologies in addition to the DMVR and BIO mentioned below.

(119) In the following examples, MVM[i].x and MVM[i].y denote the horizontal and vertical component of the motion vector in reference picture list i (i being 0 or 1) of the block at M (M being P or Q) side. Abs denotes the operation to get the absolute value of an input, and "&&" and "||" denotes the logical operation AND and OR. Referring to FIG. 10, P may denote the samples at P side and Q may denote the samples at Q side. The blocks at

P side and Q side may denote the block marked by the dash lines.

(120) Regarding Chroma QP in Deblocking

(121) 1. When chroma QP table is used to derive parameters to control chroma deblocking (e.g., in the decision process for chroma block edges), chroma QP offsets may be applied after applying chroma QP table. a. In one example, the chroma QP offsets may be added to the value outputted by a chroma QP table. b. Alternatively, the chroma QP offsets may be not considered as input to a chroma QP table. c. In one example, the chroma QP offsets may be the picture-level or other video unit-level (slice/tile/brick/subpicture) chroma quantization parameter offset (e.g., pps_cb_qp_offset, pps_cr_qp_offset in the specification). 2. QP clipping may be not applied to the input of a chroma QP table. 3. It is proposed that deblocking process for chroma components may be based on the mapped chroma QP (by the chroma QP table) on each side. a. In one example, it is proposed that deblocking parameters, (e.g., β and tC) for chroma may be based on QP derived from luma QP on each side. b. In one example, the chroma deblocking parameter may depend on chroma QP table value with QpP as the table index, where QpP, is the luma QP value on P-side. c. In one example, the chroma deblocking parameter may depend on chroma QP table value with QpQ as the table index, where QpQ, is the luma QP value on Q-side. 4. It is proposed that deblocking process for chroma components may be based on the QP applied to quantization/dequantization for the chroma block. a. In one example, QP for deblocking process may be equal to the QP in dequantization. b. In one example, the QP selection for deblocking process may depend on the indication of usage of TS and/or ACT blocks. i. In one example, the QP for the deblocking process may be derived by $\text{Max}(\text{QpPrimeTsMin}, \text{qP}) - (\text{cu_act_enabled_flag}[\text{xTbY}][\text{yTbY}]? \text{N}:0)$, where QpPrimeTsMin is the minimal QP for TS blocks and cu_act_enabled_flag is the flag of usage of ACT. 1. In one example, qP may be the Qp'.sub.Cb or Qp'.sub.Cr given in section 2.8. ii. In one example, the QP for the deblocking process may be derived by $\text{Max}(\text{QpPrimeTsMin}, \text{qP} - (\text{cu_act_enabled_flag}[\text{xTbY}][\text{yTbY}]? \text{N}:0))$, where QpPrimeTsMin is the minimal QP for TS blocks and cu_act_enabled_flag is the flag of usage of ACT. 1. In one example, qP may be the Qp'.sub.Cb or Qp'.sub.Cr given in section 2.8. iii. In the above examples, N may be set to same or different values for each color component. 1. In one example, N may be set to 5 for the Cb/B/G/U/component and/or N may be set to 3 for the Cr/R/B/V component. c. In one example, the QP of a block used in the deblocking process may be derived by the process described in section 2.8 with delta QP (e.g. CuQpDeltaVal) equal to 0. iv. In one example, the above derivation may be applied only when the coded block flag (cbf) of a block is equal to 0. d. In one example, the above examples may be applied on luma and/or chroma blocks. e. In one example, the QP of a first block used in the deblocking process may be set equal to the QP stored and utilized for predicting a second block. v. In one example, for a block with all zero coefficients, the associated QP used in the deblocking process may be set equal to the QP stored and utilized for predicting a second block. 5. It is proposed to consider the picture/slice/tile/brick/subpicture level quantization parameter offsets used for different coding methods in the deblocking filter decision process. a. In one example, selection of picture/slice/tile/brick/subpicture level quantization parameter offsets for filter decision (e.g., the chroma edge decision in the deblocking filter process) may depend on the coded methods for each side. b. In one example, the filtering process (e.g., the chroma edge decision process) which requires to use the quantization parameters for chroma blocks may depend on whether the blocks use JCCR. i. Alternatively, furthermore, the picture/slice-level QP offsets (e.g., pps_joint_cbr_qp_offset) applied to JCCR coded blocks may be further taken into consideration in the deblocking filtering process. ii. In one example, the cQpPicOffset which is used to decide Tc and β settings may be set to pps_joint_cbr_qp_offset instead of pps_cb_qp_offset or pps_cr_qp_offset under certain conditions: 1. In one example, when either block in P or Q sides uses JCCR. 2. In one example, when both blocks in P or Q sides uses JCCR. iii. Alternatively, furthermore, the filtering process may depend on the mode of JCCR (e.g., whether mode is equal to 2). 6. The chroma filtering process (e.g., the chroma edge decision process) which requires to access the decoded information of a luma block may utilize the information associated with the same luma coding block that is used to derive the chroma QP in the dequantization/quantization process. a. In one example, the chroma filtering process (e.g., the chroma edge decision process) which requires to use the quantization parameters for luma blocks may utilize the luma coding unit covering the corresponding luma sample of the center position of current chroma CU. b. An example is depicted in FIGS. 9A-B wherein the decoded information of CU.sub.Y3 may be used for filtering decision of the three chroma samples (1.sup.st, 2.sup.nd and 3.sup.rd) in FIG. 9B. 7. The chroma filtering process (e.g., the chroma edge decision process) may depend on the quantization parameter applied to the scaling process of the chroma block (e.g., quantization/dequantization). a. In one example, the QP used to derive β and Tc may depend on the QP applied to the scaling process of the chroma block. b. Alternatively, furthermore, the QP used to the scaling process of the chroma block may have taken the chroma CU level QP offset into consideration. 8. Whether to invoke above bullets may depend on the sample to be filtered is in the block at P or Q side. a. For example, whether to use the information of the luma coding block covering the corresponding luma sample of current chroma sample or use the information of the luma coding block covering the corresponding luma sample of center position of chroma coding block covering current chroma sample may depend on the block position. i. In one example, if the current chroma sample is in the block at the Q side, QP information of the luma coding block covering the corresponding luma sample of center position of chroma coding block covering current chroma sample may be used. ii. In one example, if the current chroma sample is in the block at the P side, QP information of the luma coding block covering the corresponding luma sample of the chroma sample may be used. 9. Chroma QP used in deblocking may depend on information of the corresponding transform block. a. In one example, chroma QP for deblocking at P-side may depend on the transform block's mode at P-side. i. In one example, chroma QP for deblocking at P-side may depend on if the transform block at P-side is coded with JCCR applied. ii. In one example, chroma QP for deblocking at P-side may depend on if the transform block at P-side is coded with joint_cb_cr mode and the mode of JCCR is equal to 2. b. In one example, chroma QP for deblocking at Q-side may depend on the transform block's mode at Q-side. i. In one example, chroma QP for deblocking at Q-side may depend on if the transform block at Q-side is coded with JCCR applied. ii. In one example, chroma QP for deblocking at Q-side may depend on if the transform block at Q-side is coded with JCCR applied and the mode of JCCR is equal to 2. 10. Signaling of chroma QPs may be in coding unit. a. In one example, when coding unit size is larger than the maximum transform block size, i.e., maxTB, chroma QP may be signaled in CU level. Alternatively, it may be signaled in TU level. b. In one example, when coding unit size is larger than the size of VPDU, chroma QP may be signaled in CU level. Alternatively, it may be signaled in TU level. 11. Whether a block is of joint_cb_cr mode may be indicated at coding unit level. a. In one example, whether a transform block is of joint_cb_cr mode may inherit the information of the coding unit containing the transform block. 12. Chroma QP used in deblocking may depend on chroma QP used in scaling process minus QP offset due to bit depth. a. In one example, Chroma QP used in deblocking at P-side is set to the JCCR chroma QP used in scaling process, i.e. Qp'.sub.CbCr, minus QpBdOffsetC when TuCResMode[xTb][yTb] is equal to 2 where (xTb, yTb) denotes the transform blocking containing the first sample at P-side, i.e. p.sub.0,0. b. In one example, Chroma QP used in deblocking at P-side is set to the Cb chroma QP used in scaling process, i.e. Qp'.sub.Cb, minus QpBdOffsetC when TuCResMode[xTb][yTb] is equal to 2 where (xTb, yTb) denotes the transform blocking containing the first sample at P-side, i.e. p.sub.0,0. c. In one example, Chroma QP used in deblocking at P-side is set to the Cr chroma QP used in scaling process, i.e. Qp'.sub.Cr, minus QpBdOffsetC when TuCResMode[xTb][yTb] is equal to 2 where (xTb, yTb) denotes the transform blocking containing the first sample at P-side, i.e. p.sub.0,0. d. In one example, Chroma QP used in deblocking at Q-side is set to the JCCR chroma QP used in scaling process, i.e. Qp'.sub.CbCr, minus QpBdOffsetC when TuCResMode[xTb][yTb] is equal to 2 where (xTb, yTb) denotes the transform blocking containing the last sample at Q-side, i.e. q.sub.0,0. e. In one example, Chroma QP used in deblocking at Q-side is set to the Cb chroma QP used in scaling process, i.e. Qp'.sub.Cb, minus QpBdOffsetC when TuCResMode[xTb][yTb] is equal to 2 where (xTb, yTb) denotes the transform blocking containing the last sample at Q-side, i.e. q.sub.0,0. f. In one example, Chroma QP used in deblocking at Q-side is set to the Cr chroma QP used in scaling process, i.e. Qp'.sub.Cr, minus QpBdOffsetC when TuCResMode[xTb][yTb] is equal to 2 where (xTb, yTb) denotes the transform blocking containing the last sample at Q-side, i.e. q.sub.0,0. 13. Different color components may have different deblocking strength control. a. In one example, each component may have its pps_beta_offset_div2, pps_tc_offset_div2 and/or pic_beta_offset_div2, pic_tc_offset_div2 and/or slice_beta_offset_div2, slice_tc_offset_div2. b. In one example, for joint_cb_cr mode, a different set of beta_offset_div2, tc_offset_div2 may be applied in PPS and/or picture header and/or slice header. 14. Instead of using overriding mechanism, the deblocking control offset may be

accumulated taking account of offsets at different levels. a. In one example, pps_beta_offset_div2 and/or pic_beta_offset_div2 and/or slice_beta_offset_div2 may be accumulated to get the deblocking offset at slice level. b. In one example, pps_tc_offset_div2 and/or pic_tc_offset_div2 and/or slice_tc_offset_div2 may be accumulated to get the deblocking offset at slice level.

Regarding QP Settings 15. It is proposed to signal the indication of enabling block-level chroma QP offset (e.g. slice_cu_chroma_qp_offset_enabled_flag) at the slice/tile/brick/subpicture level. a. Alternatively, the signaling of such an indication may be conditionally signaled. i. In one example, it may be signaled under the condition of JCCR enabling flag. ii. In one example, it may be signaled under the condition of block-level chroma QP offset enabling flag in picture level. iii. Alternatively, such an indication may be derived instead. b. In one example, the slice_cu_chroma_qp_offset_enabled_flag may be signaled only when the PPS flag of chroma QP offset (e.g. slice_cu_chroma_qp_offset_enabled_flag) is true. c. In one example, the slice_cu_chroma_qp_offset_enabled_flag may be inferred to false only when the PPS flag of chroma QP offset (e.g. slice_cu_chroma_qp_offset_enabled_flag) is false. d. In one example, whether to use the chroma QP offset on a block may be based on the flags of chroma QP offset at PPS level and/or slice level. 16. Same QP derivation method is used in the scaling process (quantization/dequantization) for JCCR coded blocks with different modes. a. In one example, for JCCR with mode 1 and 3, the QP is dependent on the QP offset signaled in the picture/slice level (e.g., pps_cbr_qp_offset, slice_cbr_qp_offset).

Filtering Procedures 17. Deblocking for all color components excepts for the first color component may follow the deblocking process for the first color component. a. In one example, when the color format is 4:4:4, deblocking process for the second and third components may follow the deblocking process for the first component. b. In one example, when the color format is 4:4:4 in RGB color space, deblocking process for the second and third components may follow the deblocking process for the first component. c. In one example, when the color format is 4:2:2, vertical deblocking process for the second and third components may follow the vertical deblocking process for the first component. d. In above examples, the deblocking process may refer to deblocking decision process and/or deblocking filtering process. 18. How to calculate gradient used in the deblocking filter process may depend on the coded mode information and/or quantization parameters. a. In one example, the gradient computation may only consider the gradient of a side wherein the samples at that side are not lossless coded. b. In one example, if both sides are lossless coded or nearly lossless coded (e.g., quantization parameters equal to 4), gradient may be directly set to 0. i. Alternatively, if both sides are lossless coded or nearly lossless coded (e.g., quantization parameters equal to 4), Boundary Strength (e.g., BS) may be set to 0. c. In one example, if the samples at P side are lossless coded and the samples at Q side are lossy coded, the gradients used in deblocking on/off decision and/or strong filters on/off decision may only include gradients of the samples at Q side, vice versa. i. Alternatively, furthermore, the gradient of one side may be scaled by N. 1. N is an integer number (e.g. 2) and may depend on a. Video contents (e.g. screen contents or natural contents) b. A message signaled in the DPS/SPS/VPS/PPS/APS/picture header/slice header/tile group header/Largest coding unit (LCU)/Coding unit (CU)/LCU row/group of LCUs/TU/PU block/Video coding unit c. Position of CU/PU/TU/block/Video coding unit d. Coded modes of blocks containing the samples along the edges e. Transform matrices applied to the blocks containing the samples along the edges f. Block dimension/Block shape of current block and/or its neighboring blocks g. Indication of the color format (such as 4:2:0, 4:4:4, RGB or YUV) h. Coding tree structure (such as dual tree or single tree) i. Slice/tile group type and/or picture type j. Color component (e.g. may be only applied on Cb or Cr) k. Temporal layer ID l. Profiles/Levels/Tiers of a standard m. Alternatively, N may be signalled to the decoder

Regarding Boundary Strength Derivation 19. It is proposed to treat JCCR coded blocks as those non-JCCR coded blocks in the boundary strength decision process. a. In one example, the determination of boundary strength (BS) may be independent from the checking of usage of JCCR for two blocks at P and Q sides. b. In one example, the boundary strength (BS) for a block may be determined regardless if the block is coded with JCCR or not. 20. It is proposed to derive the boundary strength (BS) without comparing the reference pictures and/or number of Ms associated with the block at P side with the reference pictures of the block at Q side. a. In one example, deblocking filter may be disabled even when two blocks are with different reference pictures. b. In one example, deblocking filter may be disabled even when two blocks are with different number of MVs (e.g., one is uni-predicted and the other is bi-predicted). c. In one example, the value of BS may be set to 1 when motion vector differences for one or all reference picture lists between the blocks at P side and Q side is larger than or equal to a threshold Th. i. Alternatively, furthermore, the value of BS may be set to 0 when motion vector differences for one or all reference picture lists between the blocks at P side and Q side is smaller than or equal to a threshold Th. d. In one example, the difference of the motion vectors of two blocks being larger than a threshold Th may be defined as $(\text{Abs}(\text{MVP}[0].x - \text{MVQ}[0].x) > \text{Th} \parallel \text{Abs}(\text{MVP}[0].y - \text{MVQ}[0].y) > \text{Th}) \parallel (\text{Abs}(\text{MVP}[1].x - \text{MVQ}[1].x) > \text{Th}) \parallel (\text{Abs}(\text{MVP}[1].y - \text{MVQ}[1].y) > \text{Th})$ ii. Alternatively, the difference of the motion vectors of two blocks being larger than a threshold Th may be defined as $(\text{Abs}(\text{MVP}[0].x - \text{MVQ}[0].x) > \text{Th} \&\& \text{Abs}(\text{MVP}[0].y - \text{MVQ}[0].y) > \text{Th} \&\& \text{Abs}(\text{MVP}[1].x - \text{MVQ}[1].x) > \text{Th}) \&\& \text{Abs}(\text{MVP}[1].y - \text{MVQ}[1].y) > \text{Th})$ iii. Alternatively, in one example, the difference of the motion vectors of two blocks being larger than a threshold Th may be defined as $(\text{Abs}(\text{MVP}[0].x - \text{MVQ}[0].x) > \text{Th} \parallel \text{Abs}(\text{MVP}[0].y - \text{MVQ}[0].y) > \text{Th}) \&\& (\text{Abs}(\text{MVP}[1].x - \text{MVQ}[1].x) > \text{Th}) \parallel \text{Abs}(\text{MVP}[1].y - \text{MVQ}[1].y) > \text{Th})$ iv. Alternatively, in one example, the difference of the motion vectors of two blocks being larger than a threshold Th may be defined as $(\text{Abs}(\text{MVP}[0].x - \text{MVQ}[0].x) > \text{Th} \&\& \text{Abs}(\text{MVP}[0].y - \text{MVQ}[0].y) > \text{Th}) \parallel ((\text{Abs}(\text{MVP}[1].x - \text{MVQ}[1].x) > \text{Th}) \&\& \text{Abs}(\text{MVP}[1].y - \text{MVQ}[1].y) > \text{Th})$ e. In one example, a block which does not have a motion vector in a given list may be treated as having a zero-motion vector in that list. f. In the above examples, Th is an integer number (e.g. 4, 8 or 16). g. In the above examples, Th may depend on v. Video contents (e.g. screen contents or natural contents) vi. A message signaled in the DPS/SPS/VPS/PPS/APS/picture header/slice header/tile group header/Largest coding unit (LCU)/Coding unit (CU)/LCU row/group of LCUs/TU/PU block/Video coding unit vii. Position of CU/PU/TU/block/Video coding unit viii. Coded modes of blocks containing the samples along the edges ix. Transform matrices applied to the blocks containing the samples along the edges x. Block dimension/Block shape of current block and/or its neighboring blocks xi. Indication of the color format (such as 4:2:0, 4:4:4, RGB or YUV) xii. Coding tree structure (such as dual tree or single tree) xiii. Slice/tile group type and/or picture type xiv. Color component (e.g. may be only applied on Cb or Cr) xv. Temporal layer ID xvi. Profiles/Levels/Tiers of a standard xvii. Alternatively, Th may be signalled to the decoder. h. The above examples may be applied under certain conditions. xviii. In one example, the condition is the blkP and blkQ are not coded with intra modes. xix. In one example, the condition is the blkP and blkQ have zero coefficients on luma component. xx. In one example, the condition is the blkP and blkQ are not coded with the CIIP mode. xxi. In one example, the condition is the blkP and blkQ are coded with a same prediction mode (e.g. IBC or Inter).

Regarding Luma Deblocking Filtering Process 21. The deblocking may use different QPs for TS coded blocks and non-TS coded blocks. a. In one example, the QP for TS may be used on TS coded blocks while the QP for non-TS may be used on non-TS coded blocks. 22. The luma filtering process (e.g., the luma edge decision process) may depend on the quantization parameter applied to the scaling process of the luma block. a. In one example, the QP used to derive beta and Tc may depend on the clipping range of transform skip, e.g. as indicated by QpPrimeTsMin. 23. It is proposed to use an identical gradient computation for large block boundaries and smaller block boundaries. a. In one example, the deblocking filter on/off decision described in section 2.1.4 may be also applied for large block boundary. i. In one example, the threshold beta in the decision may be modified for large block boundary. 1. In one example, beta may depend on quantization parameter. 2. In one example, beta used for deblocking filter on/off decision for large block boundaries may be smaller than that for smaller block boundaries. a. Alternatively, in one example, beta used for deblocking filter on/off decision for large block boundaries may be larger than that for smaller block boundaries. b. Alternatively, in one example, beta used for deblocking filter on/off decision for large block boundaries may be equal to that for smaller block boundaries. 3. In one example, beta is an integer number and may be based on a. Video contents (e.g. screen contents or natural contents) b. A message signaled in the DPS/SPS/VPS/PPS/APS/picture header/slice header/tile group header/Largest coding unit (LCU)/Coding unit (CU)/LCU row/group of LCUs/TU/PU block/Video coding unit c. Position of CU/PU/TU/block/Video coding unit d. Coded modes of blocks containing the samples along the edges e. Transform matrices applied to the blocks containing the samples along the edges f. Block dimension of current block and/or its neighboring blocks g. Block shape of current block and/or its neighboring blocks h. Indication of the color format (such as 4:2:0, 4:4:4, RGB or YUV) i.

coding tree structure (such as dual tree or single tree) j. Slice/tile group type and/or picture type k. Color component (e.g. may be only applied on Cb or Cr) l. Temporal layer ID m. Profiles/Levels/Tiers of a standard n. Alternatively, beta may be signalled to the decoder.

Regarding Scaling Matrix (Dequantization Matrix) 24. The values for specific positions of quantization matrices may be set to constant. a. In one example, the position may be the position of (x, y) wherein x and y are two integer variables (e.g., $x=y=0$), and (x, y) is the coordinate relative to a TU/TB/PU/PB/CU/CB. i. In one example, the position may be the position of DC. b. In one example, the constant value may be 16. c. In one example, for those positions, signaling of the matrix values may not be utilized. 25. A constrain may be set that the average/weighted average of some positions of quantization matrices may be a constant. a. In one example, deblocking process may depend on the constant value. b. In one example, the constant value may be indicated in DPS/VPS/SPS/PPS/Tile/Picture/Brick headers. 26. One or multiple indications may be signaled in the picture header to inform the scaling matrix to be selected in the picture associated with the picture header.

Regarding Cross Component Adaptive Loop Filter (CCALF) 27. CCALF may be applied before some loop filtering process at the decoder a. In one example, CCALF may be applied before deblocking process at the decoder. b. In one example, CCALF may be applied before SAO at the decoder. c. In one example, CCALF may be applied before ALF at the decoder. d. Alternatively, the order of different filters (e.g., CCALF, ALF, SAO, deblocking filter) may be NOT fixed. i. In one example, the invoke of CCALF may be before one filtering process for one video unit or after another one for another video unit. ii. In one example, the video unit may be a CTU/CTB/slice/tile/brick/picture/sequence. e. Alternatively, indications of the order of different filters (e.g., CCALF, ALF, SAO, deblocking filter) may be signaled or derived on-the-fly. i. Alternatively, indication of the invoking of CCALF may be signaled or derived on-the-fly. f. The explicit (e.g. signaling from the encoder to the decoder) or implicit (e.g. derived at both encoder and decoder) indications of how to control CCALF may be decoupled for different color components (such as Cb and Cr). g. Whether and/or how to apply CCALF may depend on color formats (such as RGB and YCbCr) and/or color sampling format (such as 4:2:0, 4:2:2 and 4:4:4), and/or color down-sampling positions or phases)

Regarding Chroma QP Offset Lists 28. Signaling and/or selection of chroma QP offset lists may be dependent on the coded prediction modes/picture types/slice or tile or brick types. h. Chroma QP offset lists, e.g. `cb_qp_offset_list[i]`, `cr_qp_offset_list[i]`, and `joint_cbrp_qp_offset_list[i]`, may be different for different coding modes. i. In one example, whether and how to apply chroma QP offset lists may depend on whether the current block is coded in intra mode. j. In one example, whether and how to apply chroma QP offset lists may depend on whether the current block is coded in inter mode. k. In one example, whether and how to apply chroma QP offset lists may depend on whether the current block is coded in palette mode. l. In one example, whether and how to apply chroma QP offset lists may depend on whether the current block is coded in IBC mode. m. In one example, whether and how to apply chroma QP offset lists may depend on whether the current block is coded in transform skip mode. n. In one example, whether and how to apply chroma QP offset lists may depend on whether the current block is coded in BDPCM mode. o. In one example, whether and how to apply chroma QP offset lists may depend on whether the current block is coded in transform_quant_skip or lossless mode.

Regarding Chroma Deblocking at CTU Boundary 29. How to select the QPs (e.g., using corresponding luma or chroma dequantized QP) utilized in the deblocking filter process may be dependent on the position of samples relative to the CTU/CTB/VPDU boundaries. 30. How to select the QPs (e.g., using corresponding luma or chroma dequantized QP) utilized in the deblocking filter process may depend on color formats (such as RGB and YCbCr) and/or color sampling format (such as 4:2:0, 4:2:2 and 4:4:4), and/or color down-sampling positions or phases). 31. For edges at CTU boundary, the deblocking may be based on luma QP of the corresponding blocks. p. In one example, for horizontal edges at CTU boundary, the deblocking may be based on luma QP of the corresponding blocks. i. In one example, the deblocking may be based on luma QP of the corresponding blocks at P-side. ii. In one example, the deblocking may be based on luma QP of the corresponding blocks at Q-side. q. In one example, for vertical edges at CTU boundary, the deblocking may be based on luma QP of the corresponding blocks. i. In one example, the deblocking may be based on luma QP of the corresponding blocks at P-side. ii. In one example, the deblocking may be based on luma QP of the corresponding blocks at Q-side. r. In one example, for edges at CTU boundary, the deblocking may be based on luma QP at P-side and chroma QP at Q-side. s. In one example, for edges at CTU boundary, the deblocking may be based on luma QP at Q-side and chroma QP at P-side. t. In this bullet, “CTU boundary” may refer to a specific CTU boundary such as the upper CTU boundary or the lower CTU boundary. 32. For horizontal edges at CTU boundary, the deblocking may be based on a function of chroma QPs at P-side. u. In one example, the deblocking may be based on an averaging function of chroma QPs at P-side. i. In one example, the function may be based on the average of the chroma QPs for each 8 luma samples. ii. In one example, the function may be based on the average of the chroma QPs for each 16 luma samples. iii. In one example, the function may be based on the average of the chroma QPs for each 32 luma samples. iv. In one example, the function may be based on the average of the chroma QPs for each 64 luma samples. v. In one example, the function may be based on the average of the chroma QPs for each CTU. v. In one example, the deblocking may be based on a maximum function of chroma QPs at P-side. i. In one example, the function may be based on the maximum of the chroma QPs for each 8 luma samples. ii. In one example, the function may be based on the maximum of the chroma QPs for each 16 luma samples. iii. In one example, the function may be based on the maximum of the chroma QPs for each 32 luma samples. iv. In one example, the function may be based on the maximum of the chroma QPs for each 64 luma samples. v. In one example, the function may be based on the maximum of the chroma QPs for each CTU. w. In one example, the deblocking may be based on a minimum function of chroma QPs at P-side. i. In one example, the function may be based on the minimum of the chroma QPs for each 8 luma samples. ii. In one example, the function may be based on the minimum of the chroma QPs for each 16 luma samples. iii. In one example, the function may be based on the minimum of the chroma QPs for each 32 luma samples. iv. In one example, the function may be based on the minimum of the chroma QPs for each 64 luma samples. v. In one example, the function may be based on the minimum of the chroma QPs for each CTU. x. In one example, the deblocking may be based on a subsampling function of chroma QPs at P-side. i. In one example, the function may be based on the chroma QPs of the k-th chroma sample for each 8 luma samples. 1. In one example, the k-th sample may be the first sample. 2. In one example, the k-th sample may be the last sample. 3. In one example, the k-th sample may be the third sample. 4. In one example, the k-th sample may be the fourth sample. ii. In one example, the function may be based on the chroma QPs of the k-th chroma sample for each 16 luma samples. 1. In one example, the k-th sample may be the first sample. 2. In one example, the k-th sample may be the last sample. 3. In one example, the k-th sample may be the 7-th sample. 4. In one example, the k-th sample may be the 8-th sample. iii. In one example, the function may be based on the chroma QPs of the k-th chroma sample for each 32 luma samples. 1. In one example, the k-th sample may be the first sample. 2. In one example, the k-th sample may be the last sample. 3. In one example, the k-th sample may be the 15-th sample. 4. In one example, the k-th sample may be the 16-th sample. iv. In one example, the function may be based on the chroma QPs of the k-th chroma sample for each 64 luma samples. 1. In one example, the k-th sample may be the first sample. 2. In one example, the k-th sample may be the last sample. 3. In one example, the k-th sample may be the 31-th sample. 4. In one example, the k-th sample may be the 32-th sample. v. In one example, the function may be based on the chroma QPs of the k-th chroma sample for each CTU. y. Alternatively, the above items may be applied to chroma QPs at Q-side for deblocking process. 33. It may be constrained that quantization group for chroma component must be larger than a certain size. A quantization group is a set of (one or more) coding units that carries a quantization parameter. a. In one example, it may be constrained that the width of quantization group for chroma component must be larger than a certain value, K. i. In one example, K is equal to 4. 34. It may be constrained that quantization group for luma component must be larger than a certain size. a. In one example, it may be constrained that the width of quantization group for luma component must be larger than a certain value, K. i. In one example, K is equal to 8. 35. It may be constrained that QP for chroma component may be the same for a chroma row segment with length $4*m$ starting from $(4*m*x, 2y)$ relative to top-left of the picture, where x and y are non-negative integers; and m is a positive integer. z. In one example, m may be equal to 1. aa. In one example, the width of a quantization group for chroma component must be no smaller than $4*m$. 36. It may be constrained that QP for chroma component may be the same for a chroma column segment with length $4*n$ starting from $(2*x, 4*n*y)$ relative to top-left of the picture, where x and y are non-negative integers; and n is a positive integer. bb. In one example, n may be equal to 1. cc. In one example, the height of a quantization group for chroma component must be no smaller than $4*n$.

Regarding Chroma Deblocking Filtering Process 37. A first syntax element controlling the usage of coding tool X may be signalled in a first video unit (such as picture header), depending on a second syntax element signalled in a second video unit (such as SPS or PPS, or VPS). a. In one example, the first syntax element is signalled only if the second syntax element indicates that the coding tool X is enabled. b. In one example, X is Bi-Direction Optical Flow (BDOF). c. In one example, X is Prediction Refinement Optical Flow (PROF). d. In one example, X is Decoder-side Motion Vector Refinement (DMVR). e. In one example, the signalling of the usage of a coding tool X may be under the condition check of slice types (e.g., P or B slices; non-I slices).

Regarding Chroma Deblocking Filtering Process 38. Deblocking filter decision processes for two chroma blocks may be unified to be only invoked once and the decision is applied to two chroma blocks. b. In one example, the decision for whether to perform deblocking filter may be same for Cb and Cr components. c. In one example, if the deblocking filter is determined to be applied, the decision for whether to perform stronger deblocking filter may be same for Cb and Cr components. d. In one example, the deblocking condition and strong filter on/off condition, as described in section 2.2.7, may be only checked once. However, it may be modified to check the information of both chroma components. i. In one example, the average of gradients of Cb and Cr components may be used in the above decisions for both Cb and Cr components. ii. In one example, the chroma stronger filters may be performed only when the strong filter condition is satisfied for both Cb and Cr components. 1. Alternatively, in one example, the chroma weak filters may be performed only when the strong filter condition is not satisfied at least one chroma component

On ACT 39. Whether deblocking QP is equal to dequantization QP may depend on whether ACT is applied. a. In one example, when ACT is applied to a block, deblocking QP values may depend on QP values before ACT QP adjustment. b. In one example, when ACT is not applied to a block, deblocking QP values may always equal to dequantization QP values. c. In one example, when both ACT and TS are not applied to a block, deblocking QP values may always equal to dequantization QP values. 40. ACT and BDPCM may be applied exclusively at block level. a. In one example, when ACT is applied on a block, luma BDPCM shall not be applied on that block. b. In one example, when ACT is applied on a block, chroma BDPCM shall not be applied on that block. c. In one example, when ACT is applied on a block, both luma and chroma BDPCM shall not be applied on that block. d. In one example, when luma and/or chroma BDPCM is applied on a block, ACT shall not be applied on that block. 41.

Whether to enable BDPCM mode may be inferred based on the usage of ACT (e.g. `cu_act_enabled_flag`). a. In one example, the inferred value of chroma BDPCM mode may be defined as (`cu_act_enabled_flag && intra_bdpcm_luma_flag && sps_bdpcm_chroma_enabled_flag`? true: false). b. In one example, if `sps_bdpcm_chroma_enabled_flag` is false, the `intra_bdpcm_luma_flag` may be inferred to false when it is not signalled and `cu_act_enabled_flag` is true. c. In one example, the `cu_act_enabled_flag` may be inferred to false when `intra_bdpcm_luma_flag` is true and `sps_bdpcm_chroma_enabled_flag` is false. d. In one example, the `intra_bdpcm_luma_flag` may be inferred to false when `cu_act_enabled_flag` is true and `sps_bdpcm_chroma_enabled_flag` is false.

General 42. The above proposed methods may be applied under certain conditions. a. In one example, the condition is the colour format is 4:2:0 and/or 4:2:2. i. Alternatively, furthermore, for 4:4:4 colour format, how to apply deblocking filter to the two colour chroma components may follow the current design. b. In one example, indication of usage of the above methods may be signalled in sequence/picture/slice/tile/brick/a video region-level, such as SPS/PPS/picture header/slice header. c. In one example, the usage of above methods may depend on ii. Video contents (e.g. screen contents or natural contents) iii. A message signaled in the DPS/SPS/VPS/PPS/APS/picture header/slice header/tile group header/Largest coding unit (LCU)/Coding unit (CU)/LCU row/group of LCUs/TU/PU block/Video coding unit iv. Position of CU/PU/TU/block/Video coding unit a. In one example, for filtering samples along the CTU/CTB boundaries (e.g., the first K (e.g., K=4/8) to the top/left/right/bottom boundaries), the existing design may be applied. While for other samples, the proposed method (e.g., bullets 3/4) may be applied instead. v. Coded modes of blocks containing the samples along the edges vi. Transform matrices applied to the blocks containing the samples along the edges vii. Block dimension of current block and/or its neighboring blocks viii. Block shape of current block and/or its neighboring blocks ix. Indication of the color format (such as 4:2:0, 4:4:4, RGB or YUV) x. Coding tree structure (such as dual tree or single tree) xi. Slice/tile group type and/or picture type xii. Color component (e.g. may be only applied on Cb or Cr) xiii. Temporal layer ID xiv. Profiles/Levels/Tiers of a standard xv. Alternatively, m and/or n may be signalled to the decoder.

5. Additional Embodiments

(122) The newly added texts are shown in underlined bold italicized font. The deleted texts are marked by `[ [ ] ]`.

5.1. Embodiment #1 on Chroma QP in Deblocking

(123) 8.8.3.6 Edge Filtering Process for One Direction

(124) . . .

(125) Otherwise (`cIdx` is not equal to 0), the filtering process for edges in the chroma coding block of current coding unit specified by `cIdx` consists of the following ordered steps: 1. The variable `cQpPicOffset` is derived as follows:

`cQpPicOffset=cIdx==1?pps_cb_qp_offset:pps_cr_qp_offset` (8-1065)

8.8.3.6.3 Decision Process for Chroma Block Edges

. . .

The variables `Qp.sub.Q` and `Qp.sub.P` are set equal to the `Qp.sub.Y` values of the coding units which include the coding blocks containing the sample `q.sub.0,0` and `p.sub.0,0`, respectively.

The variable `Qp.sub.C` is derived as follows:

`[qPi=Clip3(0,63,((Qp.sub.Q+Qp.sub.P+1)>>1)+cQpPicOffset)` (8-1132)

`Qp.sub.C=ChromaQpTable[cIdx-1][qPi]` (8-1133)]

`qPi=(Qp.sub.Q+Qp.sub.P+1)>>1` (8-1132)

`Qp.sub.C=ChromaQpTable[cIdx-1][qPi]+cQpPicOffset` (8-1133) NOTE—The variable `cQpPicOffset` provides an adjustment for the value of `pps_cb_qp_offset` or `pps_cr_qp_offset`, according to whether the filtered chroma component is the Cb or Cr component. However, to avoid the need to vary the amount of the adjustment within the picture, the filtering process does not include an adjustment for the value of `slice_cb_qp_offset` or `slice_cr_qp_offset` n or (when `cu_chroma_qp_offset_enabled_flag` is equal to 1) for the value of `CuQpOffset.sub.Cb`, `CuQpOffset.sub.Cr`, or `CuQpOffset.sub.CbCr`.

(126) The value of the variable β' is determined as specified in Table 8-18 based on the quantization parameter Q derived as follows:

$Q = \text{Clip3}(0, 63, Qp.sub.C + (\text{slice_beta_offset_div2} < 1))$ (8-1134)

where `slice_beta_offset_div2` is the value of the syntax element `slice_beta_offset_div2` for the slice that contains sample `q.sub.0,0`.

The variable β is derived as follows:

$\beta = \beta' * (1 < (BitDepth.sub.C - 8))$ (8-1135)

The value of the variable `t.sub.C'` is determined as specified in Table 8-18 based on the chroma quantization parameter Q derived as follows:

$Q = \text{Clip3}(0, 65, Qp.sub.C + 2 * (bS - 1) + (\text{slice_tc_offset_div2} < 1))$ (8-1136)

where `slice_tc_offset_div2` is the value of the syntax element `slice_tc_offset_div2` for the slice that contains sample `q.sub.0,0`. The variable `t.sub.C` is derived as follows:

$t.sub.C = (BitDepth.sub.C < 10) ? (t.sub.C' + 2) >> (10 - BitDepth.sub.C) : t.sub.C' * (1 < (BitDepth.sub.C - 8))$ (8-1137)

5.2. Embodiment #2 on Boundary Strength Derivation

(127) 8.8.3.5 Derivation Process of Boundary Filtering Strength

(128) Inputs to this process are:

(129) a picture sample array `recPicture`, a location (`xCb`, `yCb`) specifying the top-left sample of the current coding block relative to the top-left

sample of the current picture, a variable nCbW specifying the width of the current coding block, a variable nCbH specifying the height of the current coding block, a variable edge Type specifying whether a vertical(EDGE_VER) or a horizontal(EDGE_HOR) edge is filtered, a variable cIdx specifying the colour component of the current coding block, a two-dimensional(nCbW)×(nCbH) array edge Flags.
Output of this process is a two-dimensional(nCbW)×(nCbH) array bS specifying the boundary filtering strength.

...
For xD.sub.i with i=0 . . . xN and yD.sub.j with j=0 . . . yN, the following applies: If edgeFlags[xD.sub.i][yD.sub.j] is equal to 0, the variable bS[xD.sub.i][yD.sub.j] is set equal to 0. Otherwise, the following applies: . . . The variable bS[xD.sub.i][yD.sub.j] is derived as follows: If cIdx is equal to 0 and both samples p.sub.0 and q.sub.0 are in a coding block with intra_bdpcm_flag equal to 1, bS[xD.sub.i][yD.sub.j] is set equal to 0. Otherwise, if the sample p.sub.0 or q.sub.0 is in the coding block of a coding unit coded with intra prediction mode, bS[xD.sub.i][yD.sub.j] is set equal to 2. Otherwise, if the block edge is also a transform block edge and the sample p.sub.0 or q.sub.0 is in a coding block with ciip_flag equal to 1, bS[xD.sub.i][yD.sub.j] is set equal to 2. Otherwise, if the block edge is also a transform block edge and the sample p.sub.0 or q.sub.0 is in a transform block which contains one or more non-zero transform coefficient levels, bS[xD.sub.i][yD.sub.j] is set equal to 1. Otherwise, if the block edge is also a transform block edge, cIdx is greater than 0, and the sample p.sub.0 or q.sub.0 is in a transform unit with tu_joint_cbr_residual_flag equal to 1, bS[xD.sub.i][yD.sub.j] is set equal to 1. Otherwise, if the prediction mode of the coding subblock containing the sample p.sub.0 is different from the prediction mode of the coding subblock containing the sample q.sub.0 (i.e. one of the coding subblock is coded in IBC prediction mode and the other is coded in inter prediction mode), bS[xD.sub.i][yD.sub.j] is set equal to 1. Otherwise, if cIdx is equal to 0 and one or more of the following conditions are true, bS[xD.sub.i][yD.sub.j] is set equal to 1:                            

pictures are used to predict the sample q.sub.0 and the absolute difference between the horizontal or vertical component of the two motion vectors used in the prediction of the two coding subblocks for the same reference picture is greater than or equal to 8 in units of 1/16 luma samples. Two motion vectors for the same reference picture are used to predict the coding subblock containing the sample p.sub.0, two motion vectors for the same reference picture are used to predict the coding subblock containing the sample q.sub.0 and both of the following conditions are true: The absolute difference between the horizontal or vertical component of list 0 motion vectors used in the prediction of the two coding subblocks is greater than or equal to 8 in 1/16 luma samples, or the absolute difference between the horizontal or vertical component of the list 1 motion vectors used in the prediction of the two coding subblocks is greater than or equal to 8 in units of 1/16 luma samples. The absolute difference between the horizontal or vertical component of list 0 motion vector used in the prediction of the coding subblock containing the sample p.sub.0 and the list 1 motion vector used in the prediction of the coding subblock containing the sample q.sub.0 is greater than or equal to 8 in units of 1/16 luma samples, or the absolute difference between the horizontal or vertical component of the list 1 motion vector used in the prediction of the coding subblock containing the sample p.sub.0 and list 0 motion vector used in the prediction of the coding subblock containing the sample q.sub.0 is greater than or equal to 8 in units of 1/16 luma samples. Otherwise, the variable bS[xD.sub.i][yD.sub.j] is set equal to 0.

5.4. Embodiment #4 on Luma Deblocking Filtering Process

(133) 8.8.3.6.1 Decision Process for Luma Block Edges

(134) Inputs to this process are:

(135) a picture sample array recPicture, a location (xCb, yCb) specifying the top-left sample of the current coding block relative to the top-left sample of the current picture, a location (xBl, yBl) specifying the top-left sample of the current block relative to the top-left sample of the current coding block, a variable edge Type specifying whether a vertical(EDGE_VER) or a horizontal(EDGE_HOR) edge is filtered, a variable bS specifying the boundary filtering strength, a variable maxFilterLengthP specifying the max filter length, a variable maxFilterLengthQ specifying the max filter length.

Outputs of this process are: the variables dE, dEp and dEq containing decisions, the modified filter length variables maxFilterLengthP and maxFilterLengthQ, the variable t.sub.C.

...

The following ordered steps apply: ... 1. When side PisLargeBlk or side QisLargeBlk is greater than 0, the following applies: a. The variables dp0L, dp3L are derived and maxFilterLengthP is modified as follows: [[If side PisLargeBlk is equal to 1, the following applies:

$dp0L = (dp0 + Abs(p.sub.5,0 - 2 * p.sub.4,0 + p.sub.3,0) + 1) >> 1$ (8-1087)

$dp3L = (dp3 + Abs(p.sub.5,3 - 2 * p.sub.4,3 + p.sub.3,3) + 1) >> 1$ (8-1088) Otherwise, the following applies:]]

$dp0L = dp0$ (8-1089)

$dp3L = dp3$ (8-1090)

[[maxFilterLengthP=3 (8-1091)]] custom character custom character b. The variables dq0L and dq3L are derived as follows: [[If side QisLargeBlk is equal to 1, the following applies:

$dq0L = (dq0 + Abs(q.sub.5,0 - 2 * q.sub.4,0 + q.sub.3,0) + 1) >> 1$ (8-1092)

$dq3L = (dq3 + Abs(q.sub.5,3 - 2 * q.sub.4,3 + q.sub.3,3) + 1) >> 1$ (8-1093) Otherwise, the following applies:]]

$dq0L = dq0$ (8-1094)

$dq3L = dq3$ (8-1095) custom character custom character ... 2. The variables dE, dEp and dEq are derived as follows: ...

5.5. Embodiment #5 on Chroma Deblocking Filtering Process

(136) 8.8.3.6.3 Decision Process for Chroma Block Edges

(137) This process is only invoked when ChromaArrayType is not equal to 0.

(138) Inputs to this process are:

(139) a chroma picture sample array recPicture, a chroma location (xCb, yCb) specifying the top-left sample of the current chroma coding block relative to the top-left chroma sample of the current picture, a chroma location (xBl, yBl) specifying the top-left sample of the current chroma block relative to the top-left sample of the current chroma coding block, a variable edge Type specifying whether a vertical(EDGE_VER) or a horizontal(EDGE_HOR) edge is filtered, a variable cIdx specifying the colour component index, a variable cQpPicOffset specifying the picture-level chroma quantization parameter offset, a variable bS specifying the boundary filtering strength, a variable maxFilterLengthCbCr.

Outputs of this process are the modified variable maxFilterLengthCbCr, the variable t.sub.C.

The variable maxK is derived as follows: If edge Type is equal to EDGE_VER, the following applies:

$maxK = (SubHeightC == 1) ? 3 : 1$ (8-1124) Otherwise (edge Type is equal to EDGE_HOR), the following applies:

$maxK = (SubWidthC == 1) ? 3 : 1$ (8-1125)

The values p.sub.i and q.sub.i with i=0 ... maxFilterLengthCbCr and k=0 ... maxK are derived as follows: If edge Type is equal to EDGE_VER, the following applies:

$q.sub.i,k = recPicture[xCb + xBl + i][yCb + yBl + k]$ (8-1126)

$p.sub.i,k = recPicture[xCb + xBl - i - 1][yCb + yBl + k]$ (8-1127)

subSampleC=SubHeightC (8-1128) Otherwise (edge Type is equal to EDGE_HOR), the following applies:

$q.sub.i,k = recPicture[xCb + xBl + k][yCb + yBl + i]$ (8-1129)

$p.sub.i,k = recPicture[xCb + xBl + k][yCb + yBl - i - 1]$ (8-1130)

subSampleC=SubWidthC (8-1131) custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

The value of the variable β' is determined as specified in Table t-18 based on the quantization parameter Q derived as follows:

$Q = Clip3(0, 63, Qp.sub.C + (slice_beta_offset_div2 << 1))$ (8-1134)

where slice_beta_offset_div2 is the value of the syntax element slice_beta_offset_div2 for the slice that contains sample q.sub.0,0.

The variable β is derived as follows:

$\beta = \beta' * (1 << (BitDepth.sub.C - 8))$ (8-1135)

The value of the variable t.sub.C' is determined as specified in Table 8-18 based on the chroma quantization parameter Q derived as follows:

$Q = Clip3(0, 65, Qp.sub.C + 2 * (bS - 1) + (slice_tc_offset_div2 << 1))$ (8-1136)

where slice_tc_offset_div2 is the value of the syntax element slice_tc_offset_div2 for the slice that contains sample q.sub.0,0. The variable t.sub.C is derived as follows:

$t.sub.C = (BitDepth.sub.C < 10) ? (t.sub.C' + 2) >> (10 - BitDepth.sub.C) : t.sub.C' * (1 << (BitDepth.sub.C - 8))$ (8-1137)

When maxFilterLengthCbCr is equal to 1 and bS is not equal to 2, maxFilterLengthCbCr is set equal to 0.

5.6. Embodiment #6 on Chroma QP in Deblocking

(140) 8.8.3.6.3 Decision Process for Chroma Block Edges

(141) This process is only invoked when ChromaArrayType is not equal to 0.

(142) Inputs to this process are:

(143) a chroma picture sample array recPicture, a chroma location (xCb, yCb) specifying the top-left sample of the current chroma coding block relative to the top-left chroma sample of the current picture, a chroma location (xBl, yBl) specifying the top-left sample of the current chroma block relative to the top-left sample of the current chroma coding block, a variable edge Type specifying whether a vertical(EDGE_VER) or a horizontal(EDGE_HOR) edge is filtered, a variable cIdx specifying the colour component index, a variable cQpPicOffset specifying the picture-level chroma quantization parameter offset, a variable bS specifying the boundary filtering strength, a variable maxFilterLengthCbCr.

Outputs of this process are the modified variable maxFilterLengthCbCr, the variable t.sub.C.

The variable maxK is derived as follows: If edge Type is equal to EDGE_VER, the following applies:

$\text{maxK} = (\text{SubHeightC} == 1) ? 3 : 1$ (8-1124) Otherwise (edge Type is equal to EDGE_HOR), the following applies:

$\text{maxK} = (\text{SubWidthC} == 1) ? 3 : 1$ (8-1125)

The values p.sub.i and q.sub.i with $i = 0 \dots \text{maxFilterLengthCbCr}$ and $k = 0 \dots \text{maxK}$ are derived as follows: If edge Type is equal to EDGE_VER, the following applies:

$q.\text{sub}.i, k = \text{recPicture}[\text{xCb} + \text{xBl} + i][\text{yCb} + \text{yBl} + k]$ (8-1126)

$p.\text{sub}.i, k = \text{recPicture}[\text{xCb} + \text{xBl} - i - 1][\text{yCb} + \text{yBl} + k]$ (8-1127)

subSampleC = SubHeightC (8-1128) Otherwise (edge Type is equal to EDGE_HOR), the following applies:

$q.\text{sub}.i, k = \text{recPicture}[\text{xCb} + \text{xBl} + k][\text{yCb} + \text{yBl} + i]$ (8-1129)

$p.\text{sub}.i, k = \text{recPicture}[\text{xCb} + \text{xBl} + k][\text{yCb} + \text{yBl} - i - 1]$ (8-1130)

subSampleC = SubWidthC (8-1131)

The variables Qp.sub.Q and Qp.sub.P are set equal to the Qp.sub.Y values of the coding units which include the coding blocks containing the sample q.sub.0,0 and p.sub.0,0, respectively.

custom character custom character custom character custom character custom character custom character

The variable Qp.sub.C is derived as follows:

$[[qP_i = \text{Clip3}(0, 63, ((Qp.\text{sub}.Q + Qp.\text{sub}.P + 1) >> 1) + cP_{\text{PicOffset}})]$ (8-1132)] custom character custom character custom character

custom character custom character custom character

$Qp.\text{sub}.C = \text{ChromaQpTable}[\text{cIdx} - 1][qP_i]$ (8-1133) NOTE—The variable cQpPicOffset provides an adjustment for the value of pps_cb_qp_offset or pps_cr_qp_offset, according to whether the filtered chroma component is the Cb or Cr component. However, to avoid the need to vary the amount of the adjustment within the picture, the filtering process does not include an adjustment for the value of slice_cb_qp_offset or slice_cr_qp_offset n or (when cu_chroma_qp_offset_enabled_flag is equal to 1) for the value of CuQpOffset.sub.Cb, CuQpOffset.sub.Cr, or CuQpOffset.sub.CbCr.

...

5.7. Embodiment #7 on Chroma QP in Deblocking

(144) 8.8.3.6.3 Decision Process for Chroma Block Edges

(145) This process is only invoked when ChromaArrayType is not equal to 0.

(146) Inputs to this process are:

(147) a chroma picture sample array recPicture, a chroma location (xCb, yCb) specifying the top-left sample of the current chroma coding block relative to the top-left chroma sample of the current picture, ...

Outputs of this process are the modified variable maxFilterLengthCbCr, the variable t.sub.C.

The variable maxK is derived as follows: If edge Type is equal to EDGE_VER, the following applies:

$\text{maxK} = (\text{SubHeightC} == 1) ? 3 : 1$ (8-1124) Otherwise (edge Type is equal to EDGE_HOR), the following applies:

$\text{maxK} = (\text{SubWidthC} == 1) ? 3 : 1$ (8-1125)

The values p.sub.i and q.sub.i with $i = 0 \dots \text{maxFilterLengthCbCr}$ and $k = 0 \dots \text{maxK}$ are derived as follows: If edge Type is equal to EDGE_VER, the following applies:

$q.\text{sub}.i, k = \text{recPicture}[\text{xCb} + \text{xBl} + i][\text{yCb} + \text{yBl} + k]$ (8-1126)

$p.\text{sub}.i, k = \text{recPicture}[\text{xCb} + \text{xBl} - i - 1][\text{yCb} + \text{yBl} + k]$ (8-1127)

subSampleC = SubHeightC (8-1128) Otherwise (edge Type is equal to EDGE_HOR), the following applies:

$q.\text{sub}.i, k = \text{recPicture}[\text{xCb} + \text{xBl} + k][\text{yCb} + \text{yBl} + i]$ (8-1129)

$p.\text{sub}.i, k = \text{recPicture}[\text{xCb} + \text{xBl} + k][\text{yCb} + \text{yBl} - i - 1]$ (8-1130)

subSampleC = SubWidthC (8-1131)

[[The variables Qp.sub.Q and Qp.sub.P are set equal to the Qp.sub.Y values of the coding units which include the coding blocks containing the sample q.sub.0,0 and p.sub.0,0, respectively.]]

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character

The variable Qp.sub.C is derived as follows:

$qP_i = \text{Clip3}(0, 63, ((Qp.\text{sub}.Q + Qp.\text{sub}.P + 1) >> 1) + cQ_{\text{PicOffset}})$ (8-1132)

$Qp.\text{sub}.C = \text{ChromaQpTable}[\text{cIdx} - 1][qP_i]$ (8-1133) NOTE—The variable cQpPicOffset provides an adjustment for the value of pps_cb_qp_offset or pps_cr_qp_offset, according to whether the filtered chroma component is the Cb or Cr component. However, to avoid the need to vary the amount of the adjustment within the picture, the filtering process does not include an adjustment for the value of slice_cb_qp_offset or slice_cr_qp_offset n or (when cu_chroma_qp_offset_enabled_flag is equal to 1) for the value of CuQpOffset.sub.Cb, CuQpOffset.sub.Cr, or CuQpOffset.sub.CbCr.

The value of the variable β' is determined as specified in Table 8-18 based on the quantization parameter Q derived as follows:

$Q = \text{Clip3}(0, 63, Qp.\text{sub}.C + (\text{slice_beta_offset_div2} < < 1))$ (8-1134)

where slice_beta_offset_div2 is the value of the syntax element slice_beta_offset_div2 for the slice that contains sample q.sub.0,0.

The variable β is derived as follows:

$\beta = \beta' * (1 < < (\text{BitDepth.sub.C} - 8))$ (8-1135)

The value of the variable t.sub.C' is determined as specified in Table 8-18 based on the chroma quantization parameter Q derived as follows:

$Q = \text{Clip3}(0, 65, Qp.\text{sub}.C + 2 * (bS - 1) + (\text{slice_tc_offset_div2} < < 1))$ (8-1136)

where slice_tc_offset_div2 is the value of the syntax element slice_tc_offset_div2 for the slice that contains sample q.sub.0,0.

5.8. Embodiment #8 on Chroma QP in Deblocking

(148) When making filter decisions for the depicted three samples (with solid circles), the QPs of the luma CU that covers the center position of the chroma CU including the three samples is selected. Therefore, for the 1.sup.st, 2.sup.nd, and 3.sup.rd chroma sample (depicted in FIG. 11), only the QP of CU.sub.Y3 is utilized, respectively.

(149) In this way, how to select luma CU for chroma quantization/dequantization process is aligned with that for chroma filter decision process.

(151) Inputs to this process are:

(152) a luma location(xTbY, yTbY) specifying the top-left sample of the current luma transform block relative to the top-left luma sample of the current picture, a variable nTbW specifying the transform block width, a variable nTbH specifying the transform block height, a variable cIdx specifying the colour component of the current block, a variable bitDepth specifying the bit depth of the current colour component.

Output of this process is the (nTbW)×(nTbH) array d of scaled transform coefficients with elements d[x][y]. The quantization parameter qP is derived as follows: If cIdx is equal to 0 and transform_skip_flag[xTbY][yTbY] is equal to 0, the following applies:

$qP = QP'.sub.Y$ (8-950) Otherwise, if cIdx is equal to 0 (and transform_skip_flag[xTbY][yTbY] is equal to 1), the following applies:

$qP = \text{Max}(QPPrimeTMin, QP'.sub.Y)$ (8-951) Otherwise, if TuCResMode[xTbY][yTbY] is  equal to 2], the following applies:

$qP = QP'.sub.CbCr$ (8-952) Otherwise, if cIdx is equal to 1, the following applies:

$qP = QP'.sub.Cb$ (8-953) Otherwise (cIdx is equal to 2), the following applies:

$qP = QP'.sub.Cr$ (8-954)

5.10 Embodiment #10 on QP Used for JCCR Coded Blocks

(153) 8.8.3.2 Deblocking Filter Process for One Direction

(154) Inputs to this process are:

(155) the variable treeType specifying whether the luma (DUAL_TREE_LUMA) or chroma components (DUAL_TREE_CHROMA) are currently processed, when treeType is equal to DUAL_TREE_LUMA, the reconstructed picture prior to deblocking, i.e., the array recPicture.sub.L, when ChromaArrayType is not equal to 0 and treeType is equal to DUAL_TREE_CHROMA, the arrays recPicture.sub.Cb and recPicture.sub.Cr, a variable edge Type specifying whether a vertical(EDGE_VER) or a horizontal(EDGE_HOR) edge is filtered.

Outputs of this process are the modified reconstructed picture after deblocking, i.e: when treeType is equal to DUAL_TREE_LUMA, the array recPicture.sub.L, when ChromaArrayType is not equal to 0 and treeType is equal to DUAL_TREE_CHROMA, the arrays recPicture.sub.Cb and recPicture.sub.Cr.

(156) The variables firstCompIdx and lastCompIdx are derived as follows:

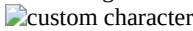
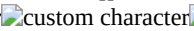
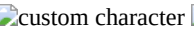
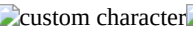
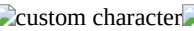
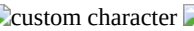
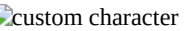
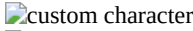
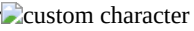
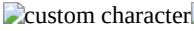
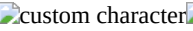
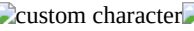
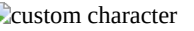
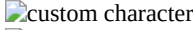
firstCompIdx=(treeType==DUAL_TREE_CHROMA)?1:0 (8-1022)

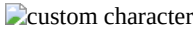
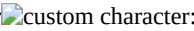
lastCompIdx=(treeType==DUAL_TREE_LUMA ChromaArrayType==0)?0:2 (8-1023)

For each coding unit and each coding block per colour component of a coding unit indicated by the colour component index cIdx ranging from firstCompIdx to lastCompIdx, inclusive, with coding block width nCbW, coding block height nCbH and location of top-left sample of the coding block (xCb, yCb), when cIdx is equal to 0, or when cIdx is not equal to 0 and edge Type is equal to EDGE_VER and xCb % 8 is equal to 0, or when cIdx is not equal to 0 and edge Type is equal to EDGE_HOR and yCb % 8 is equal to 0, the edges are filtered by the following ordered steps:

...

[[5. The picture sample array recPicture is derived as follows: If cIdx is equal to 0, recPicture is set equal to the reconstructed luma picture sample array prior to deblocking recPicture.sub.L. Otherwise, if cIdx is equal to 1, recPicture is set equal to the reconstructed chroma picture sample array prior to deblocking recPicture.sub.Cb. Otherwise (cIdx is equal to 2), recPicture is set equal to the reconstructed chroma picture sample array prior to deblocking recPicture.sub.Cr]]

 : The edge filtering process for one direction is invoked for a coding block as specified in clause 8.8.3.6 with the variable edge Type, the variable cIdx, the reconstructed picture prior to deblocking recPicture, the location (xCb, yCb), the coding block width nCbW, the coding block height nCbH, and the arrays bS, maxFilterLengthPs, and maxFilterLengthQs, as inputs, and the modified reconstructed picture recPicture as output.

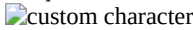
8.8.3.5 Derivation Process of Boundary Filtering Strength

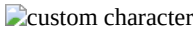
Inputs to this process are: a picture sample array recPicture, a location (xCb, yCb) specifying the top-left sample of the current coding block relative to the top-left sample of the current picture, a variable nCbW specifying the width of the current coding block, a variable nCbH specifying the height of the current coding block, a variable edge Type specifying whether a vertical(EDGE_VER) or a horizontal(EDGE_HOR) edge is filtered, a variable cIdx specifying the colour component of the current coding block, a two-dimensional(nCbW)×(nCbH) array edgeFlags.

Output of this process is a two-dimensional(nCbW)×(nCbH) array bS specifying the boundary filtering strength. The variables xD.sub.i, yD.sub.j, xN and yN are derived as follows:

...

For xD.sub.i with i=0 . . . xN and yD.sub.j with j=0 . . . yN, the following applies: If edgeFlags[xD.sub.i][yD.sub.j] is equal to 0, the variable bS[xD.sub.i][yD.sub.j] is set equal to 0. Otherwise, the following applies: The sample values p.sub.0 and q.sub.0 are derived as follows: If edge Type is equal to EDGE_VER, p.sub.0 is set equal to recPicture xCb+xD.sub.i-1][yCb+yD.sub.j] and q.sub.0 is set equal to recPicture

 xCb+xD.sub.i][yCb+yD.sub.j]. Otherwise (edge Type is equal to EDGE_HOR), p.sub.0 is set equal to recPicture

 [xCb+xD.sub.i][yCb+yD.sub.j-1] and q.sub.0 is set equal to recPicture xCb+xD.sub.i][yCb+yD.sub.j].

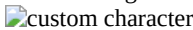
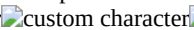
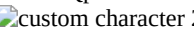
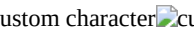
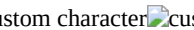
...

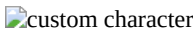
8.8.3.6 Edge Filtering Process for One Direction

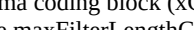
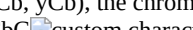
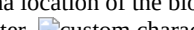
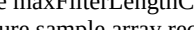
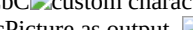
Inputs to this process are: a variable edge Type specifying whether vertical edges (EDGE_VER) or horizontal edges (EDGE_HOR) are currently processed, a variable cIdx specifying the current colour component, the reconstructed picture prior to deblocking recPicture, a location (xCb, yCb) specifying the top-left sample of the current coding block relative to the top-left sample of the current picture, a variable nCbW specifying the width of the current coding block, a variable nCbH specifying the height of the current coding block, the array bS specifying the boundary strength, the arrays maxFilterLengthPs and maxFilterLengthQs.

Output of this process is the modified reconstructed picture after deblocking recPicture.sub.i.

... Otherwise (cIdx is not equal to 0), the filtering process for edges in the chroma coding block of current coding unit specified by cIdx consists of the following ordered steps: 1. The variable cQPpicOffset is derived as follows:   

   2.    

 3. The decision process for chroma block edges as specified in clause 8.8.3.6.3 is invoked with the chroma picture sample array recPicture, the location of the chroma coding block (xCb, yCb), the location of the chroma block (xBl, yBl) set equal to (xD.sub.k, yD.sub.m), the edge direction edge Type, the variable cQPpicOffset, the boundary filtering strength bS[xD.sub.k][yD.sub.m], and the variable maxFilterLengthCbCr set equal to maxFilterLengthPs[xD.sub.k][yD.sub.m] as inputs, and the modified variable maxFilterLengthCbCr, and the variable t.sub.C as outputs.

4. When maxFilterLengthCbCr is greater than 0, the filtering process for chroma block edges as specified in clause 8.8.3.6.4 is invoked with the chroma picture sample array recPicture, the location of the chroma coding block (xCb, yCb), the chroma location of the block (xBl, yBl) set equal to (xD.sub.k, yD.sub.m), the edge direction edge Type, the variable maxFilterLengthCbC   , and the variable t.sub.C as inputs, and the modified chroma picture sample array recPicture as output.   

8.8.3.6.3 Decision Process for Chroma Block Edges

This process is only invoked when ChromaArrayType is not equal to 0.

Inputs to this process are: a chroma picture sample array recPicture, a chroma location (xCb, yCb) specifying the top-left sample of the current chroma coding block relative to the top-left chroma sample of the current picture, a chroma location (xBl, yBl) specifying the top-left sample of the current chroma block relative to the top-left sample of the current chroma coding block, a variable edge Type specifying whether a vertical(EDGE_VER) or a horizontal(EDGE_HOR) edge is filtered, [[a variable cIdx specifying the colour component index,] a variable cQpPicOffset specifying the picture-level chroma quantization parameter offset, a variable bS specifying the boundary filtering strength, a variable maxFilterLengthCbCr.

Outputs of this process are the modified variable maxFilterLengthCbCr, the variable t.sub.C.

The variable maxK is derived as follows: If edge Type is equal to EDGE_VER, the following applies:

$\text{maxK} = (\text{SubHeightC} == 1) ? 3 : 1$ (8-1124) Otherwise (edge Type is equal to EDGE_HOR), the following applies:

$\text{maxK} = (\text{SubWidthC} == 1) ? 3 : 1$ (8-1125)

The values p.sub.i and q.sub.i with custom character, $i = 0 \dots \text{maxFilterLengthCbCr}$ and $k = 0 \dots \text{maxK}$ are derived as follows: If edge Type is equal to EDGE_VER, the following applies:

$q_{\text{c}, \text{sub.i}, k} = \text{recPicture}[\text{custom character} \times \text{Cb} + xBl + i][yCb + yBl + k]$ (8-1126)

$p_{\text{c}, \text{sub.i}, k} = \text{recPicture}[\text{custom character} \times \text{Cb} + Bl - i - 1][yCb + yBl + k]$ (8-1127)

subSampleC = SubHeightC (8-1128) Otherwise (edge Type is equal to EDGE_HOR), the following applies:

$q_{\text{c}, \text{sub.i}, k} = \text{recPicture}[\text{custom character} \times \text{Cb} + xBl + k][yCb + yBl + i]$ (8-1129)

$p_{\text{c}, \text{sub.i}, k} = \text{recPicture}[\text{custom character} \times \text{Cb} + xBl + k][yCb + yBl - i - 1]$ (8-1130)

subSampleC = SubWidthC (8-1131)

The variables Qp.sub.Q and Qp.sub.P are set equal to the Qp.sub.Y values of the coding units which include the coding blocks containing the sample q.sub.0,0 and p.sub.0,0, respectively.

The variable Qp.sub.C is derived as follows:

$qPi = \text{custom character}(\text{Qp.sub.Q} + \text{Qp.sub.P} + 1) >> 1$ custom character (8-1132) custom character custom character custom character

custom character custom character custom character custom character NOTE—The variable cQpPicOffset provides an adjustment for the value of pps_cb_qp_offset or pps_cr_qp_offset, according to whether the filtered chroma component is the Cb or Cr component. However, to avoid the need to vary the amount of the adjustment within the picture, the filtering process does not include an adjustment for the value of slice_cb_qp_offset or slice_cr_qp_offset n or (when cu_chroma_qp_offset_enabled_flag is equal to 1) for the value of CuQpOffset.sub.Cb, CuQpOffset.sub.Cr, or CuQpOffset.sub.CbCr.

The value of the variable β' is determined as specified in Table 8-18 based on the quantization parameter Q derived as follows:

$Q = \text{Clip3}(0, 63, \text{Qp.sub.C} + (\text{slice_beta_offset_div2} < < 1))$ (8-1134)

where slice_beta_offset_div2 is the value of the syntax element slice_beta_offset_div2 for the slice that contains sample q.sub.0,0.

The variable β is derived as follows:

$\beta = \beta' * (1 < < (\text{BitDepth.sub.C} - 8))$ (8-1135)

The value of the variable t.sub.C' is determined as specified in Table 8-18 based on the chroma quantization parameter Q derived as follows:

$Q = \text{Clip3}(0, 65, \text{Qp.sub.C} + 2 * (bS - 1) + (\text{slice_tc_offset_div2} < < 1))$ (8-1136)

where slice_tc_offset_div2 is the value of the syntax element slice_tc_offset_div2 for the slice that contains sample q.sub.0,0. The variable t.sub.C is derived as follows:

$t.\text{sub.C} = (\text{BitDepth.sub.C} < 10) ? (t.\text{sub.C}' + 2) >> (10 - \text{BitDepth.sub.C}) : t.\text{sub.C}' * (1 < < (\text{BitDepth.sub.C} - 8))$ (8-1137)

When maxFilterLengthCbCr is equal to 1 and bS is not equal to 2, maxFilterLengthCbCr is set equal to 0.

When maxFilterLengthCbCr is equal to 3, the following ordered steps apply: 1. The variables n1, dpq0c, dpq1c, dpc, dqc and d custom character, are derived as follows:

$n1 = (\text{subSampleC} == 2) ? 1 : 3$ (8-1138)

$dp0c = \text{Abs}(p_{\text{c}, \text{sub.2}, 0} - 2 * p_{\text{c}, \text{sub.1}, 0} + p_{\text{c}, \text{sub.0}, 0})$ (8-1139)

$dp1c = \text{Abs}(p_{\text{c}, \text{sub.2}, n1} - 2 * p_{\text{c}, \text{sub.1}, n1} + p_{\text{c}, \text{sub.0}, n1})$ (8-1140)

$dq0c = \text{Abs}(q_{\text{c}, \text{sub.2}, 0} - 2 * q_{\text{c}, \text{sub.1}, 0} + q_{\text{c}, \text{sub.0}, 0})$ (8-1141)

$dq1c = \text{Abs}(q_{\text{c}, \text{sub.2}, n1} - 2 * q_{\text{c}, \text{sub.1}, n1} + q_{\text{c}, \text{sub.0}, n1})$ (8-1142)

$dpq0c = dp0c + dq0c$ (8-1143)

$dpq1c = dp1c + dq1c$ (8-1144)

$dpc = dp0c + dp1c$ (8-1145)

$dqc = dq0c + dq1c$ (8-1146)

$d_c = dpq0c + dpq1c$ (8-1147) 2. custom character 3. The variables dSam0 and dSam1 are both set equal to 0. 4. When d is less than β , the following ordered steps apply: a. The variable dpq is set equal to $2 * dpq0$. b. The variable dSam0 is derived by invoking the decision process for a chroma sample as specified in clause 8.8.3.6.8 for the sample location(xCb+xBl, yCb+yBl) with sample values p.sub.0,0, p.sub.3,0, q.sub.0,0, and q.sub.3,0, the variables dpq, β and t.sub.C as inputs, and the output is assigned to the decision dSam0. c. The variable dpq is set equal to $2 * dpq1$. d. The variable dSam1 is modified as follows: If edge Type is equal to EDGE_VER, for the sample location (xCb+xBl, yCb+yBl+n1), the decision process for a chroma sample as specified in clause 8.8.3.6.8 is invoked with sample values p.sub.0,n1, p.sub.3,n1, q.sub.0,n1, and q.sub.3,n1, the variables dpq, β and t.sub.C as inputs, and the output is assigned to the decision dSam1. Otherwise (edge Type is equal to EDGE_HOR), for the sample location(xCb+xBl+n1, yCb+yBl), the decision process for a chroma sample as specified in clause 8.8.3.6.8 is invoked with sample values p.sub.0,0, p.sub.3,n1, q.sub.0,n1 and q.sub.3,n1, the variables dpq, β and t.sub.C as inputs, and the output is assigned to the decision dSam1. 5. The variable maxFilterLengthCbCr is modified as follows: If dSam0 is equal to 1 and dSam1 is equal to 1, maxFilterLengthCbCr is set equal to 3. Otherwise, maxFilterLengthCbCr is set equal to 1.

8.8.3.6.4 Filtering Process for Chroma Block Edges

This process is only invoked when ChromaArrayType is not equal to 0.

Inputs to this process are: a chroma picture sample array recPicture, a chroma location (xCb, yCb) specifying the top-left sample of the current chroma coding block relative to the top-left chroma sample of the current picture, a chroma location (xBl, yBl) specifying the top-left sample of the current chroma block relative to the top-left sample of the current chroma coding block, a variable edge Type specifying whether a vertical(EDGE_VER) or a horizontal(EDGE_HOR) edge is filtered, a variable maxFilterLengthCbCr containing the maximum chroma filter length, 6. custom character custom character

Output of this process is the modified chroma picture sample array recPicture.

...

The values p.sub.i and q.sub.i with $i = 0 \dots \text{maxFilterLengthCbCr}$ and $k = 0 \dots \text{maxK}$ are derived as follows: If edge Type is equal to EDGE_VER,

the following applies:

$q_{\text{sub.i,k}} = \text{recPicture}[\text{custom character}[xCb + xBl + i][yCb + yBl + k]]$ (8-1150)

$p_{\text{sub.i,k}} = \text{recPicture}[\text{custom character}[xCb + xBl - i - 1][yCb + yBl + k]]$ (8-1151) Otherwise (edge Type is equal to EDGE_HOR), the following applies:

(157)

$q_{\text{sub.i,k}} = \text{recPicture}[\text{custom character}[xCb + xBl + k][yCb + yBl + i]]$ (8-1152)

$p_{\text{sub.i,k}} = \text{recPicture}[\text{custom character}[xCb + xBl + k][yCb + yBl - i - 1]]$ (8-1153)

Depending on the value of edge Type, the following applies: If edge Type is equal to EDGE_VER, for each sample location $(xCb + xBl, yCb + yBl + k)$, $k = 0 \dots \text{maxK}$, the following ordered steps apply: 1. The filtering process for a chroma sample as specified in clause 8.8.3.6.9 is invoked with the variable $\text{maxFilterLengthCbCr}$, the sample values $p_{\text{sub.i,k}}$, $q_{\text{sub.i,k}}$ with $i = 0 \dots \text{maxFilterLengthCbCr}$, the locations $(xCb + xBl - i - 1, yCb + yBl + k)$ and $(xCb + xBl + i, yCb + yBl + k)$ with $i = 0 \dots \text{maxFilterLengthCbCr} - 1$, and the variable $t_{\text{sub.C}}$ as inputs, and the filtered sample values $p_{\text{sub.i'}}$ and $q_{\text{sub.i'}}$ with $i = 0 \dots \text{maxFilterLengthCbCr} - 1$ as outputs. 2. The filtered sample values $p_{\text{sub.i'}}$ and $q_{\text{sub.i'}}$ with $i = 0 \dots \text{maxFilterLengthCbCr} - 1$ replace the corresponding samples inside the sample array recPicture as follows:

$\text{recPicture}[\text{custom character}[xCb + xBl + i][yCb + yBl + k]] = q_{\text{sub.i'}}$ (8-1154)

$\text{recPicture}[\text{custom character}[xCb + xBl - i - 1][yCb + yBl + k]] = p_{\text{sub.i'}}$ (8-1155)

Otherwise (edge Type is equal to EDGE_HOR), for each sample location $(xCb + xBl + k, yCb + yBl)$, $k = 0 \dots \text{maxK}$, the following ordered steps apply: 1. The filtering process for a chroma sample as specified in clause 8.8.3.6.9 is invoked with the variable $\text{maxFilterLengthCbCr}$, the sample values $p_{\text{sub.i,k}}$, $q_{\text{sub.i,k}}$, with $i = 0 \dots \text{maxFilterLengthCbCr}$, the locations $(xCb + xBl + k, yCb + yBl - i - 1)$ and $(xCb + xBl + k, yCb + yBl + i)$, and the variable $t_{\text{sub.C}}$ as inputs, and the filtered sample values $p_{\text{sub.i'}}$ and $q_{\text{sub.i'}}$ as outputs. 2. The filtered sample values $p_{\text{sub.i'}}$ and $q_{\text{sub.i'}}$ replace the corresponding samples inside the sample array recPicture as follows:

$\text{recPicture}[\text{custom character}[xCb + xBl + k][yCb + yBl + i]] = q_{\text{sub.i'}}$

$\text{recPicture}[\text{custom character}[xCb + xBl + k][yCb + yBl - i - 1]] = p_{\text{sub.i'}}$ (8-1156)

5.11 Embodiment #11

(158) 8.8.3.6.3 Decision Process for Chroma Block Edges

(159) . . .

(160) [[The variables $Qp_{\text{sub.Q}}$ and $Qp_{\text{sub.P}}$ are set equal to the $Qp_{\text{sub.Y}}$ values of the coding units which include the coding blocks containing the sample $q_{\text{sub.0,0}}$ and $p_{\text{sub.0,0}}$, respectively.

(161) The variable $Qp_{\text{sub.C}}$ is derived as follows:

$qPi = \text{Clip3}(0, 63, ((Qp_{\text{sub.Q}} + Qp_{\text{sub.P}} + 1) >> 1) + cQpPicOffset)$ (8-1132)

$Qp_{\text{sub.C}} = \text{ChromaQpTable}[cIdx - 1][qPi]$ (8-1133)]

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

custom character custom character custom character custom character custom character custom character custom character

transform block containing the sample $q_{\text{sub}}[0][0]$. The variable $Qp_{\text{sub}}[C]$ is derived as follows:

$$Qp_{\text{sub}}[C] = (Qp_{\text{sub}}[Q] - QpBdOffset + Qp_{\text{sub}}[P] - QpBdOffset + 1) \gg 1 \quad (1321)$$

5.14 Embodiment #14

(170) Decision Process for Chroma Block Edges

(171) This process is only invoked when ChromaArrayType is not equal to 0.

(172) Inputs to this process are:

(173) a chroma picture sample array recPicture, a chroma location (xCb, yCb) specifying the top-left sample of the current chroma coding block relative to the top-left chroma sample of the current picture, a chroma location (xBl, yBl) specifying the top-left sample of the current chroma block relative to the top-left sample of the current chroma coding block, a variable edge Type specifying whether a vertical(EDGE_VER) or a horizontal(EDGE_HOR) edge is filtered, a variable cIdx specifying the colour component index, a variable bS specifying the boundary filtering strength, a variable maxFilterLengthP specifying the maximum filter length, a variable maxFilterLengthQ specifying the maximum filter length. Outputs of this process are the modified filter length variables maxFilterLengthP and maxFilterLengthQ, the variable t.sub.C.

...

The variable $Qp_{\text{sub}}[P]$ is derived as follows: The luma location (xTb.sub.P, yTb.sub.P) is set as the top-left luma sample position of the transform block containing the sample $p_{\text{sub}}[0][0]$, relative to the top-left luma sample of the picture. If TuCResModel[xTb.sub.P][yTb.sub.P] is equal to 2, $Qp_{\text{sub}}[P]$ is set equal to $Qp'_{\text{sub}}[CbCr]$ of the transform block containing the sample $p_{\text{sub}}[0][0]$. Otherwise, if cIdx is equal to 1, $Qp_{\text{sub}}[P]$ is set equal to $Qp'_{\text{sub}}[Cb]$ of the transform block containing the sample $p_{\text{sub}}[0][0]$. Otherwise, $Qp_{\text{sub}}[P]$ is set equal to $Qp'_{\text{sub}}[Cr]$ of the transform block containing the sample $p_{\text{sub}}[0][0]$. The variable $Qp_{\text{sub}}[Q]$ is derived as follows: The luma location (xTb.sub.Q, yTb.sub.Q) is set as the top-left luma sample position of the transform block containing the sample $q_{\text{sub}}[0][0]$, relative to the top-left luma sample of the picture. If TuCResModel[xTb.sub.Q][yTb.sub.Q] is equal to 2, $Qp_{\text{sub}}[Q]$ is set equal to $Qp'_{\text{sub}}[CbCr]$ of the transform block containing the sample $q_{\text{sub}}[0][0]$. Otherwise, if cIdx is equal to 1, $Qp_{\text{sub}}[Q]$ is set equal to $Qp'_{\text{sub}}[Cb]$ of the transform block containing the sample $q_{\text{sub}}[0][0]$. Otherwise, $Qp_{\text{sub}}[Q]$ is set equal to $Qp'_{\text{sub}}[Cr]$ of the transform block containing the sample $q_{\text{sub}}[0][0]$. The variable $Qp_{\text{sub}}[C]$ is derived as follows:

$$Qp_{\text{sub}}[C] = (Qp_{\text{sub}}[Q] - QpBdOffset + Qp_{\text{sub}}[P] - QpBdOffset + 1) \gg 1 \quad (1321)$$

5.15 Embodiment #15

(174) The example controlling logic is shown in FIG. 17.

(175) 7.3.2.6 Picture Header RBSP Syntax

```
(176) TABLE-US-00009 if(!pic_dep_quant_enabled_flag) sign_data_hiding_enabled_flag u(1) [[if(deblocking_filter_override_enabled_flag) {
pic_deblocking_filter_override_present_flag u(1) if(pic_deblocking_filter_override_present_flag) { pic_deblocking_filter_override_flag u(1)
if(pic_deblocking_filter_override_flag) { pic_deblocking_filter_disabled_flag u(1) if(!pic_deblocking_filter_disabled_flag) {
pic_beta_offset_div2 se(v) pic_tc_offset_div2 se(v) } } } ]]] if(sps_lmcs_enabled_flag) {
pic_lmcs_enabled_flag u(1)
```

7.3.7.1 General Slice Header Syntax

```
(177) TABLE-US-00010 if(deblocking_filter_override_enabled_flag) [[&& !pic_deblocking_filter_override_present_flag]]
slice_deblocking_filter_override_flag u(1) if(slice_deblocking_filter_override_flag) { slice_deblocking_filter_disabled_flag u(1)
if(!slice_deblocking_filter_disabled_flag) { slice_beta_offset_div2 se(v) slice_tc_offset_div2 se(v) } }
```

5.16 Embodiment #16

(178) 7.3.2.4 Picture Parameter Set RBSP Syntax

```
(179) TABLE-US-00011 if(deblocking_filter_control_present_flag) { deblocking_filter_override_enabled_flag u(1)
pps_deblocking_filter_disabled_flag u(1) if(!pps_deblocking_filter_disabled_flag) { pps_beta_offset_div2 se(v)
pps_tc_offset_div2 se(v) }
}
```

7.3.2.6 Picture Header RBSP Syntax

```
(180) TABLE-US-00012 if(pic_deblocking_filter_override_present_flag) { pic_deblocking_filter_override_flag u(1)
if(pic_deblocking_filter_override_flag) { pic_deblocking_filter_disabled_flag u(1) if(!pic_deblocking_filter_disabled_flag) {
pic_beta_offset_div2 se(v) pic_tc_offset_div2 se(v) } } }
```

7.3.7.1 General Slice Header Syntax

```
(181) TABLE-US-00013 if(slice_deblocking_filter_override_flag) { slice_deblocking_filter_disabled_flag u(1)
if(!slice_deblocking_filter_disabled_flag) { slice_beta_offset_div2 se(v) slice_tc_offset_div2 se(v) } }
```

7.4.3.4 Picture Parameter Set RBSP Semantics

The variable $Qp_{\text{sub}}[C]$ is derived as follows: The luma location (xTb.sub.P, yTb.sub.P) is set as the top-left luma sample position of the transform block containing the sample $p_{\text{sub}}[0][0]$, relative to the top-left luma sample of the picture. If TuCResModel[xTb.sub.P][yTb.sub.P] is equal to 2, $Qp_{\text{sub}}[P]$ is set equal to $Qp'_{\text{sub}}[CbCr]$ of the transform block containing the sample $p_{\text{sub}}[0][0]$. Otherwise, if cIdx is equal to 1, $Qp_{\text{sub}}[P]$ is set equal to $Qp'_{\text{sub}}[Cb]$ of the transform block containing the sample $p_{\text{sub}}[0][0]$. Otherwise, $Qp_{\text{sub}}[P]$ is set equal to $Qp'_{\text{sub}}[Cr]$ of the transform block containing the sample $p_{\text{sub}}[0][0]$. The variable $Qp_{\text{sub}}[Q]$ is derived as follows: The luma location (xTb.sub.Q, yTb.sub.Q) is set as the top-left luma sample position of the transform block containing the sample $q_{\text{sub}}[0][0]$, relative to the top-left luma sample of the picture. If TuCResModel[xTb.sub.Q][yTb.sub.Q] is equal to 2, $Qp_{\text{sub}}[Q]$ is set equal to $Qp'_{\text{sub}}[CbCr]$ of the transform block containing the sample $q_{\text{sub}}[0][0]$. Otherwise, if cIdx is equal to 1, $Qp_{\text{sub}}[Q]$ is set equal to $Qp'_{\text{sub}}[Cb]$ of the transform block containing the sample $q_{\text{sub}}[0][0]$. Otherwise, $Qp_{\text{sub}}[Q]$ is set equal to $Qp'_{\text{sub}}[Cr]$ of the transform block containing the sample $q_{\text{sub}}[0][0]$. The variable $Qp_{\text{sub}}[C]$ is derived as follows:

7.4.3.6 Picture Header

The variable $Qp_{\text{sub}}[C]$ is derived as follows: The luma location (xTb.sub.P, yTb.sub.P) is set as the top-left luma sample position of the transform block containing the sample $p_{\text{sub}}[0][0]$, relative to the top-left luma sample of the picture. If TuCResModel[xTb.sub.P][yTb.sub.P] is equal to 2, $Qp_{\text{sub}}[P]$ is set equal to $Qp'_{\text{sub}}[CbCr]$ of the transform block containing the sample $p_{\text{sub}}[0][0]$. Otherwise, if cIdx is equal to 1, $Qp_{\text{sub}}[P]$ is set equal to $Qp'_{\text{sub}}[Cb]$ of the transform block containing the sample $p_{\text{sub}}[0][0]$. Otherwise, $Qp_{\text{sub}}[P]$ is set equal to $Qp'_{\text{sub}}[Cr]$ of the transform block containing the sample $p_{\text{sub}}[0][0]$. The variable $Qp_{\text{sub}}[Q]$ is derived as follows: The luma location (xTb.sub.Q, yTb.sub.Q) is set as the top-left luma sample position of the transform block containing the sample $q_{\text{sub}}[0][0]$, relative to the top-left luma sample of the picture. If TuCResModel[xTb.sub.Q][yTb.sub.Q] is equal to 2, $Qp_{\text{sub}}[Q]$ is set equal to $Qp'_{\text{sub}}[CbCr]$ of the transform block containing the sample $q_{\text{sub}}[0][0]$. Otherwise, if cIdx is equal to 1, $Qp_{\text{sub}}[Q]$ is set equal to $Qp'_{\text{sub}}[Cb]$ of the transform block containing the sample $q_{\text{sub}}[0][0]$. Otherwise, $Qp_{\text{sub}}[Q]$ is set equal to $Qp'_{\text{sub}}[Cr]$ of the transform block containing the sample $q_{\text{sub}}[0][0]$. The variable $Qp_{\text{sub}}[C]$ is derived as follows:

7.4.8.1 General Slice Header Semantics

The variable $Qp_{\text{sub}}[C]$ is derived as follows: The luma location (xTb.sub.P, yTb.sub.P) is set as the top-left luma sample position of the transform block containing the sample $p_{\text{sub}}[0][0]$, relative to the top-left luma sample of the picture. If TuCResModel[xTb.sub.P][yTb.sub.P] is equal to 2, $Qp_{\text{sub}}[P]$ is set equal to $Qp'_{\text{sub}}[CbCr]$ of the transform block containing the sample $p_{\text{sub}}[0][0]$. Otherwise, if cIdx is equal to 1, $Qp_{\text{sub}}[P]$ is set equal to $Qp'_{\text{sub}}[Cb]$ of the transform block containing the sample $p_{\text{sub}}[0][0]$. Otherwise, $Qp_{\text{sub}}[P]$ is set equal to $Qp'_{\text{sub}}[Cr]$ of the transform block containing the sample $p_{\text{sub}}[0][0]$. The variable $Qp_{\text{sub}}[Q]$ is derived as follows: The luma location (xTb.sub.Q, yTb.sub.Q) is set as the top-left luma sample position of the transform block containing the sample $q_{\text{sub}}[0][0]$, relative to the top-left luma sample of the picture. If TuCResModel[xTb.sub.Q][yTb.sub.Q] is equal to 2, $Qp_{\text{sub}}[Q]$ is set equal to $Qp'_{\text{sub}}[CbCr]$ of the transform block containing the sample $q_{\text{sub}}[0][0]$. Otherwise, if cIdx is equal to 1, $Qp_{\text{sub}}[Q]$ is set equal to $Qp'_{\text{sub}}[Cb]$ of the transform block containing the sample $q_{\text{sub}}[0][0]$. Otherwise, $Qp_{\text{sub}}[Q]$ is set equal to $Qp'_{\text{sub}}[Cr]$ of the transform block containing the sample $q_{\text{sub}}[0][0]$. The variable $Qp_{\text{sub}}[C]$ is derived as follows:

custom character custom character, custom character, custom character.
8.8.3.6.3 Decision Process for Chroma Block Edges

...
The value of the variable β' is determined as specified in Table 41 based on the quantization parameter Q derived as follows:
[[Q=Clip3(0,63,Qp.sub.C+(slice_beta_offset_div2<<1)) (1322)]] custom character custom character custom character custom character
custom character custom character
where slice_beta_offset_div2 is the value of the syntax element slice_beta_offset_div2 for the slice that contains sample q.sub.0,0.
The variable β is derived as follows:
 $\beta = \beta' * n(1 << (\text{BitDepth} - 8))$ (1323)
The value of the variable t.sub.C' is determined as specified in Table 41 based on the chroma quantization parameter Q derived as follows:
[[Q=Clip3(0,65,Qp.sub.C+2*(bS-1)+(slice_tc_offset_div2<<1)) (1324)]] custom character custom character custom character
custom character custom character custom character
...

5.17 Embodiment #17

(182) This embodiment is based on Embodiment #15.

(183) 7.3.2.4 Picture Parameter Set RBSP Syntax

(184) TABLE-US-00014 if(deblocking_filter_control_present_flag) { deblocking_filter_override_enabled_flag u(1)
pps_deblocking_filter_disabled_flag u(1) if(!pps_deblocking_filter_disabled_flag) { pps_beta_offset_div2 se(v)
pps_tc_offset_div2 se(v) custom character custom character custom character custom character custom character
custom character custom character custom character }

7.3.7.1 General Slice Header Syntax

(185) TABLE-US-00015 if(slice_deblocking_filter_override_flag) { slice_deblocking_filter_disabled_flag u(1) if(
!slice_deblocking_filter_disabled_flag) { slice_beta_offset_div2 se(v) slice_tc_offset_div2 se(v) custom character custom character
custom character custom character custom character custom character custom character custom character } }

7.4.3.4 Picture Parameter Set RBSP Semantics

custom character custom character custom character custom character custom character custom character custom character
custom character custom character custom character custom character custom character custom character custom character
custom character custom character custom character custom character, custom character custom character custom character.
custom character custom character custom character custom character custom character custom character custom character
custom character custom character custom character custom character custom character custom character custom character
custom character custom character, custom character custom character, custom character custom character custom character.

7.4.8.1 General Slice Header Semantics

custom character custom character custom character custom character custom character custom character, custom character
custom character custom character custom character, custom character, custom character, custom character custom character
custom character custom character custom character custom character.
custom character custom character custom character custom character custom character custom character. custom character
custom character custom character, custom character. custom character, custom character custom character custom character
custom character custom character custom character.

8.8.3.6.3 Decision Process for Chroma Block Edges

...
The value of the variable β' is determined as specified in Table 41 based on the quantization parameter Q derived as follows:
[[Q=Clip3(0,63,Qp.sub.C+(slice_beta_offset_div2<<1)) (1322)]] custom character custom character custom character custom character
custom character custom character
where slice_beta_offset_div2 is the value of the syntax element slice_beta_offset_div2 for the slice that contains sample q.sub.0,0.
The variable β is derived as follows:
 $\beta = \beta' * (1 << (\text{BitDepth} - 8))$ (1323)
The value of the variable t.sub.C' is determined as specified in Table 41 based on the chroma quantization parameter Q derived as follows:
[[Q=Clip3(0,65,Qp.sub.C+2*(bS-1)+(slice_tc_offset_div2<<1)) (1324)]] custom character custom character custom character
custom character custom character custom character
...

5.18 Embodiment #18

(186) This embodiment is based on embodiment #17.

(187) 7.4.3.4 Picture Parameter Set RBSP Semantics

(188) custom character custom character custom character custom character custom character custom character. custom character
custom character custom character custom character, custom character. custom character custom character custom character
custom character custom character.

(189) custom character custom character custom character custom character custom character custom character. custom character
custom character custom character custom character, custom character. custom character, custom character custom character
custom character custom character.

(190) 7.4.8.1 General Slice Header Semantics

(191) custom character custom character custom character custom character custom character custom character. custom character
custom character custom character custom character custom character, custom character.

(192) custom character custom character custom character custom character custom character custom character. custom character
custom character custom character custom character, custom character.

(193) 8.8.3.6.1 Decision Process for Luma Block Edges

(194) ...

(195) The value of the variable β' is determined as specified in Table 41 based on the quantization parameter Q derived as follows:
[[Q=Clip3(0,63,qP+(slice_beta_offset_div2<<1)) (1262)]] custom character custom character custom character custom character
where slice_beta_offset_div2 is the value of the syntax element slice_beta_offset_div2 for the slice that contains sample q.sub.0,0.

The variable β is derived as follows:

$\beta = \beta' * (1 << (\text{BitDepth} - 8))$ (1263)

The value of the variable t.sub.C' is determined as specified in Table 41 based on the quantization parameter Q derived as follows:

[[Q=Clip3(0,65,qP+2*(bS-1)+(slice_t_offset_div2<<1)) (1264)]] custom character custom character custom character custom character
...

8.8.3.6.3 Decision Process for Chroma Block Edges

...
The value of the variable β' is determined as specified in Table 41 based on the quantization parameter Q derived as follows:

$$[[Q = \text{Clip3}(0, 63, Q_{p.\text{sub.C}} + (\text{slice_beta_offset_div2} < 1)) \quad (1322)]]$$
where slice_beta_offset_div2 is the value of the syntax element slice_beta_offset_div2 for the slice that contains sample q.sub.0,0.
The variable β is derived as follows:

$$\beta = \beta' * (1 < (\text{BitDepth} - 8)) \quad (1323)$$
The value of the variable t.sub.C' is determined as specified in Table 41 based on the chroma quantization parameter Q derived as follows:

$$[[Q = \text{Clip3}(0, 65, Q_{p.\text{sub.C}} + 2 * (bS - 1) + (\text{slice_tc_offset_div2} < 1)) \quad (1324)]]$$
...

5.19 Embodiment #19

(196) This embodiment is related to the ACT.
(197) intra_bdpcm_chroma_flag equal to 1 specifies that BDPCM is applied to the current chroma coding blocks at the location (x0, y0), i.e. the transform is skipped, the intra chroma prediction mode is specified by intra_bdpcm_chroma_dir_flag. intra_bdpcm_chroma_flag equal to 0 specifies that BDPCM is not applied to the current chroma coding blocks at the location(x0, y0).
When intra_bdpcm_chroma_flag is not present [[it is inferred to be equal to 0.]]
The variable BdpFlag[x][y][cIdx] is set equal to intrabdpcm_chroma_flag for x=x0 . . . x0+cbWidth-1, y=y0 . . . y0+cbHeight-1 and cIdx=1 . . . 2.
intra_bdpcm_chroma_dir_flag equal to 0 specifies that the BDPCM prediction direction is horizontal. intra_bdpcm_chroma_dir_flag equal to 1 specifies that the BDPCM prediction direction is vertical.
The variable BdpDir[x][y][cIdx] is set equal to intra_bdpcm_chroma_dir_flag for x=x0 . . . x0+cbWidth-1, y=y0 . . . y0+cbHeight-1 and cIdx=1 . . . 2.

5.20 Embodiment #20

(198) This embodiment is related to the QP derivation for deblocking.
(199) 8.8.3.6.1 Decision Process for Luma Block Edges
(200) Inputs to this process are:
(201) a picture sample array recPicture, a location (xCb, yCb) specifying the top-left sample of the current coding block relative to the top-left sample of the current picture, a location (xBl, yBl) specifying the top-left sample of the current block relative to the top-left sample of the current coding block, a variable edge Type specifying whether a vertical(EDGE_VER) or a horizontal(EDGE_HOR) edge is filtered, a variable bS specifying the boundary filtering strength, a variable maxFilterLengthP specifying the maximum filter length, a variable maxFilterLengthQ specifying the maximum filter length.
Outputs of this process are: the variables dE, dEp and dEq containing decisions, the modified filter length variables maxFilterLengthP and maxFilterLengthQ, the variable t.sub.C.
[[The variables Qp.sub.Q and Qp.sub.P are set equal to the Qp.sub.Y values of the coding units which include the coding blocks containing the sample q.sub.0,0 and p.sub.0,0 respectively.]]

8.8.3.6.3 Decision Process for Chroma Block Edges

This process is only invoked when ChromaArrayType is not equal to 0.
Inputs to this process are: a chroma picture sample array recPicture, a chroma location (xCb, yCb) specifying the top-left sample of the current chroma coding block relative to the top-left chroma sample of the current picture, a chroma location (xBl, yBl) specifying the top-left sample of the current chroma block relative to the top-left sample of the current chroma coding block, a variable edge Type specifying whether a vertical(EDGE_VER) or a horizontal(EDGE_HOR) edge is filtered, a variable cIdx specifying the colour component index, a variable bS specifying the boundary filtering strength, a variable maxFilterLengthP specifying the maximum filter length, a variable maxFilterLengthQ specifying the maximum filter length.
Outputs of this process are the modified filter length variables maxFilterLengthP and maxFilterLengthQ, the variable t.sub.C.
The variable Qp.sub.P is derived as follows: The luma location (xTb.sub.P, yTb.sub.P) is set as the top-left luma sample position of the transform block containing the sample p.sub.0,0, relative to the top-left luma sample of the picture. If TuCResMode[xTb.sub.P][yTb.sub.P] is equal to 2, Qp.sub.P is set equal to Qp'.sub.CbCr of the transform block containing the sample p.sub.0,0. Otherwise, if cIdx is equal to 1, Qp.sub.P is set equal to Qp'.sub.Cb of the transform block containing the sample p.sub.0,0. Otherwise, Qp.sub.P is set equal to Qp'.sub.Cr of the transform block containing the sample p.sub.0,0.
The variable Qp.sub.Q is derived as follows: The luma location (xTb.sub.Q, yTb.sub.Q) is set as the top-left luma sample position of the transform block containing the sample q.sub.0,0, relative to the top-left luma sample of the picture. If TuCResMode[xTb.sub.Q][yTb.sub.Q] is equal to 2, Qp.sub.Q is set equal to Qp'.sub.CbCr of the transform block containing the sample q.sub.0,0. Otherwise, if cIdx is equal to 1, Qp.sub.Q is set equal to Qp'.sub.Cb of the transform block containing the sample q.sub.0,0. Otherwise, Qp.sub.Q is set equal to Qp'.sub.Cr of the transform block containing the sample q.sub.0,0.
The variable Qp.sub.C is derived as follows:

$$Q_{p.\text{sub.C}} = (Q_{p.\text{sub.Q}} - Q_{p.\text{BdOffset}} + Q_{p.\text{sub.P}} - Q_{p.\text{BdOffset}} + 1) >> 1 \quad (1321)$$

6. Example Implementations of the Disclosed Technology

(202) FIG. 12 is a block diagram of a video processing apparatus 1200. The apparatus 1200 may be used to implement one or more of the methods described herein. The apparatus 1200 may be embodied in a smartphone, tablet, computer, Internet of Things (IoT) receiver, and so on. The apparatus 1200 may include one or more processors 1202, one or more memories 1204 and video processing hardware 1206. The processor(s) 1202 may be configured to implement one or more methods described in the present document. The memory (memories) 1204 may be used for storing data and code used for implementing the methods and techniques described herein. The video processing hardware 1206 may be used to implement, in hardware circuitry, some techniques described in the present document, and may be partly or completely be a part of the processors 1202 (e.g., graphics processor core GPU or other signal processing circuitry).
(203) In the present document, the term “video processing” may refer to video encoding, video decoding, video compression or video

decompression. For example, video compression may be applied during conversion of a video to a corresponding bitstream representation or vice versa. The bitstream representation of a current video block may, for example, correspond to bits that are either co-located or spread in different places within the bitstream, as is defined by the syntax. For example, a macroblock may be encoded in terms of transformed and coded error residual values and also using bits in headers and other fields in the bitstream.

(204) It will be appreciated that the disclosed methods and techniques will benefit video encoder and/or decoder embodiments incorporated within video processing devices such as smartphones, laptops, desktops, and similar devices by allowing the use of the techniques disclosed in the present document.

(205) FIG. 13 is a flowchart for an example method 1300 of video processing. The method 1300 includes, at 1310, performing a conversion between a video unit and a bitstream representation of the video unit, wherein, during the conversion, a deblocking filter is used on boundaries of the video unit such that when a chroma quantization parameter (QP) table is used to derive parameters of the deblocking filter, processing by the chroma QP table is performed on individual chroma QP values.

(206) Some embodiments may be described using the following clause-based format.

(207) 1. A method of video processing, comprising:

(208) performing a conversion between a video unit and a bitstream representation of the video unit, wherein, during the conversion, a deblocking filter is used on boundaries of the video unit such that when a chroma quantization parameter (QP) table is used to derive parameters of the deblocking filter, processing by the chroma QP table is performed on individual chroma QP values.

(209) 2. The method of clause 1, wherein chroma QP offsets are added to the individual chroma QP values subsequent to the processing by the chroma QP table.

(210) 3. The method of any of clauses 1-2, wherein the chroma QP offsets are added to values outputted by the chroma QP table.

(211) 4. The method of any of clauses 1-2, wherein the chroma QP offsets are not considered as input to the chroma QP table.

(212) 5. The method of clause 2, wherein the chroma QP offsets are at a picture-level or at a video unit-level.

(213) 6. A method of video processing, comprising:

(214) performing a conversion between a video unit and a bitstream representation of the video unit, wherein, during the conversion, a deblocking filter is used on boundaries of the video unit such that chroma QP offsets are used in the deblocking filter, wherein the chroma QP offsets are at picture/slice/tile/brick/subpicture level.

(215) 7. The method of clause 6, wherein the chroma QP offsets used in the deblocking filter are associated with a coding method applied on a boundary of the video unit.

(216) 8. The method of clause 7, wherein the coding method is a joint coding of chrominance residuals (JCCR) method.

(217) 9. A method of video processing, comprising:

(218) performing a conversion between a video unit and a bitstream representation of the video unit, wherein, during the conversion, a deblocking filter is used on boundaries of the video unit such that chroma QP offsets are used in the deblocking filter, wherein information pertaining to a same luma coding unit is used in the deblocking filter and for deriving a chroma QP offset.

(219) 10. The method of clause 9, wherein the same luma coding unit covers a corresponding luma sample of a center position of the video unit, wherein the video unit is a chroma coding unit.

(220) 11. The method of clause 9, wherein a scaling process is applied to the video unit, and wherein one or more parameters of the deblocking filter depend at least in part on quantization/dequantization parameters of the scaling process.

(221) 12. The method of clause 11, wherein the quantization/dequantization parameters of the scaling process include the chroma QP offset.

(222) 13. The method of any of clauses 9-12, wherein the luma sample in the video unit is in the P side or Q side.

(223) 14. The method of clause 13, wherein the information pertaining to the same luma coding unit depends on a relative position of the coding unit with respect to the same luma coding unit.

(224) 15. A method of video processing, comprising:

(225) performing a conversion between a video unit and a bitstream representation of the video unit, wherein, during the conversion, a deblocking filter is used on boundaries of the video unit such that chroma QP offsets are used in the deblocking filter, wherein an indication of enabling usage of the chroma QP offsets is signaled in the bitstream representation.

(226) 16. The method of clause 15, wherein the indication is signaled conditionally in response to detecting one or more flags.

(227) 17. The method of clause 16, wherein the one or more flags are related to a JCCR enabling flag or a chroma QP offset enabling flag.

(228) 18. The method of clause 15, wherein the indication is signaled based on a derivation.

(229) 19. A method of video processing, comprising:

(230) performing a conversion between a video unit and a bitstream representation of the video unit, wherein, during the conversion, a deblocking filter is used on boundaries of the video unit such that chroma QP offsets are used in the deblocking filter, wherein the chroma QP offsets used in the deblocking filter are identical of whether JCCR coding method is applied on a boundary of the video unit or a method different from the JCCR coding method is applied on the boundary of the video unit.

(231) 20. A method of video processing, comprising:

(232) performing a conversion between a video unit and a bitstream representation of the video unit, wherein, during the conversion, a deblocking filter is used on boundaries of the video unit such that chroma QP offsets are used in the deblocking filter, wherein a boundary strength (BS) of the deblocking filter is calculated without comparing reference pictures and/or a number of motion vectors (MVs) associated with the video unit at a P side boundary with reference pictures and/or a number of motion vectors (MVs) associated with the video unit at a Q side.

(233) 21. The method of clause 20, wherein the deblocking filter is disabled under one or more conditions.

(234) 22. The method of clause 21, wherein the one or more conditions are associated with: a magnitude of the motion vectors (MVs) or a threshold value.

(235) 23. The method of clause 22, wherein the threshold value is associated with at least one of: i. contents of the video unit, ii. a message signaled in DPS/SPS/VPS/PPS/APS/picture header/slice header/tile group header/Largest coding unit (LCU)/Coding unit (CU)/LCU row/group of LCUs/TU/PU block/Video coding unit, iii. a position of CU/PU/TU/block/Video coding unit, iv. a coded mode of blocks with samples along the boundaries, v. a transform matrix applied to the video units with samples along the boundaries, vi. a shape or dimension of the video unit, vii. an indication of a color format, viii. a coding tree structure, ix. a slice/tile group type and/or picture type, x. a color component, xi. a temporal layer ID, or xii. a profile/level/tier of a standard.

(236) 24. The method of clause 20, wherein different QP offsets are used for TS coded video units and non-TS coded video units.

(237) 25. The method of clause 20, wherein a QP used in a luma filtering step is related to a QP used in a scaling process of a luma block.

(238) 26. A video decoding apparatus comprising a processor configured to implement a method recited in one or more of clauses 1 to 25.

(239) 27. A video encoding apparatus comprising a processor configured to implement a method recited in one or more of clauses 1 to 25.

(240) 28. A computer program product having computer code stored thereon, the code, when executed by a processor, causes the processor to implement a method recited in any of clauses 1 to 25.

(241) 29. A method, apparatus or system described in the present document.

(242) FIG. 18 is a block diagram that illustrates an example video coding system 100 that may utilize the techniques of this disclosure.

(243) As shown in FIG. 18, video coding system 100 may include a source device 110 and a destination device 120. Source device 110 generates

encoded video data which may be referred to as a video encoding device. Destination device **120** may decode the encoded video data generated by source device **110** which may be referred to as a video decoding device.

(244) Source device **110** may include a video source **112**, a video encoder **114**, and an input/output (I/O) interface **116**.

(245) Video source **112** may include a source such as a video capture device, an interface to receive video data from a video content provider, and/or a computer graphics system for generating video data, or a combination of such sources. The video data may comprise one or more pictures. Video encoder **114** encodes the video data from video source **112** to generate a bitstream. The bitstream may include a sequence of bits that form a coded representation of the video data. The bitstream may include coded pictures and associated data. The coded picture is a coded representation of a picture. The associated data may include sequence parameter sets, picture parameter sets, and other syntax structures. I/O interface **116** may include a modulator/demodulator (modem) and/or a transmitter. The encoded video data may be transmitted directly to destination device **120** via I/O interface **116** through network **130a**. The encoded video data may also be stored onto a storage medium/server **130b** for access by destination device **120**.

(246) Destination device **120** may include an I/O interface **126**, a video decoder **124**, and a display device **122**.

(247) I/O interface **126** may include a receiver and/or a modem. I/O interface **126** may acquire encoded video data from the source device **110** or the storage medium/server **130b**. Video decoder **124** may decode the encoded video data. Display device **122** may display the decoded video data to a user. Display device **122** may be integrated with the destination device **120**, or may be external to destination device **120** which be configured to interface with an external display device.

(248) Video encoder **114** and video decoder **124** may operate according to a video compression standard, such as the High Efficiency Video Coding (HEVC) standard, Versatile Video Coding (VVM) standard and other current and/or further standards.

(249) FIG. **19** is a block diagram illustrating an example of video encoder **200**, which may be video encoder **114** in the system **100** illustrated in FIG. **18**.

(250) Video encoder **200** may be configured to perform any or all of the techniques of this disclosure. In the example of FIG. **19**, video encoder **200** includes a plurality of functional components. The techniques described in this disclosure may be shared among the various components of video encoder **200**. In some examples, a processor may be configured to perform any or all of the techniques described in this disclosure.

(251) The functional components of video encoder **200** may include a partition unit **201**, a predication unit **202** which may include a mode select unit **203**, a motion estimation unit **204**, a motion compensation unit **205** and an intra prediction unit **206**, a residual generation unit **207**, a transform unit **208**, a quantization unit **209**, an inverse quantization unit **210**, an inverse transform unit **211**, a reconstruction unit **212**, a buffer **213**, and an entropy encoding unit **214**.

(252) In other examples, video encoder **200** may include more, fewer, or different functional components. In an example, predication unit **202** may include an intra block copy (IBC) unit. The IBC unit may perform predication in an IBC mode in which at least one reference picture is a picture where the current video block is located.

(253) Furthermore, some components, such as motion estimation unit **204** and motion compensation unit **205** may be highly integrated, but are represented in the example of FIG. **19** separately for purposes of explanation.

(254) Partition unit **201** may partition a picture into one or more video blocks. Video encoder **200** and video decoder **300** may support various video block sizes.

(255) Mode select unit **203** may select one of the coding modes, intra or inter, e.g., based on error results, and provide the resulting intra- or inter-coded block to a residual generation unit **207** to generate residual block data and to a reconstruction unit **212** to reconstruct the encoded block for use as a reference picture. In some example, Mode select unit **203** may select a combination of intra and inter predication (CIIP) mode in which the predication is based on an inter predication signal and an intra predication signal. Mode select unit **203** may also select a resolution for a motion vector (e.g., a sub-pixel or integer pixel precision) for the block in the case of inter-predication.

(256) To perform inter prediction on a current video block, motion estimation unit **204** may generate motion information for the current video block by comparing one or more reference frames from buffer **213** to the current video block. Motion compensation unit **205** may determine a predicted video block for the current video block based on the motion information and decoded samples of pictures from buffer **213** other than the picture associated with the current video block.

(257) Motion estimation unit **204** and motion compensation unit **205** may perform different operations for a current video block, for example, depending on whether the current video block is in an I slice, a P slice, or a B slice.

(258) In some examples, motion estimation unit **204** may perform uni-directional prediction for the current video block, and motion estimation unit **204** may search reference pictures of list 0 or list 1 for a reference video block for the current video block. Motion estimation unit **204** may then generate a reference index that indicates the reference picture in list 0 or list 1 that contains the reference video block and a motion vector that indicates a spatial displacement between the current video block and the reference video block. Motion estimation unit **204** may output the reference index, a prediction direction indicator, and the motion vector as the motion information of the current video block. Motion compensation unit **205** may generate the predicted video block of the current block based on the reference video block indicated by the motion information of the current video block.

(259) In other examples, motion estimation unit **204** may perform bi-directional prediction for the current video block, motion estimation unit **204** may search the reference pictures in list 0 for a reference video block for the current video block and may also search the reference pictures in list 1 for another reference video block for the current video block. Motion estimation unit **204** may then generate reference indexes that indicate the reference pictures in list 0 and list 1 containing the reference video blocks and motion vectors that indicate spatial displacements between the reference video blocks and the current video block. Motion estimation unit **204** may output the reference indexes and the motion vectors of the current video block as the motion information of the current video block. Motion compensation unit **205** may generate the predicted video block of the current video block based on the reference video blocks indicated by the motion information of the current video block.

(260) In some examples, motion estimation unit **204** may output a full set of motion information for decoding processing of a decoder.

(261) In some examples, motion estimation unit **204** may do not output a full set of motion information for the current video. Rather, motion estimation unit **204** may signal the motion information of the current video block with reference to the motion information of another video block. For example, motion estimation unit **204** may determine that the motion information of the current video block is sufficiently similar to the motion information of a neighboring video block.

(262) In one example, motion estimation unit **204** may indicate, in a syntax structure associated with the current video block, a value that indicates to the video decoder **300** that the current video block has the same motion information as the another video block.

(263) In another example, motion estimation unit **204** may identify, in a syntax structure associated with the current video block, another video block and a motion vector difference (MVD). The motion vector difference indicates a difference between the motion vector of the current video block and the motion vector of the indicated video block. The video decoder **300** may use the motion vector of the indicated video block and the motion vector difference to determine the motion vector of the current video block.

(264) As discussed above, video encoder **200** may predictively signal the motion vector. Two examples of predictive signaling techniques that may be implemented

(265) by video encoder **200** include advanced motion vector predication (AMVP) and merge mode signaling.

(266) Intra prediction unit **206** may perform intra prediction on the current video block. When intra prediction unit **206** performs intra prediction on the current video block, intra prediction unit **206** may generate prediction data for the current video block based on decoded samples of other video blocks in the same picture. The prediction data for the current video block may include a predicted video block and various syntax elements.

(267) Residual generation unit **207** may generate residual data for the current video block by subtracting (e.g., indicated by a minus sign) the predicted video block(s) of the current video block from the current video block. The residual data of the current video block may include residual video blocks that correspond to different sample components of the samples in the current video block.

(268) In other examples, there may be no residual data for the current video block for the current video block, for example in a skip mode, and residual generation unit **207** may not perform the subtracting operation.

(269) Transform processing unit **208** may generate one or more transform coefficient video blocks for the current video block by applying one or more transforms to a residual video block associated with the current video block.

(270) After transform processing unit **208** generates a transform coefficient video block associated with the current video block, quantization unit **209** may quantize the transform coefficient video block associated with the current video block based on one or more quantization parameter (QP) values associated with the current video block.

(271) Inverse quantization unit **210** and inverse transform unit **211** may apply inverse quantization and inverse transforms to the transform coefficient video block, respectively, to reconstruct a residual video block from the transform coefficient video block. Reconstruction unit **212** may add the reconstructed residual video block to corresponding samples from one or more predicted video blocks generated by the predication unit **202** to produce a reconstructed video block associated with the current block for storage in the buffer **213**.

(272) After reconstruction unit **212** reconstructs the video block, loop filtering operation may be performed reduce video blocking artifacts in the video block.

(273) Entropy encoding unit **214** may receive data from other functional components of the video encoder **200**. When entropy encoding unit **214** receives the data, entropy encoding unit **214** may perform one or more entropy encoding operations to generate entropy encoded data and output a bitstream that includes the entropy encoded data.

(274) FIG. **20** is a block diagram illustrating an example of video decoder **300** which may be video decoder **114** in the system **100** illustrated in FIG. **18**.

(275) The video decoder **300** may be configured to perform any or all of the techniques of this disclosure. In the example of FIG. **20**, the video decoder **300** includes a plurality of functional components. The techniques described in this disclosure may be shared among the various components of the video decoder **300**. In some examples, a processor may be configured to perform any or all of the techniques described in this disclosure.

(276) In the example of FIG. **20**, video decoder **300** includes an entropy decoding unit **301**, a motion compensation unit **302**, an intra prediction unit **303**, an inverse quantization unit **304**, an inverse transformation unit **305**, and a reconstruction unit **306** and a buffer **307**. Video decoder **300** may, in some examples, perform a decoding pass generally reciprocal to the encoding pass described with respect to video encoder **200** (FIG. **19**).

(277) Entropy decoding unit **301** may retrieve an encoded bitstream. The encoded bitstream may include entropy coded video data (e.g., encoded blocks of video data). Entropy decoding unit **301** may decode the entropy coded video data, and from the entropy decoded video data, motion compensation unit **302** may determine motion information including motion vectors, motion vector precision, reference picture list indexes, and other motion information. Motion compensation unit **302** may, for example, determine such information by performing the AMVP and merge mode.

(278) Motion compensation unit **302** may produce motion compensated blocks, possibly performing interpolation based on interpolation filters. Identifiers for interpolation filters to be used with sub-pixel precision may be included in the syntax elements.

(279) Motion compensation unit **302** may use interpolation filters as used by video encoder **20** during encoding of the video block to calculate interpolated values for sub-integer pixels of a reference block. Motion compensation unit **302** may determine the interpolation filters used by video encoder **200** according to received syntax information and use the interpolation filters to produce predictive blocks.

(280) Motion compensation unit **302** may use some of the syntax information to determine sizes of blocks used to encode frame(s) and/or slice(s) of the encoded video sequence, partition information that describes how each macroblock of a picture of the encoded video sequence is partitioned, modes indicating how each partition is encoded, one or more reference frames (and reference frame lists) for each inter-encoded block, and other information to decode the encoded video sequence.

(281) Intra prediction unit **303** may use intra prediction modes for example received in the bitstream to form a prediction block from spatially adjacent blocks. Inverse quantization unit **303** inverse quantizes, i.e., de-quantizes, the quantized video block coefficients provided in the bitstream and decoded by entropy decoding unit **301**. Inverse transform unit **303** applies an inverse transform.

(282) reconstruction unit **306** may sum the residual blocks with the corresponding prediction blocks generated by motion compensation unit **202** or intra-prediction unit **303** to form decoded blocks. If desired, a deblocking filter may also be applied to filter the decoded blocks in order to remove blockiness artifacts. The decoded video blocks are then stored in buffer **307**, which provides reference blocks for subsequent motion compensation/intra predication and also produces decoded video for presentation on a display device.

(283) FIG. **21** is a block diagram showing an example video processing system **2100** in which various techniques disclosed herein may be implemented. Various implementations may include some or all of the components of the system **2100**. The system **2100** may include input **2102** for receiving video content. The video content may be received in a raw or uncompressed format, e.g. 8 or 10 bit multi-component pixel values, or may be in a compressed or encoded format. The input **2102** may represent a network interface, a peripheral bus interface, or a storage interface. Examples of network interface include wired interfaces such as Ethernet, passive optical network (PON), etc. and wireless interfaces such as Wi-Fi or cellular interfaces.

(284) The system **2100** may include a coding component **2104** that may implement the various coding or encoding methods described in the present document. The coding component **2104** may reduce the average bitrate of video from the input **2102** to the output of the coding component **2104** to produce a coded representation of the video. The coding techniques are therefore sometimes called video compression or video transcoding techniques. The output of the coding component **2104** may be either stored, or transmitted via a communication connected, as represented by the component **2106**. The stored or communicated bitstream (or coded) representation of the video received at the input **2102** may be used by the component **2108** for generating pixel values or displayable video that is sent to a display interface **2110**. The process of generating user-viewable video from the bitstream representation is sometimes called video decompression. Furthermore, while certain video processing operations are referred to as “coding” operations or tools, it will be appreciated that the coding tools or operations are used at an encoder and corresponding decoding tools or operations that reverse the results of the coding will be performed by a decoder.

(285) Examples of a peripheral bus interface or a display interface may include universal serial bus (USB) or high definition multimedia interface (HDMI) or Displayport, and so on. Examples of storage interfaces include SATA (serial advanced technology attachment), PCI, IDE interface, and the like. The techniques described in the present document may be embodied in various electronic devices such as mobile phones, laptops, smartphones or other devices that are capable of performing digital data processing and/or video display.

(286) FIG. **22** is a flowchart representation of a method for video processing in accordance with the present technology. The method **2200** includes, at operation **2210**, applying, in a conversion between a video comprising multiple components and a bitstream representation of the video, a deblocking filter to video blocks of the multiple components. A deblocking filter strength for the deblocking filter of each of the multiple components is determined according to a rule that specifies to use a different manner for determining the deblocking filter strength for the video blocks of each of the multiple components.

(287) In some embodiments, the multiple color components comprise at least a Cb component and a Cr component. In some embodiments, each of the multiple color components is associated with deblocking parameter offsets beta and tc including a first syntax element beta_offset_div2 and a second syntax element tc_offset_div2 in a video unit. In some embodiments, the video unit comprises a part corresponding to picture parameter set. In some embodiments, the video unit comprises a part corresponding to picture header. In some embodiments, the video unit further comprises a part

corresponding to slice header. In some embodiments, different syntax elements are applicable to a video block of a color components in case a joint coding of chroma residuals mode is applied to the video block.

(288) FIG. 23 is a flowchart representation of a method for video processing in accordance with the present technology. The method 2300 includes, at operation 2310, performing a conversion between a first video unit of a video and a bitstream representation of the video. During the conversion, a deblocking filtering process is applied to the first video unit. A deblocking control offset of the first video unit is determined based on accumulating one or more deblocking control offset values at other video unit levels.

(289) In some embodiments, the deblocking control offset comprises at least β_offset_div2 or tc_offset_div2 . In some embodiments, the first video unit comprises a slice, and the other video unit levels comprise at least a picture parameter set or a picture.

(290) FIG. 24 is a flowchart representation of a method for video processing in accordance with the present technology. The method 2400 includes, at operation 2410, determining, for a conversion between a block of a video and a bitstream representation of the video, a quantization parameter used in a deblocking process based on usage of a transform skip (TS) mode or an adaptive color transform (ACT) mode for coding the block. The method 2400 also includes, at operation 2420, performing the conversion based on the determining.

(291) In some embodiments, the quantization parameter is determined based on $\text{Max}(\text{QpPrimeTsMin}, \text{qP}) - (\text{cu_act_enabled_flag}[\text{xTbY}][\text{yTbY}]? \text{N}: 0)$, N being a positive integer and qP being a real number. QpPrimeTsMin represents a minimal quantization parameter for blocks encoded in the TS mode, and cu_act_enabled_flag is a flag indicating usage of the ACT mode. In some embodiments, the quantization parameter is determined based on $\text{Max}(\text{QpPrimeTsMin}, \text{qP} - (\text{cu_act_enabled_flag}[\text{xTbY}][\text{yTbY}]? \text{N}: 0))$, N being a positive integer and qP being a real number. QpPrimeTsMin represents a minimal quantization parameter for blocks encoded in the TS mode, and cu_act_enabled_flag is a flag indicating usage of the ACT mode. In some embodiments, qP is equal to a chroma quantization parameter for Cb or Cr component. In some embodiments, N is different for blocks of different color components. In some embodiments, N is equal to 5 in case the block is of Cb, B, G, or U component. In some embodiments, N is equal to 3 in case the block of Cr, R, B or V component.

(292) FIG. 25 is a flowchart representation of a method for video processing in accordance with the present technology. The method 2500 includes, at operation 2510, performing a conversion between a block of a color component of a video and a bitstream representation of the video. The bitstream representation conforms to a rule specifying that a size of a quantization group of the chroma component is greater than a threshold K. The quantization group includes one or more coding units carrying a quantization parameter.

(293) In some embodiments, wherein the size comprises a width of the quantization group. In some embodiments, the color component is a chroma component. In some embodiments, K is 4. In some embodiments, the color component is a luma component. In some embodiments, K is 8.

(294) In some embodiments, the conversion includes encoding the video into the bitstream representation. In some embodiments, the conversion includes decoding the bitstream representation into the video.

(295) Some embodiments of the disclosed technology include making a decision or determination to enable a video processing tool or mode. In an example, when the video processing tool or mode is enabled, the encoder will use or implement the tool or mode in the processing of a block of video, but may not necessarily modify the resulting bitstream based on the usage of the tool or mode. That is, a conversion from the block of video to the bitstream representation of the video will use the video processing tool or mode when it is enabled based on the decision or determination. In another example, when the video processing tool or mode is enabled, the decoder will process the bitstream with the knowledge that the bitstream has been modified based on the video processing tool or mode. That is, a conversion from the bitstream representation of the video to the block of video will be performed using the video processing tool or mode that was enabled based on the decision or determination.

(296) Some embodiments of the disclosed technology include making a decision or determination to disable a video processing tool or mode. In an example, when the video processing tool or mode is disabled, the encoder will not use the tool or mode in the conversion of the block of video to the bitstream representation of the video. In another example, when the video processing tool or mode is disabled, the decoder will process the bitstream with the knowledge that the bitstream has not been modified using the video processing tool or mode that was enabled based on the decision or determination.

(297) The disclosed and other solutions, examples, embodiments, modules and the functional operations described in this document can be implemented in digital electronic circuitry, or in computer software, firmware, or hardware, including the structures disclosed in this document and their structural equivalents, or in combinations of one or more of them. The disclosed and other embodiments can be implemented as one or more computer program products, i.e., one or more modules of computer program instructions encoded on a computer readable medium for execution by, or to control the operation of, data processing apparatus. The computer readable medium can be a machine-readable storage device, a machine-readable storage substrate, a memory device, a composition of matter effecting a machine-readable propagated signal, or a combination of one or more of them. The term “data processing apparatus” encompasses all apparatus, devices, and machines for processing data, including by way of example a programmable processor, a computer, or multiple processors or computers. The apparatus can include, in addition to hardware, code that creates an execution environment for the computer program in question, e.g., code that constitutes processor firmware, a protocol stack, a database management system, an operating system, or a combination of one or more of them. A propagated signal is an artificially generated signal, e.g., a machine-generated electrical, optical, or electromagnetic signal, that is generated to encode information for transmission to suitable receiver apparatus.

(298) A computer program (also known as a program, software, software application, script, or code) can be written in any form of programming language, including compiled or interpreted languages, and it can be deployed in any form, including as a stand-alone program or as a module, component, subroutine, or other unit suitable for use in a computing environment. A computer program does not necessarily correspond to a file in a file system. A program can be stored in a portion of a file that holds other programs or data (e.g., one or more scripts stored in a markup language document), in a single file dedicated to the program in question, or in multiple coordinated files (e.g., files that store one or more modules, subprograms, or portions of code). A computer program can be deployed to be executed on one computer or on multiple computers that are located at one site or distributed across multiple sites and interconnected by a communication network.

(299) The processes and logic flows described in this document can be performed by one or more programmable processors executing one or more computer programs to perform functions by operating on input data and generating output. The processes and logic flows can also be performed by, and apparatus can also be implemented as, special purpose logic circuitry, e.g., an FPGA (field programmable gate array) or an ASIC (application specific integrated circuit).

(300) Processors suitable for the execution of a computer program include, by way of example, both general and special purpose microprocessors, and any one or more processors of any kind of digital computer. Generally, a processor will receive instructions and data from a read only memory or a random-access memory or both. The essential elements of a computer are a processor for performing instructions and one or more memory devices for storing instructions and data. Generally, a computer will also include, or be operatively coupled to receive data from or transfer data to, or both, one or more mass storage devices for storing data, e.g., magnetic, magneto optical disks, or optical disks. However, a computer need not have such devices. Computer readable media suitable for storing computer program instructions and data include all forms of non-volatile memory, media and memory devices, including by way of example semiconductor memory devices, e.g., EPROM, EEPROM, and flash memory devices; magnetic disks, e.g., internal hard disks or removable disks; magneto optical disks; and CD ROM and DVD-ROM disks. The processor and the memory can be supplemented by, or incorporated in, special purpose logic circuitry.

(301) While this patent document contains many specifics, these should not be construed as limitations on the scope of any subject matter or of what may be claimed, but rather as descriptions of features that may be specific to particular embodiments of particular techniques. Certain features that are described in this patent document in the context of separate embodiments can also be implemented in combination in a single embodiment.

Conversely, various features that are described in the context of a single embodiment can also be implemented in multiple embodiments separately or in any suitable subcombination. Moreover, although features may be described above as acting in certain combinations and even initially claimed as such, one or more features from a claimed combination can in some cases be excised from the combination, and the claimed combination may be directed to a subcombination or variation of a subcombination.

(302) Similarly, while operations are depicted in the drawings in a particular order, this should not be understood as requiring that such operations be performed in the particular order shown or in sequential order, or that all illustrated operations be performed, to achieve desirable results. Moreover, the separation of various system components in the embodiments described in this patent document should not be understood as requiring such separation in all embodiments.

(303) Only a few implementations and examples are described and other implementations, enhancements and variations can be made based on what is described and illustrated in this patent document.

Claims

1. A method of processing video data, comprising: applying, in a conversion between a video comprising multiple color components and a bitstream of the video, a deblocking filter to video blocks of the multiple color components; performing the conversion based on the applying, wherein a deblocking filter strength for the deblocking filter of the video blocks of each of the multiple color components is determined according to a rule; wherein the rule specifies to use a different manner for determining the deblocking filter strength for the video blocks of each of the multiple color components; wherein the multiple color components comprise at least a Cb component and a Cr component; and wherein each of the multiple color components is associated with deblocking parameter offsets for a variable beta and a variable tC in different video unit levels which are used to determine the deblocking filter strength; and wherein the different video unit levels comprise a picture parameter set (PPS), wherein a first syntax element associated with the Cb component indicating the deblocking parameter offsets for the variable beta in the PPS is pps_cb_beta_offset_div2 and a first syntax element associated with the Cr component indicating the deblocking parameter offsets for the variable beta in PPS is pps_cr_beta_offset_div2; and wherein a second syntax element associated with the Cb component indicating the deblocking parameter offsets for the variable tC in PPS is pps_cb_tc_offset_div2 and a second syntax element associated with the Cr component indicating the deblocking parameter offsets for the variable tC in the PPS is pps_cr_tc_offset_div2.
2. The method of claim 1, wherein pps_cb_beta_offset_div2 and pps_cb_tc_offset_div2 specify default deblocking parameter offsets for the variable beta and the variable tC (divided by 2) that are applied to the Cb component for slices referring to the PPS, unless the default deblocking parameter offsets are overridden by the deblocking parameter offsets present in slice headers of the slices referring to the PPS; and wherein pps_cr_beta_offset_div2 and pps_cr_tc_offset_div2 specify default deblocking parameter offsets for the variable beta and the variable tC (divided by 2) that are applied to the Cr component for slices referring to the PPS, unless the default deblocking parameter offsets are overridden by the deblocking parameter offsets present in the slice headers of the slices referring to the PPS.
3. The method of claim 1, wherein the different video unit levels comprise a picture header, wherein a third syntax element associated with the Cb component indicating the deblocking parameter offsets for the variable beta in the picture header is pic_cb_beta_offset_div2 and a third syntax element associated with the Cr component indicating the deblocking parameter offsets for the variable beta in the picture header is pic_cr_beta_offset_div2; and a fourth syntax element associated with the Cb component indicating the deblocking parameter offsets for the variable tC in the picture header is pic_cb_tc_offset_div2 and a fourth syntax element associated with the Cr component indicating the deblocking parameter offsets for the variable tC in the picture header is pic_cr_tc_offset_div2.
4. The method of claim 3, wherein pic_cb_beta_offset_div2 and pic_cb_tc_offset_div2 specify deblocking parameter offsets for the variable beta and the variable tC (divided by 2) that are applied to the Cb component for slices associated with the picture header; and wherein pic_cr_beta_offset_div2 and pic_cr_tc_offset_div2 specify deblocking parameter offsets for the variable beta and the variable tC (divided by 2) that are applied to the Cr component for the slices associated with the picture header.
5. The method of claim 1, wherein the different video unit levels comprise a slice header, wherein a fifth syntax element associated with the Cb component indicating the deblocking parameter offsets for the variable beta in the slice header is slice_cb_beta_offset_div2 and a fifth syntax element associated with the Cr component indicating the deblocking parameter offsets for the variable beta in the slice header is slice_cr_beta_offset_div2; and a sixth syntax element associated with the Cb component indicating the deblocking parameter offsets for the variable tC in the slice header is slice_cb_tc_offset_div2 and a sixth syntax element associated with the Cr component indicating the deblocking parameter offsets for the variable tC in the slice header is slice_cr_tc_offset_div2.
6. The method of claim 5, wherein slice_cb_beta_offset_div2 and slice_cb_tc_offset_div2 specify the deblocking parameter offsets for the variable beta and the variable tC (divided by 2) that are applied to the Cb component for a current slice; and wherein slice_cr_beta_offset_div2 and slice_cr_tc_offset_div2 specify the deblocking parameter offsets for the variable beta and the variable tC (divided by 2) that are applied to the Cr component for the current slice.
7. The method of claim 1, wherein the conversion includes encoding the video into the bitstream.
8. The method of claim 1, wherein the conversion includes decoding the video from the bitstream.
9. An apparatus for processing video data comprising a processor and a non-transitory memory with instructions thereon, wherein the instructions upon execution by the processor, cause the processor to: apply, in a conversion between a video comprising multiple color components and a bitstream of the video, a deblocking filter to video blocks of the multiple color components; and perform the conversion based on the applied deblocking filter, wherein a deblocking filter strength for the deblocking filter of the video blocks of each of the multiple color components is determined according to a rule; wherein the rule specifies to use a different manner for determining the deblocking filter strength for the video blocks of each of the multiple color components; wherein the multiple color components comprise at least a Cb component and a Cr component; and wherein each of the multiple color components is associated with deblocking parameter offsets for a variable beta and a variable tC in different video unit levels which are used to determine the deblocking filter strength; and wherein the different video unit levels comprise a picture parameter set (PPS), wherein a first syntax element associated with the Cb component indicating the deblocking parameter offsets for the variable beta in the PPS is pps_cb_beta_offset_div2 and a first syntax element associated with the Cr component indicating the deblocking parameter offsets for the variable beta in the PPS is pps_cr_beta_offset_div2; and a second syntax element associated with the Cb component indicating the deblocking parameter offsets for the variable tC in the PPS is pps_cb_tc_offset_div2 and a second syntax element associated with the Cr component indicating the deblocking parameter offsets for the variable tC in the PPS is pps_cr_tc_offset_div2.
10. The apparatus of claim 9, wherein pps_cb_beta_offset_div2 and pps_cb_tc_offset_div2 specify default deblocking parameter offsets for the variable beta and the variable tC (divided by 2) that are applied to the Cb component for slices referring to the PPS, unless the default deblocking parameter offsets are overridden by the deblocking parameter offsets present in slice headers of the slices referring to the PPS; and wherein pps_cr_beta_offset_div2 and pps_cr_tc_offset_div2 specify default deblocking parameter offsets for the variable beta and the variable tC (divided by 2) that are applied to the Cr component for slices referring to the PPS, unless the default deblocking parameter offsets are overridden by the deblocking parameter offsets present in the slice headers of the slices referring to the PPS.
11. The apparatus of claim 9, wherein the different video unit levels comprise a picture header, wherein a third syntax element associated with the Cb component indicating the deblocking parameter offsets for the variable beta in the picture header is pic_cb_beta_offset_div2 and a third syntax element associated with the Cr component indicating the deblocking parameter offsets for the variable beta in the picture header is

pic_cr_beta_offset_div2; and a fourth syntax element associated with the Cr component indicating the deblocking parameter offsets for the variable tC in the picture header is pic_cb_tc_offset_div2 and a fourth syntax element associated with the Cr component indicating the deblocking parameter offsets for the variable tC in the picture header is pic_cr_tc_offset_div2; wherein pic_cb_beta_offset_div2 and pic_cb_tc_offset_div2 specify deblocking parameter offsets for the variable beta and the variable tC (divided by 2) that are applied to the Cb component for slices associated with the picture header; and wherein pic_cr_beta_offset_div2 and pic_cr_tc_offset_div2 specify deblocking parameter offsets for the variable beta and the variable tC (divided by 2) that are applied to the Cr component for the slices associated with the picture header.

12. The apparatus of claim 9, wherein the different video unit levels comprise a slice header, wherein a fifth syntax element associated with the Cb component indicating the deblocking parameter offsets for the variable beta in the slice header is slice_cb_beta_offset_div2 and a fifth syntax element associated with the Cr component indicating the deblocking parameter offsets for the variable beta in the slice header is slice_cr_beta_offset_div2; and a sixth syntax element associated with the Cb component indicating the deblocking parameter offsets for the variable tC in the slice header is slice_cb_tc_offset_div2 and a sixth syntax element associated with the Cr component indicating the deblocking parameter offsets for the variable tC in the slice header is slice_cr_tc_offset_div2; wherein slice_cb_beta_offset_div2 and slice_cb_tc_offset_div2 specify the deblocking parameter offsets for the variable beta and the variable tC (divided by 2) that are applied to the Cb component for a current slice; and wherein slice_cr_beta_offset_div2 and slice_cr_tc_offset_div2 specify the deblocking parameter offsets for the variable beta and the variable tC (divided by 2) that are applied to the Cr component for the current slice.

13. A non-transitory computer-readable storage medium storing instructions that cause a processor to: apply, in a conversion between a video comprising multiple color components and a bitstream of the video, a deblocking filter to video blocks of the multiple color components; and perform the conversion based on the applied deblocking filter, wherein a deblocking filter strength for the deblocking filter of the video blocks of each of the multiple color components is determined according to a rule; wherein the rule specifies to use a different manner for determining the deblocking filter strength for the video blocks of each of the multiple color components; wherein the multiple color components comprise at least a Cb component and a Cr component; and wherein each of the multiple color components is associated with deblocking parameter offsets for a variable beta and a variable tC in different video unit levels which are used to determine the deblocking filter strength; and wherein the different video unit levels comprise a picture parameter set (PPS), wherein a first syntax element associated with the Cb component indicating the deblocking parameter offsets for the variable beta in the PPS is pps_cb_beta_offset_div2 and a first syntax element associated with the Cr component indicating the deblocking parameter offsets for the variable beta in the PPS is pps_cr_beta_offset_div2; and a second syntax element associated with the Cb component indicating the deblocking parameter offsets for the variable tC in the PPS is pps_cb_tc_offset_div2 and a second syntax element associated with the Cr component indicating the deblocking parameter offsets for the variable tC in the PPS is pps_cr_tc_offset_div2.

14. The non-transitory computer-readable storage medium of claim 13, wherein pps_cb_beta_offset_div2 and pps_cb_tc_offset_div2 specify default deblocking parameter offsets for the variable beta and the variable tC (divided by 2) that are applied to the Cb component for slices referring to the PPS, unless the default deblocking parameter offsets are overridden by the deblocking parameter offsets present in slice headers of the slices referring to the PPS; and wherein pps_cr_beta_offset_div2 and pps_cr_tc_offset_div2 specify default deblocking parameter offsets for the variable beta and the variable tC (divided by 2) that are applied to the Cr component for slices referring to the PPS, unless the default deblocking parameter offsets are overridden by the deblocking parameter offsets present in the slice headers of the slices referring to the PPS.

15. The non-transitory computer-readable storage medium of claim 13, wherein the different video unit levels comprise a picture header, wherein a third syntax element associated with the Cb component indicating the deblocking parameter offsets for the variable beta in the picture header is pic_cb_beta_offset_div2 and a third syntax element associated with the Cr component indicating the deblocking parameter offsets for the variable beta in the picture header is pic_cr_beta_offset_div2; and a fourth syntax element associated with the Cb component indicating the deblocking parameter offsets for the variable tC in the picture header is pic_cb_tc_offset_div2 and a fourth syntax element associated with the Cr component indicating the deblocking parameter offsets for the variable tC in the picture header is pic_cr_tc_offset_div2; wherein pic_cb_beta_offset_div2 and pic_cb_tc_offset_div2 specify deblocking parameter offsets for the variable beta and the variable tC (divided by 2) that are applied to the Cb component for slices associated with the picture header; and wherein pic_cr_beta_offset_div2 and pic_cr_tc_offset_div2 specify deblocking parameter offsets for the variable beta and the variable tC (divided by 2) that are applied to the Cr component for the slices associated with the picture header.

16. The non-transitory computer-readable storage medium of claim 13, wherein the different video unit levels comprise a slice header, wherein a fifth syntax element associated with the Cb component indicating the deblocking parameter offsets for the variable beta in the slice header is slice_cb_beta_offset_div2 and a fifth syntax element associated with the Cr component indicating the deblocking parameter offsets for the variable beta in the slice header is slice_cr_beta_offset_div2; and a sixth syntax element associated with the Cb component indicating the deblocking parameter offsets for the variable tC in the slice header is slice_cb_tc_offset_div2 and a sixth syntax element associated with the Cr component indicating the deblocking parameter offsets for the variable tC in the slice header is slice_cr_tc_offset_div2; wherein slice_cb_beta_offset_div2 and slice_cb_tc_offset_div2 specify the deblocking parameter offsets for the variable beta and the variable tC (divided by 2) that are applied to the Cb component for a current slice; and wherein slice_cr_beta_offset_div2 and slice_cr_tc_offset_div2 specify the deblocking parameter offsets for the variable beta and the variable tC (divided by 2) that are applied to the Cr component for the current slice.

17. A non-transitory computer-readable recording medium storing a bitstream of a video which is generated by a method performed by a video processing apparatus, wherein the method comprises: applying a deblocking filter to video blocks of multiple color components of a video; and generating the bitstream based on the applying, wherein a deblocking filter strength for the deblocking filter of the video blocks of each of the multiple color components is determined according to a rule; wherein the rule specifies to use a different manner for determining the deblocking filter strength for the video blocks of each of the multiple color components; wherein the multiple color components comprise at least a Cb component and a Cr component; and wherein each of the multiple color components is associated with deblocking parameter offsets for a variable beta and a variable tC in different video unit levels which are used to determine the deblocking filter strength; and wherein the different video unit levels comprise a picture parameter set (PPS), wherein a first syntax element associated with the Cb component indicating the deblocking parameter offsets for the variable beta in the PPS is pps_cb_beta_offset_div2 and a first syntax element associated with the Cr component indicating the deblocking parameter offsets for the variable beta in the PPS is pps_cr_beta_offset_div2; and a second syntax element associated with the Cb component indicating the deblocking parameter offsets for the variable tC in the PPS is pps_cb_tc_offset_div2 and a second syntax element associated with the Cr component indicating the deblocking parameter offsets for the variable tC in the PPS is pps_cr_tc_offset_div2.

18. The non-transitory computer-readable recording medium of claim 17, wherein pps_cb_beta_offset_div2 and pps_cb_tc_offset_div2 specify default deblocking parameter offsets for the variable beta and the variable tC (divided by 2) that are applied to the Cb component for slices referring to the PPS, unless the default deblocking parameter offsets are overridden by the deblocking parameter offsets present in slice headers of the slices referring to the PPS; and wherein pps_cr_beta_offset_div2 and pps_cr_tc_offset_div2 specify default deblocking parameter offsets for the variable beta and the variable tC (divided by 2) that are applied to the Cr component for slices referring to the PPS, unless the default deblocking parameter offsets are overridden by the deblocking parameter offsets present in the slice headers of the slices referring to the PPS.

19. The non-transitory computer-readable recording medium of claim 17, wherein the different video unit levels comprise a picture header, wherein a third syntax element associated with the Cb component indicating the deblocking parameter offsets for the variable beta in the picture header is pic_cb_beta_offset_div2 and a third syntax element associated with the Cr component indicating the deblocking parameter offsets for the variable beta in the picture header is pic_cr_beta_offset_div2; and a fourth syntax element associated with the Cb component indicating the deblocking parameter offsets for the variable tC in the picture header is pic_cb_tc_offset_div2 and a fourth syntax element associated with the Cr component

indicating the deblocking parameter offsets for the variable tC in the picture header is `pic_cr_tc_offset_div2`; wherein `pic_cb_beta_offset_div2` and `pic_cb_tc_offset_div2` specify deblocking parameter offsets for the variable beta and the variable tC (divided by 2) that are applied to the Cb component for slices associated with the picture header; wherein `pic_cr_beta_offset_div2` and `pic_cr_tc_offset_div2` specify deblocking parameter offsets for the variable beta and the variable tC (divided by 2) that are applied to the Cr component for the slices associated with the picture header; wherein the different video unit levels comprise a slice header, wherein a fifth syntax element associated with the Cb component indicating the deblocking parameter offsets for the variable beta in the slice header is `slice_cb_beta_offset_div2` and a fifth syntax element associated with the Cr component indicating the deblocking parameter offsets for the variable beta in the slice header is `slice_cr_beta_offset_div2`; and a sixth syntax element associated with the Cb component indicating the deblocking parameter offsets for the variable tC in the slice header is `slice_cb_tc_offset_div2` and a sixth syntax element associated with the Cr component indicating the deblocking parameter offsets for the variable tC in the slice header is `slice_cr_tc_offset_div2`; wherein `slice_cb_beta_offset_div2` and `slice_cb_tc_offset_div2` specify the deblocking parameter offsets for the variable beta and the variable tC (divided by 2) that are applied to the Cb component for a current slice; and wherein `slice_cr_beta_offset_div2` and `slice_cr_tc_offset_div2` specify the deblocking parameter offsets for the variable beta and the variable tC (divided by 2) that are applied to the Cr component for the current slice.
